The Pitch

What to say, out loud, tomorrow

15 July 2026

Contents

1 The pitch — what to say tomorrow

Everything below is written to be **said out loud**. Plain words. Where it says *"say this"*, that is the actual sentence.

Total: about **12 minutes** if he lets you run. He will interrupt. That is fine — every likely interruption has an answer further down.

1.1 Part 0 — Before he sits down (do this in the morning)

```
cd 3
./run.sh rl > /tmp/partB.txt      # 4 minutes. Do NOT run this while he watches.
./run.sh ui                       # leave the browser open on Part A
```

Have open, in this order, ready to switch between:

1. The browser, on **Part A**.
2. `results/plots/collective_behaviour.png`
3. `/tmp/partB.txt`
4. `results/plots/rl_policy_maps.png`
5. `results/plots/rl_model_mismatch.png`
6. `report/main.pdf` — your fallback for any number you forget.

Your name and email are already in `report/main.tex` and the PDF is built. Nothing left to fill in.

1.2 Part 1 — The opening (30 seconds)

Say this first. It makes the two halves look like one assignment instead of two homeworks stapled together.

”The assignment had two parts, so I built two problems, and I put them both in the same University of Dhaka hall. Part A places the hall's Wi-Fi using a swarm. Part B decides when to run the hall's water pump, using an MDP. Same building, two completely different kinds of hard.”

Then stop talking and let him steer. If he says nothing, go to Part A.

1.3 Part 2 — PART A, the swarm (about 4 minutes)

1.3.1 2.1 Set the problem up (30 seconds, no maths)

”The hall floor has 40 rooms and some concrete walls. I can afford 3 access points. Where do I bolt them to the ceiling?”

Then give him the one fact that explains everything:

”An access point reaches about 25 metres through open air. Through one concrete wall, it reaches 14. So a wall costs you 11 metres of range. That's why you can't just put them in the middle.”

1.3.2 2.2 Why a swarm and not calculus (30 seconds — this is the key idea)

This is the sentence that shows you understand the problem rather than just coding it.

”The reason this needs a swarm is the walls. The number of walls between a room and an access point is a whole number — zero walls, or one, or two. It's never one and a half. So when I slide an access point smoothly across the floor, the signal in some room doesn't fade smoothly — it drops 8 decibels in one jump the moment the line of sight crosses a wall.

The objective has a step in it. You can't do calculus on a step. There's no slope to roll down, so gradient descent has nothing to work with. That's why I reached for a population method.”

1.3.3 2.3 Run it live (30 seconds)

In the UI, hit **Run the swarm**. It takes 30 seconds. Running something live is worth a lot — it proves it's real.

Point at the picture:

”Red stars are where the swarm put the access points. The orange lines are the best-known solution moving over time. You can drag this slider back to zero and watch them converge.”

Drag the iteration slider back and forth once. It looks good and it costs nothing.

1.3.4 2.4 THE MOMENT — the communication ablation (90 seconds)

This is the most important 90 seconds of the whole viva. Slow down.

”The assignment asked for a problem where one weak agent fails but many together succeed. I didn't want to just assert that, so I tested it.”

Untick the box that says ”Particles share their global best.” It re-runs by itself — you do not need to press the button.

”I've just set c2 to zero. That's the social term in the velocity update. The thirty particles are still there. They still search. They still remember their own best. They spend exactly the same 3,030 evaluations. The only thing I've taken away is that they can no longer tell each other anything.”

Point at the two numbers on screen:

”The swarm scores 87.2. Random search scores 87.1. They are the same number.

Thirty searchers who don't communicate are worth no more than throwing 3,030 darts at the wall.”

Note on the exact digits. The UI runs ONE seed, so what's on screen will be close to but not identical to the report, which averages 15 or 30 seeds. On screen you'll see roughly 87.2 vs 87.1; in the report it's **87.38 vs 87.43**. If he spots the difference, that's a gift — say: *”The UI is a single run, the report is a mean over 30 seeds. Over 30 seeds it's 87.38 against random's 87.43 — if anything the gap is even smaller than what you're seeing.”*

Tick the box back on. It re-runs by itself again.

”Put one single number back — the global best — and the same thirty agents, at exactly the same cost, jump to 89.9. And they now connect every room in the hall, where random search leaves one room stranded.

So it isn't the population that's doing the work. It's the communication. That's the ant-in-a-bowl point from your briefing, and that's the experiment that proves it.”

If he only remembers one thing from your viva, this is it.

1.3.5 2.5 If he asks ”is that a fair comparison?”

”The budget is identical in every single bar — 3,030 fitness evaluations. And the code asserts it at runtime, it doesn't just trust me: `assert problem.n_fitness_calls == cfg.n_evals`*. If any variant used one evaluation more than another, the program would crash.”*

1.3.6 2.6 Your headline table (if he wants numbers)

	score	rooms connected
PSO	**89.9**	**100%**
Random search	87.4	97.5%
Grid search	89.5	100%

”PSO beats random by 2.47 points, and it's significant — Wilcoxon signed-rank test over 15 seeds, p is 3 times 10 to the minus 5.”

1.4 Part 3 — PART B, decision making (about 5 minutes)

1.4.1 3.1 Set it up (45 seconds, no maths)

”Same hall. There's a water tank on the roof, filled by an electric pump. Three things make this hard.

One — load-shedding. The power dies, and it dies worst in the evening, exactly when everyone wants a shower.

Two — there's a diesel generator, but the hall only buys two hours of diesel a day. Waste it at 7 in the morning and you have nothing at 9 at night.

Three — demand spikes, morning and evening.

So every hour you decide: do nothing, pump on grid power, or burn diesel. And the trick is you have to fill the tank BEFORE the power goes — because once it's gone, it's too late.”

1.4.2 3.2 Prove the problem is worth solving (30 seconds)

He will wonder whether a simple rule would do. Answer it before he asks.

> *"First thing I checked was whether this actually needs planning at all, or whether a simple rule would do.*
>
> *The greedy policy — that's value iteration with gamma set to zero, so it only cares about this hour — loses by 121 percent.*
>
> *And even the best hand-tuned threshold rule, the one a real caretaker would use, where I grid-searched both thresholds to give it its best shot — still loses 14 and a half percent.*
>
> *So the lookahead is doing real work. This isn't a fake problem."*

1.4.3 3.3 Show the policy map (60 seconds)

Open `rl_policy_maps.png`.

First, tell him how to read it, because it is not obvious.

> *"This isn't a timeline. Nothing happens left to right. It's a lookup table.*
>
> *Pick an hour along the bottom. Pick a tank level up the side. The colour tells you what to do in that exact situation. Grey is do nothing, blue is pump on grid power, red is burn diesel.*
>
> *And the left and right panels are two different worlds, both of which exist at every hour. Left is 'suppose the power is ON right now'. Right is 'suppose the power is OUT right now'.*
>
> *The shaded band is just the risky hours — 5 to 10pm, when demand peaks and outages are most likely. It does NOT mean the power is off. That's what the left/right split is for."*

That last sentence matters. Without it, the blue inside the shaded band on the left panel looks like a contradiction — "you said the power is out, why is it pumping on the grid?" It isn't a contradiction. It's the cleverest thing in the figure, and you should say so:

> *"Look at the blue inside the shaded band on the left. That's the agent saying: it's 8pm, the power could go at any moment — but right now it's still on, so pump HARD while I still can. It even tops up a nearly-full tank, which it never bothers doing at 3am. That's it grabbing cheap electricity before it disappears."*

Now point at the **top-right** panel.

> *"Hours across the bottom, tank level up the side. Red means burn diesel.*
>
> *Watch the red region GROW as you go into the shaded evening window. At 7 in the morning with a low tank, it does nothing. At 7 in the evening, with exactly the same tank level, it burns diesel — because it knows the outage is coming and it knows the outage will be long.*
>
> *That's the agent anticipating. That's the whole point."*

If he wants it sharper, you have a witness state:

> *"I can point at one state. At 2pm, with the tank at 4 out of 10, pumping costs 0.97 MORE than doing nothing that hour. The optimal policy pumps anyway, because the action is worth plus 4.69 overall. So more than 5 points of that come purely from the future. The agent takes a certain loss now to hold water it'll need in four hours."*

Then point at the **bottom row**:

> *"That's Q-learning. Same shape, but speckled and messy. That's what 'hasn't seen enough data yet' looks like."*

1.4.4 3.4 THE FINDING — and it went against you (90 seconds)

Lead with the fact that you were wrong. It is the strongest thing you have.

Open `rl_model_mismatch.png`.

> *”Here's where the assignment actually asks 'which algorithm, and when'.*
>
> *Dhaka publishes a load-shedding schedule and it's famously optimistic. So I gave Value Iteration a deliberately WRONG model — I told it outages are rarer than they really are — and then scored the policy it produced in the REAL hall.*
>
> *I fully expected Q-learning to win here. It does not.*
>
> *A planner whose model is wrong by a factor of four still beats Q-learning after 720,000 steps — that's 82 simulated years of running the hall.*
>
> *And then I added the one baseline that could have destroyed my whole thesis. I took the exact same samples Q-learning had already seen, counted them into a model, and planned on that. That beat Q-learning by a factor of ten. Same data.*
>
> *So the honest answer to 'which one, when' is: if you can write the transition structure down at all, write it down and plan. A roughly-right model is worth more than 82 years of experience. Model-free is what you reach for when you CAN'T write the model down — not when the model is merely wrong.”*

Pause here. Let it land. This is a negative result about your own idea, reported honestly, and it is the most impressive thing in the submission.

1.4.5 3.45 The order to show the Part B graphs

You have five. **Show three. Keep two in reserve.**

#	figure	what it is for	when
1	`rl_policy_maps.png`	what the agent *learned*	always
2	`rl_learning_curves.png`	**why Q-learning is slow**	always
3	`rl_model_mismatch.png`	**the finding**	always
4	`rl_vi_convergence.png`	the theory checks out	only if he asks
5	`rl_hyperparams.png`	which knobs matter	only if he asks

1.4.6 The middle one — why Q-learning ”fails”

Open `rl_learning_curves.png`. **Red is Q-learning. Blue is a model learned from Q-learning's own samples.** Both lines are fed *exactly the same data*.

Say the correction first, because it makes you sound precise rather than defensive:

> *”First — it doesn't actually fail. It gets to 15% regret, which beats the greedy policy at 121% and random at 326%. It's sample-INEFFICIENT, not broken.”*

Then the argument, which is airtight:

> *”Now, why is it so slow? The obvious answer would be 'not enough data'. But look at the blue line. That's a model I built from the EXACT SAME 720,000 samples, and it gets to 1.4%. If the problem were not enough data, blue would have failed too. It didn't.*
>
> *So the data was plenty. Q-learning wasted it.*
>
> *Here's how. A Q-learning update takes one sample, nudges ONE cell of the table a little bit, and then throws the sample away forever. There are 3,432 cells.*
>
> *A learned model stores that same sample as a count. And then value iteration re-reads that count in every one of its 2,273 sweeps. So one sample gets used thousands of times, and its consequences spread across the whole table.*

It's the difference between a student who does a practice problem, scribbles the answer on a sticky note and bins the problem — and a student who writes every problem in a notebook and then re-reads the notebook a thousand times working out all the knock-on consequences. Same problems seen. Completely different amount learned."

Point at the arrow on the right of the figure: **11x apart, on identical data.**

If he pushes for the deeper reason, you have two more:

"And gamma makes it worse. Value has to crawl backwards one hop per update. Pumping at 2pm only pays off at 8pm — that's six hops, and each hop needs its own visits. At gamma = 0.99 the horizon is 100 hours, so news travels very slowly.

It's also a theorem, not just my observation. Li et al. 2024 prove vanilla tabular Q-learning is stuck at 1 over (1 minus gamma) to the fourth, while model-based needs 1 over (1 minus gamma) cubed. At gamma = 0.99 that missing factor is a hundred. My Q-learner isn't slow because I coded it badly. It's slow because it mathematically has to be."

The two you keep in reserve

- `rl_vi_convergence.png` — if he asks "how do you know value iteration converged?"
  "The Bellman operator is a gamma-contraction, so the error must shrink by a factor of gamma every sweep. My measured rate is 0.9900. Gamma is 0.99. That's the theory, confirmed to four decimal places."
- `rl_hyperparams.png` — if he asks about tuning.
  "Two things mattered enormously and one didn't. Turning off exploring starts triples the regret — 77% against 25%. A constant learning rate stalls at 50% where the Robbins-Monro schedule reaches 25%, exactly as Even-Dar and Mansour say it must. And epsilon? I tried 0.01, 0.05, 0.2 — barely any difference, inside the seed noise. So I report that too."

That last line is worth saying deliberately. He explicitly asked for honest null results.

1.4.7 3.5 If he asks "so why did you bother with Q-learning?"

"Because that's the answer the experiment gives, and I'd rather report it than the story I went looking for. My problem has 1,584 states and I CAN write the model down — so it's an honest advertisement for Value Iteration and a poor one for Q-learning. Knowing why is the point.

And I'll go further than you were going to: my Q-learner also loses to that two-parameter threshold rule, which does no learning at all. I'd rather say that myself than have you find it."

1.5 Part 4 — What to say about the papers

He asked you to read papers. Here is the honest, strong answer.

1.5.1 The one-liner if he just asks in passing

"Two papers actually changed the project — not just got cited. One corrected a claim I'd already written down, and one explained a result I couldn't explain."

Then give the two.

1.5.2 Paper 1 — Kennedy & Mendes (2002), swarm topology

"My first ablation was just on/off — either the particles share, or they don't. Then I read Kennedy and Mendes, who compare seventy different swarm communication structures. Their finding is that more connectivity converges FASTER but doesn't find BETTER answers.

So I implemented their ring topology, where each particle only hears its two neighbours instead of everybody. And my first write-up said the ring was better.

Then a unit test failed on different seeds, and I tested it properly on 30 paired seeds. The mean difference is NOT significant — Wilcoxon p equals 0.92. But the VARIANCE is — the ring is six times steadier, p equals 6 times 10 to the minus 6. And the fully-connected swarm gives up at iteration 84 while the ring is still improving at 98.

So the ring doesn't find a better answer. It finds a consistent one. Which is exactly what the paper says, and I had it wrong.

And one more thing — the paper does NOT actually recommend the ring. It ranks it 64th out of 70 and recommends a von Neumann lattice instead. I haven't tried that. That's the obvious next experiment."

That last paragraph is gold. It proves you read past the abstract.

1.5.3 Paper 2 — Agarwal et al. (2020) and Li et al. (2024), sample complexity

"I had the measurement — a model learned from Q-learning's own samples beats Q-learning by ten times on identical data — but I had no explanation for WHY.

The explanation turns out to be a proved theorem. Certainty-equivalence planning is minimax optimal, at 1 over (1 minus gamma) cubed. But vanilla tabular Q-learning is provably stuck at 1 over (1 minus gamma) to the FOURTH.

That's a missing factor of 1 over (1 minus gamma). I picked gamma equals 0.99. So that factor is a HUNDRED.

My Q-learner isn't slow because I coded it badly. It's slow because there's a proved sample-complexity gap, and my discount factor makes it a hundred times worse."

And then the bit that will genuinely impress him:

"I nearly cited the wrong paper for this. I was going to cite Azar 2013, and when I actually checked what it proves, it doesn't support my claim at all — its lower bound is information-theoretic and it binds model-free methods equally. So citing it would have been wrong. That's in my notes as a lesson about reading past the abstract."

1.5.4 The other papers, one line each

- **Shi & Eberhart 1998** — *"That's the inertia weight, w decaying from 0.9 to 0.4. That IS my exploration-to-exploitation mechanism, straight from the paper."*
- **Even-Dar & Mansour 2003** — *"They say a constant learning rate can't converge to Q-star, only to a neighbourhood. I didn't take it on faith — I put both in the sweep. The constant alpha stalls at 50% regret where the polynomial one reaches 25%. Their prediction, tested on my problem."*
- **No Free Lunch (Wolpert & Macready 1997)** — *"That's WHY there's an ablation at all. NFL says you can't claim 'PSO is good' — only 'PSO is good on this landscape, and here's the evidence.' So I built the experiment that could have killed my own claim."*

1.5.5 The honest bit — say it, don't hide it

"I'll be straight with you — I also have a twelve-application literature review, and those papers didn't change a single design decision. They're context, not input. If I did it again I'd mine them for encoding tricks."

Volunteering a weakness he hasn't found is worth more than hiding it.

1.6 Part 5 — If he asks about the equations

He probably will. Keep the answers short. **Don't recite the formula — say what it means, then offer the formula.**

1.6.1 ”Explain your fitness function.”

”It's two things subtracted. The first part is what percentage of STUDENTS have signal — weighted by how many people live in each room, not just how many rooms. The second part is a fine for putting two access points too close together — it's zero when they're far enough apart, and it grows quadratically when they're not.

So the swarm learns to spread them out without me ever telling it to.”

1.6.2 ”Why the sigmoid / why not just yes-or-no coverage?”

”Because a yes/no gives the optimiser nothing to follow. A room is either in or out, and nudging an access point 10 centimetres changes nothing until it suddenly flips.

The sigmoid rewards MARGIN — a room with a strong signal scores better than one that barely scrapes past the threshold. That gives the swarm a sense of warmer and colder.”

1.6.3 ”What does gamma actually do?”

”It's how far ahead the agent can see. One over one-minus-gamma. I used 0.99, so that's 100 hours — comfortably more than the 24-hour cycle the whole problem depends on.

If I'd used 0.9 it would only see 10 hours ahead and it would never see the evening outage coming from lunchtime.”

1.6.4 ”Explain the Bellman equation.”

Say it as a sentence, not a formula.

”The best I can do from here equals: pick the action that maximises what I get right now, plus how good the place I land in is, discounted.

It looks circular because V-star is defined in terms of V-star. It is. But if you guess it, plug the guess into the right-hand side, and take the answer as your new guess — you get closer every time. That's value iteration. And the error shrinks by a factor of gamma every sweep, which is why my measured contraction rate is 0.9900 — that's gamma, to four decimal places.”

1.6.5 ”Explain the Q-learning update.”

”I had a guess. Reality gave me a number. Nudge the guess towards reality, a bit.

The bracket is the surprise — what actually happened minus what I expected. If I was right, it's zero and nothing changes.

And notice what's NOT in that formula: there's no P. No probabilities anywhere. It only needs the state, the action, the reward that happened, and where it landed. That's what model-free means.”

1.6.6 ”Why is R(s,a) an expectation?”

This is the sharpest question he can ask, and you have the sharpest answer.

”Because demand is random. Some hours two students shower, some hours five. So the reward is random too, and R(s,a) is the AVERAGE reward.

And that's exactly the model-based/model-free line. Value Iteration works with the average, because it has the probability table and can compute it. Q-learning only ever sees the ONE actual reward that actually happened on that particular hour. It never sees the average, because it never sees the probabilities.

That difference IS the whole assignment, written in one line.”

1.6.7 If you don't know

> *"I don't know — let me look."*

Then open `report/main.pdf` and find it. **Looking something up is not a failure. Inventing a number is.**

1.7 Part 6 — Own your weaknesses before he finds them

Volunteer these. Each one makes you look stronger, not weaker.

- *"My load-shedding model is memoryless — a two-state Markov chain. Real Dhaka load-shedding is SCHEDULED, so an outage that's already run four hours is more likely to end than one that just started. My state can't represent that. If I did it again I'd add time-since-outage to the state."*
- *"My Q-learner loses to a threshold rule that does no learning at all. I'd rather say that than have you extract it."*
- *"I chose the discounted criterion, not average reward. For a continuing operations problem, average cost per hour is arguably more natural. I say so in the report."*
- *"Grid search actually BEAT my PSO at first — because my rooms were on a perfect grid, so I'd handed the discrete method the answer. I added jitter so the optimum sits off the lattice. That was my mistake and it's in the report."*

1.8 Part 7 — If something breaks

- **UI won't start** → all the figures are already in `results/plots/`. Show the PNGs.
- **He wants a number you don't have** → `report/main.pdf`. Open it and find it, out loud.
- **He asks something you can't answer** → *"I don't know. Here's how I'd find out."* Then say how.

1.9 THE CHEAT CARD — print this, keep it in front of you

Part A, one sentence:

> One particle scores 76.8. Thirty that can't talk score 87.4 — the same as random search (87.4). Thirty that share ONE number score 89.9. It's not the population, it's the communication.

Why a swarm:

> Wall count is an integer, so the objective has a STEP in it. No gradient. No calculus.

Part B, one sentence:

> A model that's wrong by 4× still beats Q-learning after 82 simulated years. And a model learned from Q-learning's own samples beats it tenfold. Model-free is for when you CAN'T write the model down — not when it's merely wrong.

Does it need lookahead:

> Greedy loses 121%. Best tuned threshold rule loses 14.5%. Yes.

Papers:

> Kennedy & Mendes corrected me — the ring is steadier (6×), not better ($p=0.92$). Li et al. explained me — Q-learning is provably stuck a factor of $1/(1-\gamma)$ behind. At $\gamma=0.99$ that's 100×.

The numbers:

Part A		Part B	
PSO	89.9	VI optimal	0%
Random	87.4	VI, 4× wrong model	2.7%
No communication	87.4	Learned model (CE)	1.4%
One particle	76.8	Q-learning (720k steps)	15.1%
Grid	89.5	Threshold rule	14.5%
Wilcoxon p	3e-05	Myopic (γ=0)	121%

If stuck: *"I don't know — let me look."* Then open the report.

Start Here

Assignment 3, from zero

15 July 2026

Contents

1 Start here

You are about to be asked to explain this project to someone who knows more than you do. This document gets you from zero to being able to do that. Read it in order. It assumes you remember nothing.

Budget about two hours for the whole thing, including running the code.

1.1 1. What the assignment actually asked for

The teacher set one assignment with **two parts**.

Part A — population-based / swarm. Invent a problem where *one agent is too weak to solve it, but many weak agents working together can*. His words: put a single ant in a bowl and it just wanders. Put a colony together and they find food. Build something with that shape.

Part B — decision making. Invent a problem, model it as a Markov Decision Process, and solve it two ways: **Value Iteration** (which is handed the rules of the world) and **Q-Learning** (which is handed nothing and has to learn by trying things). Then say *in which setting you would use which one, and why*. That last bit is the actual assignment. The coding is the easy part and he said so out loud.

We built two separate problems, both set in a University of Dhaka hall, because he said separate problems are usually more convenient than forcing one problem to do both jobs.

1.2 2. The two problems, in plain language

1.2.1 Part A: where do you put the Wi-Fi routers?

A hall floor has 40 rooms and concrete walls. You can afford **3** access points. Where do you bolt them to the ceiling so that the most students get a usable signal?

You cannot just try every position, because position is a *continuous* thing — an AP can go at x = 12.3m or x = 12.31m or anywhere in between. There are infinitely many options. And you

cannot use calculus to ”roll downhill” to the best answer, because the walls make the signal jump in steps rather than change smoothly, so there is no hill to roll down.

So: **guess 30 layouts at random. Score them. Let them talk to each other and move towards the best one anybody has found. Repeat.** That is Particle Swarm Optimization. Each ”particle” is one complete guess at where all 3 APs go.

1.2.2 Part B: when do you switch on the water pump?

The same hall stores water in a tank on the roof, filled by an electric pump. Three things make this hard:

- **Load-shedding.** The power dies, and it dies *worst in the evening*, exactly when everyone wants a shower.
- **A diesel generator** can run the pump during an outage, but the hall buys only **2 hours of diesel per day**. Waste it in the morning and you have nothing at 9pm.
- **Demand spikes** morning and evening.

Every hour you choose: do nothing, pump using grid power, or burn diesel. The trick is that you should pump *before* the power goes out, filling the tank while electricity is still cheap and available. A rule like ”pump when the tank gets low” fails, because by the time the tank is low, the power is already gone.

1.3 3. The four ideas you need. That is genuinely all of them.

Fitness / reward. A number that says how good something is. In Part A, fitness = what percentage of students get signal. In Part B, reward = negative cost (running the pump costs money, students with no water costs a *lot*). Both algorithms are just machines for making that number bigger.

State. A snapshot of the world that is *enough* to decide what to do next. In Part B the state is `(how full the tank is, what hour it is, is the power on, how much diesel is left)`. That is 11 x 24 x 2 x 3 = **1,584** possible situations. If you know those four things, you do not need to know anything about the past. That property has a name — the **Markov property** — and it is the only reason either algorithm is allowed to work.

Policy. A rule that says, for *every* state, what to do. Not a plan (”pump at 3pm”), a *rule* (”if the tank is below 8 and the power is on, pump”). The output of Part B is a policy.

Model-based vs model-free. This is the whole point of Part B, so slow down here.

> A **model** is the rulebook: ”if the tank is at 5 and I pump, then 80% of the time it goes to 6, 20% of the time to 4,” and so on for every situation. If somebody hands you that rulebook, you can just *calculate* the best policy without ever touching reality. That is **Value Iteration** — it is model-based.
>
> If nobody gives you the rulebook, you have to *try things and see what happens*. That is **Q-Learning** — it is model-free. It pokes the world one hour at a time and slowly figures out what works.
>
> This is a difference in **what information you have**, not in how clever the algorithm is. Remember that sentence. It is the answer to half the questions he will ask.

1.4 4. Run it. Do not skip this.

```
cd 3
pip install -r requirements.txt

./run.sh swarm      # Part A, about 30 seconds
./run.sh rl         # Part B, about 4 minutes
```

Now open `results/plots/` and look at these three, in this order:

`spatial_swarm.png` — the hall floor. Red stars are the final AP positions. The orange lines are the swarm's best-guess *moving* over time. Watch how the APs spread out to get around the red walls. This is the picture that makes people understand Part A instantly.

`collective_behaviour.png` — the most important figure in the project. Every point on it costs the *same* amount of computing. One particle alone scores 76.8. Thirty particles that are forbidden from talking to each other score 87.3 — which is *no better than pure random guessing*. Thirty particles that share one number score 89.9.

> That is the teacher's ant-in-a-bowl point, proven. It is not the *population* that helps. It is the **communication**. If you only remember one result, remember this one.

`rl_policy_maps.png` — the one people misread, so read this first.

It is not a timeline. Nothing happens left to right. It is a **lookup table**: pick an hour (column), pick a tank level (row), and the colour tells you what to do *in that situation*. Grey = do nothing, blue = pump on grid, red = burn diesel.

The left and right panels are two different worlds, and both exist at every hour:

- **Left:** ”suppose the power is **ON** right now — what do I do?”
- **Right:** ”suppose the power is **OUT** right now — what do I do?”

The shaded band is just **the risky hours** (5–10pm, when demand peaks and outages are most likely). It does **not** mean the power is off — that is what the left/right split is for.

Once you know that, two things jump out:

1. **Blue inside the shaded band, on the LEFT.** It's 8pm, the power could go any second, but right now it's still on — so pump *hard* while you can. It even tops up a nearly-full tank, which it never bothers doing at 3am. That is the agent grabbing cheap electricity before it disappears.
2. **The red region GROWS in the shaded band, on the RIGHT.** With the power out at 7am and a low tank it does nothing; at 7pm with the same tank it burns diesel. It is willing to spend its scarce fuel in the evening because it knows the outage will be long.

Both of those are *anticipation*. That is the whole point of Part B.

Then compare the bottom row (Q-learning) to the top row (Value Iteration). Same shape, but speckled and messy. That is what ”hasn't seen enough data yet” looks like.

1.5 5. The results, and the one that surprised us

Part A, everything at an identical budget of 3,030 attempts:

	score
1 particle, alone	76.8
30 particles, not allowed to communicate	87.3
(random guessing, for reference)	*87.4*
30 particles sharing one number	**89.9**

Part B, ranked by how far from perfect each method lands (”regret” — lower is better):

	what it knows	regret
Value Iteration, correct rulebook	everything	**0%**
Value Iteration, rulebook 25% wrong	almost everything	0.1%
Value Iteration on a rulebook *learned from data*	720,000 hours of experience	1.4%
Value Iteration, rulebook **4x** wrong	badly wrong	2.7%
A simple hand-written rule	nothing	14.5%

	what it knows	regret
Q-Learning	720,000 hours of experience, no rulebook	**15.1%**
Doing whatever is cheapest right now	no future	120.6%

Read that table twice. We expected Q-Learning to win, because Dhaka's published load-shedding schedule is famously optimistic, so a planner using it should be badly misled. It did not win. A planner with a *four-times-wrong* rulebook still beat Q-Learning. And a rulebook we *learned from Q-Learning's own data* beat Q-Learning by a factor of ten.

So the honest conclusion is not "model-free is better when the model is wrong." It is:

> **If you can write down the rules of your world at all, do that and plan. A roughly-right model is worth more than 82 simulated years of trial and error. Model-free learning is what you reach for when you cannot write the rules down — not when they are merely wrong.**

That is a *negative* result about our own idea, and it is the strongest thing in the report. If he asks "so why did you bother with Q-Learning?", that paragraph is your answer.

1.6 6. Reading order for the docs

Do them in this order. Do not start with PART1, it will drown you.

0. **`PITCH.md`** — if the viva is *tomorrow*, start there instead. It is the run-of-show: what to say out loud, what to click, and every likely question answered.
1. **This file.** Done.
2. **`statement.md`** — the two problems written down formally, with the maths. Read it *after* you understand them informally, and the symbols will just be labels for things you already know.

   If the maths in it looks like a wall, open `MATH_EXPLAINED.md` beside it. That file takes every equation in `statement.md`, names every symbol in plain English, and then runs the formula with real numbers from our own hall so you can watch it compute. Start with its section 0 — a table of what the symbols are *said out loud as*. Most of what makes a formula frightening is notation, not ideas.
3. **`../report/main.pdf`** — the actual submission. Sections 5 and 6 are Part A and Part B. Read the "Results and Discussion" bits first; skip the literature review on a first pass.
4. **`PART3_viva.md`** — the question bank. This is what you rehearse from. It is long, and it is the single most useful file here the night before.
5. **`PART5_code_defense.md`** — line-by-line: "he points at the screen and asks *this*, you say *that*."
6. **`PART7_literature_applied.md`** — read this if he asks "did you actually read the papers?". It lists, paper by paper, which line of code changed because of it, and names the two papers that caught us claiming something false.
7. **`PART1_foundations.md`** (swarm theory) and **`PART6_decision_making.md`** (MDP theory) — the deep background. Read these only if you want to be able to go one level deeper than the question asked. They are reference material, not a tutorial.
8. **`PART4_problem_design.md`** — the problems we considered and rejected. Useful if he asks "why this problem and not something else?".

1.7 7. The five sentences you must be able to say without thinking

If you can say these five things cold, you will be fine.

1. *"A single particle scores 76.8, thirty particles that cannot communicate score 87.3, which is the same as random search, and thirty that share their best-found solution score 89.9. The population is not what helps — the communication is."*
2. *"The objective is not differentiable, because the signal drops in a step every time it crosses a concrete wall. So there is no gradient to follow, and gradient descent has nothing to work with."*
3. *"Value Iteration is model-based: it is handed P(s'|s,a) and it computes the answer. Q-Learning is model-free: it only ever sees one sampled transition at a time. That is a difference in information access, not in algorithm quality."*
4. *"We proved the problem genuinely needs lookahead: the formally-correct greedy policy — value iteration with gamma set to zero — loses by 121%. And at 2pm with a 4/10 tank, pumping costs 0.97 more that hour, but the optimal policy pumps anyway because it is worth +4.69 overall. Over five of those points come purely from the future."*
5. *"Our headline result is negative. A model that is wrong by a factor of four still beats Q-Learning trained on 720,000 steps, and a model learned from Q-Learning's own samples beats it by ten times. Model-free earns its place when you cannot write the model down at all — not when the model is merely wrong."*

1.8 8. What to do if he asks something you cannot answer

Say so, and say what you would do to find out. Every serious weakness in this project is already written down in the report's limitations section — the outage model is memoryless when real load-shedding is scheduled, we used discounted rather than average reward, and the Assignment 1 heuristic is admissible but loose. Naming your own weakness before he does is worth more than bluffing, and he will know the difference.

The Maths, Decoded

Every equation, with real numbers

15 July 2026

Contents

1 The maths in `statement.md`, decoded

`statement.md` is written the way a professor expects to read it: compressed, symbolic, no hand-holding. That is correct for a report and useless for learning from. This file is the other half. Same equations, but every symbol named, and every formula run with real numbers from our own hall so you can watch it work.

Read this with `statement.md` open beside it.

1.1 0. First: the symbols that are not really maths

Most of what makes a formula look scary is notation, not ideas. Here is the whole vocabulary used in `statement.md`.

Symbol	Say it out loud as	Example
$\sum_i x_i$	"add up all the x_i"	$\sum$ of 2, 5, 3 is 10
$\max(a, b)$	"whichever is bigger"	$\max(0, -4) = 0$
$\max_j \text{RSSI}_{ij}$	"the best one, over all the j's"	best signal room i can hear
$\arg\max_a$	"**which** a gives the biggest value" (not the value — the *choice*)	which action is best

Symbol	Say it out loud as	Example
$\|\mathbf{a}-\mathbf{b}\|$	"the straight-line distance from a to b"	Pythagoras
$x \in S$	"x is one of the things in S"	$3 \in \{1,2,3\}$
$\mathbb{R}^6$	"a list of 6 ordinary numbers"	our 3 APs, x and y each
$\mathbb{E}[X]$	"the **average** value of X"	expected value
$\mathbb{1}[...]$	"1 if that's true, 0 if not"	a switch
$\sigma(z)$	"squash z into a number between 0 and 1"	the S-curve, below
γ (gamma)	"how much I care about the future"	0.99
π (pi)	**not** 3.14 here — it means "policy", a rule	$\pi(s)$ = what to do in state s

That table is genuinely most of it. If a formula still looks bad, it is usually because several of these are stacked, not because it is deep.

---

2 PART A — the Wi-Fi equations

2.1 A.1 Distance

$$d_{ij} = \max\left(\|\mathbf{r}_i - \mathbf{a}_j\|,\ 1\right)$$

In words: how far room i is from access point j, in metres — but never let it be less than 1.

- $\mathbf{r}_i$ = where room i is. $\mathbf{a}_j$ = where AP j is.
- $\|\cdot\|$ = ordinary straight-line distance (Pythagoras).
- **Why the** $\max(\cdot, 1)$**?** The next formula takes $\log_{10}$ of this. And $\log_{10}(0) = -\infty$, which would blow up. So we refuse to go below 1 metre. It is a guard rail, not physics.

2.2 A.2 Signal strength — the important one

$$\text{RSSI}_{ij} \;=\; P_{\text{tx}} \;-\; \left(L_0 + 10\,n \log_{10} d_{ij}\right) \;-\; \gamma\, c_{ij}$$

In words: *how loud the AP shouts, minus how much the air eats, minus how much the walls eat.*

Symbol	Meaning	Our value
RSSI	how strong the signal is when it arrives (dBm; more negative = weaker)	—
P_{tx}	how loud the AP transmits	20 dBm
L_0	how much is lost in the very first metre	40 dB
n	how fast the signal dies with distance	3.3
c_{ij}	**how many walls the signal has to pass through**	0, 1, 2…
γ	how much *each* wall costs you	8 dB

Now run it. An AP transmitting at 20 dBm, and a room 10 m away:

- **no walls:** $20 - (40 + 10 \times 3.3 \times \log_{10} 10) = 20 - (40 + 33) = -\mathbf{53}$ dBm
- **one wall:** $-53 - 8 = -\mathbf{61}$ dBm
- **two walls:** $-61 - 8 = -\mathbf{69}$ dBm

Our "usable signal" threshold is $\tau = -66$ dBm. So the *same room at the same distance* is **fine** with one wall in the way and **dead** with two.

Push it further and you get the fact that the whole problem turns on:

An access point reaches 24.8 m through open air — but only 14.2 m through a single concrete wall. One wall costs you 11 metres of range.

That is why AP placement is hard, and why the answer is not ”put them in the middle”.

And here is the bit that matters for the algorithm. c_{ij} is a *count of walls*. It is 0, or 1, or 2. It is never 1.5. So as you slide an AP smoothly across the floor, the signal in some room drops by **8 dB in one jump** the instant the line of sight crosses a wall. The objective has a **step** in it.

You cannot do calculus on a step. There is no slope to roll down. **That one fact is the entire reason we use a swarm instead of gradient descent**, and if you can say only one thing about Part A's maths, say this.

2.3 A.3 The S-curve (soft coverage)

$$\rho_i = \sigma\left(\frac{\max_j \mathrm{RSSI}_{ij} - \tau}{s}\right), \qquad \sigma(z) = \frac{1}{1+e^{-z}}$$

In words: each room listens to whichever AP it hears best, and we score it between 0 (”no signal”) and 1 (”great signal”).

- $\max_j$ — **each room is served by its strongest AP.** It does not matter that the other two are far away.
- σ (the ”sigmoid” or logistic) is just an S-shaped squasher. Feed it any number, get back something between 0 and 1. $\sigma(0) = 0.5$ exactly. Big positive input → close to 1. Big negative → close to 0.
- $\tau = -66$ dBm is the pass mark; $s = 4$ dB controls how sharp the S is.

Why not just a hard yes/no? Because a yes/no gives the optimiser no gradient to follow at all — a room is either in or out, and nudging the AP by 10 cm changes nothing until it suddenly flips. The S-curve rewards *margin*: a room with a strong signal scores better than a room that barely scrapes past. That gives the swarm a sense of ”warmer / colder”.

Running it:

signal	ρ	reading
−53 dBm (10 m, no wall)	0.963	comfortably covered
−61 dBm (10 m, one wall)	0.777	covered, but not luxurious
−66 dBm (exactly at threshold)	0.500	on the knife edge
−69 dBm (10 m, two walls)	0.321	effectively dead

2.4 A.4 Weighted coverage — the score

$$C(\mathbf{x}) = 100 \cdot \frac{\sum_i w_i \, \rho_i}{\sum_i w_i}$$

In words: the percentage of *students* with signal — not the percentage of rooms.

w_i is how many students live in room i. A 4-person room counts four times as much as a 1-person room. Divide by the total so it lands between 0 and 100.

Tiny example — 3 rooms:

room	students w_i	coverage ρ_i	$w_i\rho_i$
A	4	1.0	4.0
B	1	0.5	0.5
C	2	0.0	0.0
	7		**4.5**

$C = 100 \times 4.5/7 = \mathbf{64.3}\%$. Two of three rooms have some signal, but only 64% of *students* do, because the dead room has two people in it.

2.5 A.5 The objective (what we maximise)

$$F(\mathbf{x}) = C(\mathbf{x}) \;-\; \lambda_{\text{sep}} \sum_{j<k} \big[\max(0,\; d_{\min} - \|\mathbf{a}_j - \mathbf{a}_k\|)\big]^2$$

Scary-looking. It is two things minus each other.

Left half: the coverage score from A.4. We want it big.

Right half: a fine for putting two APs too close together. Read it inside-out:

1. $\|\mathbf{a}_j - \mathbf{a}_k\|$ — how far apart APs j and k are.
2. $d_{\min} -$ (that) — how much *closer than allowed* they are. Our $d_{\min}$ is 10 m.
3. $\max(0, \ldots)$ — **if they're far enough apart, this is zero and there is no fine at all.** The fine only switches on when they're too close.
4. $[\ldots]^2$ — square it, so being *very* close is punished much harder than being slightly close.
5. $\sum_{j<k}$ — do that for every *pair* of APs (the $j < k$ just means don't count the same pair twice).
6. $\lambda_{\text{sep}} = 0.30$ — how much we care.

Running it:

APs are…	breach	fine
12 m apart	0	**0.00** — legal, no penalty
10 m apart	0	**0.00** — exactly on the limit
8 m apart	$2^2 = 4$	1.20
5 m apart	$5^2 = 25$	7.50
stacked on top of each other	$10^2 = 100$	**30.00**

So piling all three APs in one corner costs you 30 points of fitness. The optimiser learns to spread them out without us ever telling it to.

This is called a **soft penalty**: the rule is not physically impossible to break, it is just expensive. Compare with the floor boundary, which we enforce by **repair** — if a particle wanders off the floor, we just shove it back on.

2.6 A.6 The PSO update

$$\mathbf{v} \leftarrow \underbrace{w\mathbf{v}}_{\text{momentum}} + \underbrace{c_1 r_1(\mathbf{p}_{\text{best}} - \mathbf{x})}_{\text{my own best}} + \underbrace{c_2 r_2(\mathbf{g}_{\text{best}} - \mathbf{x})}_{\text{the swarm's best}}, \qquad \mathbf{x} \leftarrow \mathbf{x} + \mathbf{v}$$

In words: *keep going the way I was going, plus pull back towards the best spot I've personally found, plus pull towards the best spot anyone has found.*

- **x** is one particle — **a complete guess at where all 3 APs go** (6 numbers).
- **v** is its velocity: which way it is drifting.
- w is **momentum**, and it shrinks from 0.9 to 0.4 over the run. Early on the particles are careening about (exploring); later they settle (exploiting).
- c_1 pulls it to its own memory; c_2 pulls it to the flock's discovery.
- r_1, r_2 are fresh random numbers each step — the jitter that stops everyone moving in lockstep.

Set $c_2 = 0$ and the third term vanishes. No particle ever hears about anyone else's discovery. That is the ablation, and the result is the whole point of Part A: thirty non-communicating particles score 87.3, which is the same as random guessing.

---

3 PART B — the water-pump equations

3.1 B.1 The state

$$s = (L,\; t,\; g,\; F)$$

Four numbers that tell you everything you need to know right now: **how full the tank is, what hour it is, is the power on, how much diesel is left.**

$11 \times 24 \times 2 \times 3 = \mathbf{1584}$ possible situations.

The reason this list is exactly right (not shorter, not longer) has a name: the **Markov property**. It means *knowing these four things is enough — the past adds nothing*. If you told me the tank is at 6, it's 3pm, the power is on, and there's one hour of diesel left, I do not need to know what happened yesterday to work out what's likely to happen next.

That property is the *only* reason either algorithm is allowed to work. Both of them assume it.

3.2 B.2 One hour of physics

$$\tilde{L} = \min(L + q,\ L_{\max}), \qquad U = \max(0,\ D - \tilde{L}), \qquad L' = \max(0,\ \tilde{L} - D)$$

Read left to right, it is just a story:

- $\tilde{L}$ — **pump first.** Add $q = 3$ units. But the tank only holds $L_{\max} = 10$, so min caps it. Anything above spills.
- U — **then the students draw D units.** If they wanted more than the tank had, U is the water they *didn't get*. If the tank had plenty, $\max(0, \cdot)$ makes this 0.
- L' — what's left in the tank afterwards. Never negative.

$\max(0, \cdot)$ appears twice and both times it means the same thing: *"this can't go below zero"*. You can't have negative unmet demand, and you can't have a negative tank.

3.3 B.3 The reward

$$r = -\Big(\underbrace{\kappa}_{\text{pump cost}} + \underbrace{c_{\text{spill}} O}_{\text{wasted water}} + \underbrace{c_{\text{short}} U}_{\text{thirsty students}} \Big)$$

Everything is negative — it's all cost. The agent's job is to make the total cost as small as possible (i.e. the reward as close to zero as possible).

- grid pump-hour: **1**
- diesel pump-hour: **6** (six times dearer — that's why it's a last resort)
- each unit of water a student wanted and didn't get: **20** ← *by far the worst thing*

That 1-vs-6-vs-20 ratio is the entire personality of the agent. Shortage is so expensive that it will happily burn diesel to avoid it — but diesel is expensive enough that it would much rather have filled the tank earlier on cheap grid power.

3.4 B.4 The subtle one: $R(s, a) = \mathbb{E}_w[r]$

This is the formula in `statement.md` that looks like it's hiding something. It isn't.

$\mathbb{E}[\cdot]$ just means **average**. Demand D is random — some hours 2 students shower, some hours 5. So the reward is random too.

$R(s, a)$ is the **average** reward you'd get if you took action a in state s over and over.

Why it matters: Value Iteration works with the *average* $R(s, a)$ — it has the whole probability table, so it can compute the average directly. Q-learning only ever sees the *actual* reward that actually happened on one particular hour. It never sees the average, because it never sees the probabilities.

That difference **is** model-based versus model-free, written in one line.

3.5 B.5 Discounting — what γ actually does

$$V^{\pi}(s) = \mathbb{E}\Big[\, r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \gamma^3 r_{t+3} + \cdots \Big]$$

In words: the value of being in a state = all the rewards you'll collect from here on, added up — but the further away a reward is, the less it counts *now*.

With $\gamma = 0.99$ and one step = one hour:

a reward this far away…	…is worth this much today
1 hour	0.990
10 hours	0.904
24 hours (a full day)	0.786
100 hours	0.366

The useful rule of thumb: $1/(1-\gamma)$ **is roughly how far ahead the agent can see.** At $\gamma = 0.99$ that's **100 hours** — comfortably more than the 24-hour cycle the whole problem depends on. That is *why* we picked 0.99 and not 0.9 (which would give only 10 hours of foresight, and the agent would never see the evening outage coming from lunchtime).

Set $\gamma = 0$ and the agent only cares about *this hour*. That's the "myopic" baseline — and it loses by 121%.

3.6 B.6 The Bellman optimality equation

$$V^*(s) = \max_a \Big[\underbrace{R(s,a)}_{\text{reward now}} + \gamma \underbrace{\textstyle\sum_{s'} P(s' \mid s,a)\, V^*(s')}_{\text{average value of where I land}} \Big]$$

This is the most important formula in Part B and it says something almost obvious:

> **The best I can do from here = pick the action that maximises (what I get right now + how good the place I land in is, discounted).**

Term by term:

- $\max_a$ — try every action, keep the best.
- $R(s,a)$ — the immediate reward, on average.
- $P(s' \mid s,a)$ — "if I'm in s and do a, what's the chance I end up in s'?" **This is the model.** It is the thing Value Iteration is handed and Q-learning never sees.
- $\sum_{s'} P(\cdot)V^*(s')$ — average the value of all the places I might land, weighted by how likely each is.

It looks circular — V^* is defined in terms of V^*. It is. The trick is that if you just guess V^*, plug it into the right-hand side, and take the answer as your new guess, you get *closer to the truth every single time*. Repeat until it stops changing. That is **Value Iteration**, and the amount you're wrong shrinks by a factor of γ every sweep — which is exactly what our measured contraction rate of **0.9900** is showing.

3.7 B.7 The Q-learning update

$$Q(s,a) \leftarrow Q(s,a) + \alpha \Big[\underbrace{r + \gamma \max_{a'} Q(s',a') - Q(s,a)}_{\text{how surprised I am}} \Big]$$

In words: *I had a guess. Reality gave me a number. Nudge my guess towards reality, a bit.*

- $Q(s,a)$ = my current guess of "how good is doing a in s".
- $r + \gamma \max_{a'} Q(s',a')$ = what actually happened, plus what I reckon the place I landed is worth. Call this "reality's answer".
- The bracket = reality's answer *minus* my guess. This is the **surprise** (its proper name is the TD error). If I was right, it's zero and nothing changes.
- α = how far to move towards reality. Small α = stubborn, learns slowly but steadily. Big α = jumpy, over-reacts to one unlucky hour.

Note what is NOT in this formula: P. No probabilities anywhere. It only needs s, a, the r that happened, and the s' it landed in. That is what "model-free" means — and it is the whole difference from B.6.

One subtlety worth knowing, because it's a good viva question: that $\max_{a'}$ means the update assumes it will act *optimally* from the next state onwards — even though the ε-greedy agent might actually go and do something random. That mismatch is what makes Q-learning **off-policy**, and it's why it can learn the optimal policy while behaving badly the whole time.

3.8 B.8 Regret

$$\text{regret} = 100 \cdot \frac{V^* - V^\pi}{|V^*|}$$

How much worse than perfect, as a percentage. 0% = optimal. 15% = you're losing 15% relative to the best possible.

We can only compute this because our problem is small enough that Value Iteration gives us the *actual* V^* to compare against. On a big problem you'd have no ground truth and would be comparing approximations to approximations.

3.9 The three sentences to take away

1. **Part A's objective has a step in it** (the wall-crossing count is an integer), so it has no gradient, so gradient descent is not an option. Hence a swarm.
2. $P(s' \mid s, a)$ **is the whole story of Part B.** Value Iteration is *given* it and computes the exact answer. Q-learning never sees it and has to feel its way. That is a difference in *what information you have*, not in how clever the algorithm is.
3. **Everything else is bookkeeping** — sums, averages, and "don't go below zero".

Formal Problem Statements

Assignment 3, Parts A and B

15 July 2026

Contents

1 Formal Problem Statements — Assignment 3

> **New to this, or the notation is hard going?** Read `docs/MATH_EXPLAINED.md` alongside this file. It decodes every equation below symbol by symbol, in plain English, and runs each formula with real numbers from our own hall so you can see it work. This file is deliberately written the compressed way a report expects; that one is written the way you actually learn it.

The assignment has two parts, and we chose two separate problems rather than forcing one problem to serve both. They share a setting: a University of Dhaka residential hall. Part A places its Wi-Fi. Part B runs its water pump.

- **Part A** — population-based search. A continuous, non-differentiable, constrained max-coverage problem, solved with a particle swarm optimiser.
- **Part B** — decision making under uncertainty. A finite discounted Markov decision process, solved exactly with Value Iteration and approximately with Q-learning.

---

2 Part A — Wi-Fi Access-Point Placement

2.1 Maximum Weighted Coverage on a University Residential-Hall Floor

2.1.1 1. Informal description

A University of Dhaka residential hall has a fixed, tight budget of K Wi-Fi access points (APs) to mount on one floor. Rooms far from an AP, or shadowed by concrete partitions, receive unusable signal. We must choose the **continuous positions of the K APs on the floor plan** so as to **maximise the occupancy-weighted fraction of rooms that receive a usable link**, subject to a co-channel-interference separation rule and the physical boundary of the floor.

This is a **continuous, constrained, single-objective max-coverage facility-location problem**.

2.1.2 2. Given data (problem instance)

Symbol	Meaning
W, H	floor width and height (metres); floor domain $\Omega = [0, W] \times [0, H]$
N	number of rooms (demand points)
$\mathbf{r}_i = (r_i^x, r_i^y) \in \Omega$	position of room i, $i = 1, \dots, N$
$w_i > 0$	occupancy weight of room i (students)
$\mathcal{W}$	set of interior wall segments (line segments in Ω)
P_{tx}	AP transmit power (dBm)
L_0	reference path loss at $d_0 = 1$ m (dB)
n	log-distance path-loss exponent
γ	attenuation per wall crossing (dB)
τ	usable-link RSSI threshold (dBm)
s	logistic softness around τ (dB)
K	number of APs to place (budget)
$d_{\min}$	minimum allowed separation between two APs (m)
λ_{sep}	separation-penalty weight

2.1.3 3. Decision variables

The positions of the K APs, stacked into one vector:

$$\mathbf{x} = (\mathbf{a}_1, \dots, \mathbf{a}_K) = (a_1^x, a_1^y, \dots, a_K^x, a_K^y) \in \mathbb{R}^{2K}, \qquad \mathbf{a}_j = (a_j^x, a_j^y) \in \Omega.$$

The search space is the box $\mathcal{S} = \Omega^K \subset \mathbb{R}^{2K}$.

2.1.4 4. Signal and coverage model

Distance between room i and AP j (floored at $d_0 = 1$ m, the model's validity radius):

$$d_{ij}(\mathbf{x}) = \max\left(\|\mathbf{r}_i - \mathbf{a}_j\|_2,\ 1\right).$$

Received signal strength (log-distance indoor path loss with discrete wall attenuation):

$$\text{RSSI}_{ij}(\mathbf{x}) = P_{\text{tx}} - \left(L_0 + 10\, n \log_{10} d_{ij}(\mathbf{x})\right) - \gamma\, c_{ij}(\mathbf{x}),$$

where $c_{ij}(\mathbf{x}) \in \mathbb{Z}_{\geq 0}$ is the number of wall segments in $\mathcal{W}$ intersected by the segment $\overline{\mathbf{r}_i\, \mathbf{a}_j}$.

Best-AP soft coverage of room i (each room is served by its strongest AP; a logistic gives a smooth threshold rewarding signal *margin*):

$$\rho_i(\mathbf{x}) = \sigma\left(\frac{\max_{j=1,\dots,K} \text{RSSI}_{ij}(\mathbf{x}) - \tau}{s}\right) \in (0, 1), \qquad \sigma(z) = \frac{1}{1 + e^{-z}}.$$

Weighted coverage (the quantity to maximise, as a percentage):

$$C(\mathbf{x}) = 100 \cdot \frac{\sum_{i=1}^{N} w_i\, \rho_i(\mathbf{x})}{\sum_{i=1}^{N} w_i} \ \in [0, 100].$$

2.1.5 5. Optimization problem

$$\max_{\mathbf{x} \in \mathbb{R}^{2K}} \quad F(\mathbf{x}) = C(\mathbf{x}) \ - \ \lambda_{\text{sep}} \sum_{1 \leq j < k \leq K} \left[\max\left(0,\ d_{\min} - \|\mathbf{a}_j - \mathbf{a}_k\|_2\right)\right]^2$$

$$\text{subject to} \qquad 0 \leq a_j^x \leq W, \quad 0 \leq a_j^y \leq H, \qquad j = 1, \dots, K.$$

- The **box constraints** (AP inside the floor) are enforced by **repair** (clamping) during optimization.
- The **separation constraint** ($\|\mathbf{a}_j - \mathbf{a}_k\| \geq d_{\min}$, co-channel interference) is a soft **quadratic penalty** — the second term of F — which is 0 when feasible and grows with the breach.
- The **cardinality constraint** (exactly K APs) is structural: it is baked into the dimension $2K$ of $\mathbf{x}$ and can never be violated.

2.1.6 6. Why the problem is hard (landscape characterisation)

F is:

- **non-differentiable** — it contains $\max_j(\cdot)$ (a kink), a logistic threshold, and the **integer wall-crossing counts** c_{ij}, which make F piecewise-constant in $\mathbf{x}$ (so $\nabla F = 0$ a.e. and undefined on the kinks);
- **non-convex and multimodal** — many spatially distinct layouts yield similar coverage (which AP serves which region is interchangeable), producing many local optima and broad symmetric basins;
- **black-box in structure** — cheap to evaluate pointwise but with no closed-form inverse; only sampling is available.

Hence gradient-based methods have no usable gradient, and exact solvers require discretising Ω — an NP-hard max-coverage selection that explodes combinatorially and discards the continuous optimum. A **population-based metaheuristic (Particle Swarm Optimization)** is the appropriate solver: each particle is a full candidate layout in $\mathbb{R}^{2K}$, and the swarm searches the floor collectively.

2.1.7 7. Reference instance (as implemented)

$W = 60,\ H = 40$ m; $\quad N = 40$ rooms on a jittered 10×4 grid, $w_i \in \{1, 2, 3, 4\}$;
$|\mathcal{W}| = 3$ concrete partitions; $\quad K = 3 \ \Rightarrow\ \mathcal{S} = \mathbb{R}^6$;
$P_{\text{tx}} = 20$ dBm, $\ L_0 = 40$ dB, $\ n = 3.3,\ \gamma = 8$ dB, $\ \tau = -66$ dBm, $\ s = 4$ dB;
$d_{\min} = 10$ m, $\ \lambda_{\text{sep}} = 0.30$.

Solution quality metric. Alongside the soft objective F, we report **hard coverage** — the unweighted percentage of rooms whose best-AP RSSI meets the threshold, $\frac{100}{N}\sum_i \mathbb{1}[\max_j \text{RSSI}_{ij} \geq \tau]$ — as the deployment-facing measure of how many rooms are actually connected.

---

3 Part B — Pump Control under Load-Shedding

3.1 Minimum-Cost Water Supply for a Residential Hall, as a Markov Decision Process

3.1.1 1. Informal description

The same hall keeps its water in an overhead tank, refilled by an electric pump from an underground reservoir. Three things turn this from plumbing into a decision problem:

1. **The grid fails**, and it fails hardest in the evening, exactly when the hall wants water. Evening outages are also long: once the power goes it tends to stay gone.
2. **A diesel generator** can drive the pump through an outage, but the hall buys a **fixed ration of diesel each morning**. Burn it at 7am on a hunch and there is none left at 9pm.
3. **Demand is not flat.** It spikes when the hall bathes, morning and evening.

Every hour the caretaker picks one action. Pumping early on cheap grid power banks water against an outage that has not happened yet; pumping diesel early spends a ration that cannot be recovered. This is a **finite, discounted, continuing Markov decision process**.

3.1.2 2. Given data (problem instance)

Symbol	Meaning	Value used
$L_{\max}$	tank capacity, in units of 100 L	10
q	pump throughput per hour (units)	3
$D_{\max}$	demand truncation	6
$F_{\max}$	diesel ration delivered each morning (generator-hours)	2
c_{grid}	cost of one grid pump-hour	1.0
c_{gen}	cost of one diesel pump-hour	6.0
c_{short}	cost per unit of water demanded and not delivered	20.0
c_{spill}	cost per unit overflowed	0.5
λ_t	mean hourly demand (truncated Poisson)	peaks 2.5 at 06–09 and 18–22
p_t^{fail}	$P(\text{grid drops} \mid \text{grid up})$	0.02 night / 0.10 day / **0.35 evening**
p_t^{rest}	$P(\text{grid returns} \mid \text{grid down})$	0.50 night / 0.35 day / **0.15 evening**
γ	discount factor	0.99

The evening figures are the heart of the instance: failure is likely *and* restoration is slow, so an evening outage lasts $1/0.15 \approx 6.7$ hours in expectation.

3.1.3 3. State, actions, and availability

$$s = (L,\ t,\ g,\ F) \in \mathcal{S}, \qquad L \in \{0, \dots, L_{\max}\},\ \ t \in \{0, \dots, 23\},\ \ g \in \{0, 1\},\ \ F \in \{0, \dots, F_{\max}\}$$

so $|\mathcal{S}| = 11 \times 24 \times 2 \times 3 = 1584$. The hour wraps, which makes this a **continuing** task, not an episodic one.

$$\mathcal{A} = \{\mathsf{idle},\ \mathsf{pump}_{\text{grid}},\ \mathsf{pump}_{\text{diesel}}\}$$

Actions are **state-dependent**:

$$\mathcal{A}(s) = \{\mathsf{idle}\} \ \cup\ \{\mathsf{pump}_{\text{grid}} : g = 1\} \ \cup\ \{\mathsf{pump}_{\text{diesel}} : F > 0\}$$

Pumping on grid during an outage is not an action, it is a dead switch; offering it would make it an exact duplicate of `idle` in all **792** outage states ($11 \times 24 \times 3$), which wastes a third of the exploration budget on a provable no-op and reduces any policy-agreement metric to a coin flip. Masking leaves $4752 - 792 - 528 = 3432$ legal (s, a) pairs. It cannot change V^* — a duplicate action cannot change a max — so it costs nothing in optimality and buys sample efficiency and honest metrics.

3.1.4 4. Dynamics

Let the inflow and pump cost be

$$q(s,a) = \begin{cases} q & a = \mathsf{pump}_{\text{grid}},\ g = 1 \\ q & a = \mathsf{pump}_{\text{diesel}},\ F > 0 \\ 0 & \text{otherwise} \end{cases} \qquad \kappa(s,a) = \begin{cases} c_{\text{grid}} & a = \mathsf{pump}_{\text{grid}},\ g = 1 \\ c_{\text{gen}} & a = \mathsf{pump}_{\text{diesel}},\ F > 0 \\ 0 & \text{otherwise.} \end{cases}$$

The pump acts, then the students draw. Write $\tilde{L} = \min(L + q(s,a),\ L_{\max})$ for the post-pump level and $O = \max(0,\ L + q(s,a) - L_{\max})$ for the spill. With demand $D \sim \text{Poisson}(\lambda_t)$ truncated to $\{0, \dots, D_{\max}\}$ and **renormalised** (not clipped), the unmet demand and next level are

$$U = \max(0,\ D - \tilde{L}), \qquad L' = \max(0,\ \tilde{L} - D).$$

The grid follows a two-state Markov chain $g' \sim P(\cdot \mid g, t)$, independent of D. The diesel decrements on use and is **redelivered each midnight**:

$$F' = \begin{cases} F_{\max} & t = 23 \quad \text{(the morning drum arrives)} \\ F - \mathbb{1}[a = \mathsf{pump}_{\text{diesel}}] & \text{otherwise,} \end{cases} \qquad t' = (t+1) \bmod 24.$$

3.1.5 5. Reward, and a subtlety that matters

$$r(s,a,w) = -\Big(\kappa(s,a)\ +\ c_{\text{spill}}\, O\ +\ c_{\text{short}}\, U\Big), \qquad w = (D,\ g').$$

The realised shortage U depends on the realised demand D, and D **is not recoverable from** s' — once the tank hits zero, $L' = 0$ tells you nothing about how much demand went unmet. So r is **not** a function of (s, a, s').

This is not a defect. It is the standard *disturbance* form of an MDP. Value Iteration only ever needs the **expected** reward,

$$R(s,a) = \mathbb{E}_w\big[r(s,a,w)\big] = -\Big(\kappa(s,a) + c_{\text{spill}} O + c_{\text{short}} \textstyle\sum_d \Pr[D = d] \max(0, d - \tilde{L})\Big),$$

which we tabulate, and the Bellman operator remains a γ-contraction. Q-learning consumes the **realised** r. It must: feeding it $\mathbb{E}[U]$ would leak the demand distribution into a supposedly model-free agent and collapse the entire distinction the assignment exists to study.

3.1.6 6. The optimization problem

Find a stationary policy $\pi : \mathcal{S} \to \mathcal{A}$ with $\pi(s) \in \mathcal{A}(s)$ maximising

$$\boxed{V^\pi(s) = \mathbb{E}\Big[\textstyle\sum_{k=0}^{\infty} \gamma^k\, r_{t+k} \;\Big|\; s_t = s,\ \pi\Big]}$$

The optimum satisfies the Bellman optimality equation

$$V^*(s) = \max_{a \in \mathcal{A}(s)} \Big[R(s,a) + \gamma \textstyle\sum_{s'} P(s' \mid s, a)\, V^*(s')\Big].$$

Choice of criterion. We use discounted return, not average reward. At one hour per step, $\gamma = 0.99$ gives an effective horizon $1/(1-\gamma) = 100$ hours, comfortably longer than the 24-hour cycle the problem turns on. For a continuing operations problem average cost per hour is arguably more natural, and the discounted-optimal policy is not guaranteed to be gain-optimal; we report discounted return consistently everywhere and never mix the two axes.

3.1.7 7. Why both algorithms belong here

$P(s' \mid s, a)$ can be written down, so **Value Iteration** applies and returns the exact optimum. That is what makes it *model-based*: the sum over s' is only computable because somebody handed us P.

Q-learning is given only a simulator it can poke — one (s', r) at a time, no probabilities. That is what *model-free* means.

The scientific question is therefore not "which one wins" (with a correct model VI wins by construction, it is exact). It is: **how wrong must the model be, and how much experience must you buy, before learning beats planning?** Dhaka publishes a load-shedding schedule that is famously optimistic, so this is not hypothetical.

We also run the arm that can falsify the obvious thesis: estimate $\hat{P}$ from Q-learning's *own* samples and plan on that (certainty equivalence). If a learned model beats the learner on identical data, then "model-free wins when the model is wrong" is the wrong lesson.

3.1.8 8. Reporting metrics

Every policy — Value Iteration's, Q-learning's, certainty-equivalence's, and the baselines' — is scored the same way: by **exactly solving** $(I - \gamma P_\pi)V^\pi = R_\pi$ under the **true** model. This removes Monte-Carlo noise from the comparison completely. Policies are ranked by exact expected return, not by which one drew a luckier rollout.

We report **regret**, $100 \cdot (V^* - V^\pi)/|V^*|$, plus rollout statistics (cost per day, shortage units per day, diesel hours per day) purely to translate abstract return into consequences a hall warden would recognise.

Companion documents: alternatives considered in `docs/PART4_problem_design.md`; viva preparation in `docs/PART3_viva.md`; solvers, baselines and results in `src/pso_wifi_placement.py`, `src/rl_water_tank.py`, `src/rl_experiments.py`, and the README.

Assignment 3 - Overview

Results, layout, how to run it

15 July 2026

Contents

1 Assignment 3 — Population-Based Search and Decision Making

CSEDU AI Lab. This assignment has **two parts**, and we chose two separate problems rather than bending one problem to fit both. They share a setting: a University of Dhaka residential hall.

	Part A	Part B
Unit	Population-based / swarm	Decision making
Problem	Where do you put 3 Wi-Fi access points on a hall floor?	When do you run the water pump, given Dhaka load-shedding?
Method	Particle Swarm Optimization, from scratch	Value Iteration + Q-learning, from scratch
Code	`src/pso_wifi_placement.py`	`src/rl_water_tank.py`, `src/rl_experiments.py`

The final report is `report/main.pdf` (18 pages, built from the instructor's template). It also covers Assignments 1 and 2 from `../1` and `../2`.

1.1 The two headline results

Part A — the collective is doing the work, not the compute. Everything below runs at an identical 3,030 fitness-evaluation budget:

	mean fitness
1 particle, all 3,030 evaluations to itself	76.78 ± 6.66
30 particles that never communicate (`c2 = 0`)	87.30 ± 1.15
(Random Search, for reference)	*87.43 ± 0.43*
30 particles sharing one `gbest`	**89.91 ± 0.26**

Thirty independent searchers that do not talk are worth no more than throwing 3,030 darts. Restore a single shared number and the same thirty agents, at the same cost, gain 2.47 points (Wilcoxon p = 3.05e-05) and connect every room in the hall instead of stranding one.

But *more* communication is not better, which is a result we took from Kennedy & Mendes (2002) and then had to be corrected by:

topology (same 30 particles, same budget)	mean	std	stagnates
fully connected (`gbest`)	89.83	0.317	iter **84**
ring, k=1	89.96	**0.129**	iter **98**

Over 30 paired seeds the mean difference is **not significant** (Wilcoxon $p = 0.92$) but the variance is — the ring is **6× steadier** (F-test p = 6e-06) and is still improving when the fully-connected swarm has already quit. The ring does not find a *better* answer; it finds a *consistent* one. That is exactly what the paper predicts, and our first draft got it wrong by claiming ”better”.

Part B — we set out to prove something and the data said no. The plan was to show that Q-learning beats a planner working from Dhaka's optimistic published load-shedding schedule. It does not. Ranked by regret against the exact optimum:

	what it knows	regret
Value Iteration, correct model	the exact P	0% (0.23 s)
Value Iteration, **mildly** wrong model	P off by 1.25×	0.1%
Certainty-equivalence VI	$\hat{P}$ estimated from Q-learning's own 720k samples	**1.4% ± 0.2**
Value Iteration, **badly** wrong model	P off by 4×	2.7%
Tuned caretaker threshold rule	nothing; two tuned numbers	14.5%
Q-learning	720,000 samples, no model	15.1% ± 2.4
Value Iteration, believes the grid never fails	no outage model at all	21.4%
Myopic greedy (γ=0)	the model, but no future	120.6%

Read that table twice. **A roughly-right model is worth more than 82 simulated years of experience** — a planner who is merely 25% off still beats the model we *learned* from 720,000 samples. Even a fourfold error beats Q-learning outright. And our Q-learner also loses to a two-parameter threshold rule that does no learning at all.

So the real argument for model-free control is not ”your model might be wrong” — it can be quite wrong and still win. It is that sometimes you cannot write a model down at all. Ours has 1,584 states and we could, which makes this an honest advertisement for Value Iteration and a poor one for Q-learning. Knowing *why* is the point.

(An earlier draft claimed the learned model beat every mis-specified prior. The table we printed directly underneath it said otherwise. The corrected claim is the more interesting one, and the script now computes its conclusion instead of asserting it.)

1.2 Running it

```
pip install -r requirements.txt

./run.sh swarm          # Part A: PSO + Random/Grid baselines + the collective ablation
./run.sh rl             # Part B: Value Iteration vs Q-learning     (~4 min)
./run.sh rl --quick     # Part B, fewer seeds and episodes          (~40 s)
./run.sh test           # 42 unit tests
./run.sh report         # compile report/main.pdf  (the submission)
./run.sh pdfs           # convert every doc in docs/ to PDF -> pdf/
```

Figures land in `results/plots/`, tables in `results/tables/`.

Only `numpy`, `scipy`, `matplotlib` and `pandas`. No DEAP, no PySwarm, no Gym, no stable-baselines. Every algorithm is written from scratch, because the point of a lab is to be able to defend every line.

1.3 Layout

```
3/
├── report/
│   ├── main.tex               # the report, in the instructor's template
│   ├── main.pdf               # 18 pages, covers Assignments 1, 2 and 3
│   ├── references.bib         # all citations verified against Crossref
│   └── figures/
├── src/
│   ├── pso_wifi_placement.py  # Part A: PSO, baselines, collective ablation
│   ├── rl_water_tank.py      # Part B: the MDP, VI, Q-learning, certainty equivalence
│   └── rl_experiments.py      # Part B: experiments, tables, figures
├── tests/                    # 42 tests, including the model-vs-simulator parity gate
├── statement.md               # formal problem statements for both parts
├── docs/
│   ├── START_HERE.md          # <-- read this first if you are new to the project
│   ├── MATH_EXPLAINED.md    # <-- every equation in statement.md, decoded with real numbers
│       ├──  PART1_foundations.md      #  swarm  theory:  NFL,  explore/exploit,  8-
algorithm comparison
│   ├── PART2_applications.md  # 12 cited real-world applications
│   ├── PART3_viva.md          # viva prep, both parts
│   ├── PART4_problem_design.md# the problems we considered and rejected, both parts
│   ├── PART5_code_defense.md  # code-to-theory map + "he points at the screen" questions
│   ├── PART6_decision_making.md # MDP theory: Bellman, contraction, Q-learning convergence
│   └── PART7_literature_applied.md # which papers changed which line, and the 2 that corrected us
├── pdf/                  # every doc above as a PDF, + ALL_DOCS.pdf (95 pages, combined)
├── scripts/build_pdfs.sh      # regenerates pdf/  ->  ./run.sh pdfs
└── results/
    ├── plots/
    ├── tables/
    └── rl_full_run.log
```

The PDFs in `pdf/` are the docs, for reading offline or printing. They are **not** the submission — `report/main.pdf` is.

1.4 Two things worth knowing before you read the code

The room jitter in Part A is not cosmetic. Our first instance had rooms on a perfect rectangular grid, and Grid Search *beat* PSO (89.19 vs 88.41). That result was correct and it was our fault: if the demand points are perfectly gridded, grid-aligned AP positions are exactly where the optimum lives, and we had handed the discrete method the answer. Real rooms are not perfectly gridded. We added positional jitter ($\sigma = 1.5$ m) so the optimum sits *off* any lattice, which is both more realistic and the whole reason to reach for a continuous optimiser.

The diesel ration in Part B is not decoration either. In our first version the generator was unlimited, and mis-estimating the outage rate cost the planner almost nothing — it could always buy its way out one hour later. That made the central experiment measure approximately nothing. Making the diesel a finite daily drum introduces irreversibility: burn it early on a bad forecast and you have none left when the real outage lands. Only then does model error actually hurt.

Both of these were failures first. They are in the report because they are the sharpest things we learned.

Viva Preparation

Question bank, both parts

15 July 2026

Contents

1 Part 3 — Viva & Presentation Preparation

Everything here is tied to the actual project: a from-scratch **Particle Swarm Optimizer** placing **K = 3 Wi-Fi access points** on a **60 m × 40 m University of Dhaka hall floor** (40 occupancy-weighted rooms, concrete partitions) to maximise weighted coverage. Headline result: PSO **89.9 %** weighted coverage vs Random Search **87.4 %** at an identical 3030-evaluation budget, Wilcoxon **$p = 3.05 \times 10^{-5}$**; PSO also beats an exhaustive coarse Grid Search.

1.1 3.1 Sixty-second spoken opening (memorise this)

> ”My project optimises Wi-Fi in a University of Dhaka hall. Administration has a tight budget of three access points for a floor, and where you mount them decides whether the far and wall-shadowed rooms get usable signal. I framed it as a continuous max-coverage problem: a solution is the (x, y) coordinates of the three routers, and the objective is the occupancy-weighted percentage of rooms whose strongest signal clears a usability threshold, using a real log-distance path-loss model with concrete-wall attenuation, minus a penalty for placing routers too close together.
>
> I solve it with a Particle Swarm Optimizer I wrote from scratch — no libraries — because the objective is non-differentiable and multimodal: it has a max over access points, a threshold, and discrete wall-crossings, so there is no usable gradient, and an exact solver would explode over a continuous search space. In PSO each particle *is* a candidate layout, so the swarm literally searches the floor.
>
> Against Random Search and Grid Search at the exact same evaluation budget, my PSO improves weighted coverage by 2.47 points over random (2.83% relative) and is statistically significant by a Wilcoxon signed-rank test, with much lower run-to-run variance. And because a particle is a coordinate, I can show the swarm converging on the floor plan itself.”

(Timing: ~55–60 s at a calm pace. If cut short, keep sentences 1, 3, and 5.)

1.2 3.2 Question & Answer bank

1.2.1 (a) Conceptual & Theoretical

Q. What is a population-based metaheuristic, in one sentence? A stochastic optimizer that maintains a *set* of candidate solutions and iteratively improves them by biased random variation plus selection, sampling the objective rather than differentiating it.

Q. Why a *population* rather than a single point? A population samples many basins simultaneously, shares information (in PSO, via `gbest`), and resists getting trapped — one particle escaping a local optimum can pull the swarm. A single hill-climber sees only its neighbourhood.

Q. What is the exploration–exploitation trade-off here? Exploration = roaming the floor for new promising regions; exploitation = refining near the best-known layout. Too much exploration never converges; too much exploitation converges prematurely to a mediocre spot. I balance them by decaying the inertia weight `w` from 0.9 to 0.4 over the run.

Q. State the No Free Lunch theorem and its relevance. Wolpert & Macready (1997): averaged over *all* possible objective functions, no optimizer beats any other. Implication: PSO is not universally best — it is a good match *for this landscape* (non-differentiable, multimodal, continuous). My job is to justify the match, not to claim PSO is "the best algorithm."

Q. Define convergence for a stochastic optimizer. Convergence in probability: $P(\|\mathbf{g}_{\text{best}}^{(t)} - \mathbf{x}^*\| > \epsilon) \to 0$ as $t \to \infty$. In practice I track the monotone `gbest` history and the iteration after which it no longer improves.

Q. What makes a fitness landscape "hard"? Multimodality (many local optima), ruggedness (low fitness autocorrelation between neighbours), neutrality (flat plateaus giving no gradient), and deceptiveness (structure that lures search *away* from the optimum). My objective is multimodal and rugged (wall kinks), which is exactly where swarms shine.

Q. Is your problem convex? No. `max_j RSSI` is a pointwise maximum (can create non-convex ridges), the soft threshold is sigmoidal, and wall-crossings inject discontinuities. A convex problem would not need PSO — I'd use a convex solver.

1.2.2 (b) Algorithm-Specific Mechanics

Q. Write the PSO update and define every symbol. $\mathbf{v} \leftarrow w\mathbf{v} + c_1 r_1(\mathbf{p}_{\text{best}} - \mathbf{x}) + c_2 r_2(\mathbf{g}_{\text{best}} - \mathbf{x})$; $\mathbf{x} \leftarrow \mathbf{x} + \mathbf{v}$. `x` = position (a layout), `v` = velocity, `w` = inertia, `c1,c2` = cognitive/social coefficients, `r1,r2` ~ U(0,1) per-dimension random, `pbest` = particle's own best, `gbest` = swarm best.

Q. What do the cognitive and social terms do? Cognitive $c_1 r_1(\mathbf{p}_{\text{best}} - \mathbf{x})$ pulls a particle toward its *own* best (individual memory / exploitation of personal experience); social $c_2 r_2(\mathbf{g}_{\text{best}} - \mathbf{x})$ pulls it toward the *swarm's* best (information sharing). Their random weights keep the search from being deterministic.

Q. Why the velocity clamp `v_max`? Without it velocities can grow unboundedly ("swarm explosion") and particles overshoot the floor every step. Clamping to 20 % of each axis range keeps steps physically meaningful and the swarm stable.

Q. How do you handle a particle leaving the floor? Repair: clamp the position back into $[0, W] \times [0, H]$ and zero the velocity component that hit the wall (an "absorbing wall"), so it doesn't grind along the boundary.

Q. PSO vs GA for this problem? Both work. PSO is preferred because the variables are continuous coordinates — PSO moves them natively, whereas a GA needs a real-valued encoding and crossover that respects geometry. PSO also gives the striking spatial visualisation.

Q. PSO vs ACO here? ACO is for discrete/graph construction (routing). This is continuous placement, so ACO would need artificial discretisation — the wrong tool.

Q. What are `pbest` and `gbest` initialised to? `pbest` = each particle's initial position; `gbest` = the best of the initial random swarm. Both are updated only on strict improvement (elitist memory).

1.2.3 (c) Problem Design & Hyperparameter Justifications

Q. Why K = 3 access points? It models a realistically *tight* hall budget and makes placement matter — three APs cannot blanket a 60×40 block, so a poor layout visibly strands rooms. It also keeps the search space small ($\mathbb{R}^6$) and the demo legible.

Q. Justify your radio model constants. Standard 2.4 GHz indoor values: transmit power `P_tx = 20 dBm` (100 mW EIRP), reference loss L_0 `= 40 dB` at 1 m, path-loss exponent `n = 3.3` (dense indoor), per-wall loss γ `= 8 dB` (concrete partition), usability threshold τ `= −66 dBm`. These aren't tuned for the result; they're literature-typical.

Q. How did you choose swarm size and iterations? `P = 30`, `T = 100`. Swarm 20–50 is the standard PSO range; 30 balances per-iteration cost against diversity in $\mathbb{R}^6$. `T = 100` was chosen because the `gbest` reliably stagnates well before then (around iteration 79 in the traced run, 74-90 across the 15 seeds), so the budget is sufficient without waste — verified empirically, not guessed.

Q. Why `c1 = c2 = 1.49445` and `w: 0.9→0.4`? This is the well-established stable configuration (Shi–Eberhart inertia + Clerc-consistent coefficients). Equal `c1,c2` gives no a-priori bias between personal and social learning; decaying `w` schedules exploration→exploitation.

Q. What does the separation penalty encode and how did you weight it? It encodes the co-channel interference rule (APs on the same channel shouldn't sit within `d_min = 10 m`). Weight `λ_sep = 0.30` is small enough that coverage dominates but large enough to break ties toward physically deployable layouts.

Q. Why weight rooms by occupancy? A router should prioritise a 4-student room over a store-room. Weighting makes the objective reflect *people served*, not just rooms counted.

1.2.4 (d) Professor "Trap" Questions — see §3.3 for the bulletproof set

Q. Your fitness is ~90 %, not 100 %. Is that a failure? No — distinguish two coverages. The **90 %** is the *soft, occupancy-weighted* objective, which rewards signal **margin** above the usability threshold (a room at −55 dBm scores higher than one barely at −66 dBm) — it is the optimum under a tight 3-AP budget. The **hard** coverage (rooms with any usable link) is a separate report: PSO reaches **100 %** — it connects *every* room — while Random Search reaches only **97.5 %**, i.e., it strands a room. So PSO wins twice: it connects the last stranded room *and* gives the connected rooms more headroom (better throughput and resilience to fading/interference). Grid Search also hits 100 % hard coverage but lower soft margin (89.5) — its optimum is stuck on the grid; PSO's is not.

Q. Isn't this just a facility-location problem people solve exactly? Only the *discretised* version is exactly solvable (and it's NP-hard as max-coverage). The continuous, wall-attenuated, weighted variant has no tractable exact form — hence a metaheuristic.

1.3 3.3 The seven classic trap questions — bulletproof answers

1. Why a computationally expensive swarm/EA instead of an exact solver or gradient descent?

- The objective is **non-differentiable** (max-over-APs, a threshold, discrete wall-crossings) — gradient descent has no reliable gradient; finite differences are corrupted by the kinks.
- It is **non-convex and multimodal** — a local method converges to whatever basin it starts in.
- An **exact solver** must discretise the continuous floor; the resulting max-coverage selection is NP-hard and explodes, and discretisation discards the continuous optimum. PSO searches the continuous space directly at a fixed, modest budget. *When would I NOT use PSO? If the objective were convex and smooth — then a gradient or convex solver would win. It isn't.*

2. How can you prove your implementation avoids premature convergence and local-optima traps?

- **Empirically:** I track a **diversity metric** (mean particle distance to the swarm centroid) — `diversity.png` shows it starting high (broad exploration) and decaying smoothly, not collapsing instantly (which would signal premature convergence).
- **Statistically:** across 15 seeds PSO's best-fitness **std is 0.26** — it finds essentially the same high-quality optimum every time, i.e., it is not getting stuck in seed-dependent local traps.
- **Mechanistically:** decaying inertia + fresh `r1,r2` each step + velocity clamping keep the swarm exploring early. I can add random immigration if a run ever stagnated early, but it doesn't.

3. What was your methodology for tuning control parameters, and why is relying on default library settings an academic failure?

- I did not use any library, so there are no hidden defaults to hide behind — every constant is in the `Config` dataclass and defended (§3.2c).
- Radio constants are **literature-typical**, not fitted to flatter the result.
- PSO constants are the **published stable regime** (Shi–Eberhart); `T` was set from the **observed stagnation iteration**, not guessed.
- Relying on library defaults is a failure because defaults are tuned for other problems (NFL), are often undocumented, and cannot be defended — you cannot explain a number you did not choose.

4. How does your code mathematically penalise or handle boundary and structural constraint violations?

- **Boundary (box) constraint:** *repair* — `np.clip` positions into the floor and zero the offending velocity component (absorbing wall).
- **Separation (inequality) constraint:** *penalty* — $\lambda_{\text{sep}} \sum_{j<k} [\max(0, d_{\min} - \|\mathbf{a}_j - \mathbf{a}_k\|)]^2$, a quadratic penalty that is zero when feasible and grows with the breach.
- **Cardinality (exactly K APs):** *structural* — encoded as the fixed vector dimension $2K$, so it can never be violated.

5. How do you prove the statistical significance of your results over the baseline? A one-sided **Wilcoxon signed-rank test** (paired by seed) on the 15 PSO-vs-Random best-fitness pairs, `H₁: PSO > Random`, giving **p = 3.05 × 10⁻⁵ ≪ 0.05**. I use Wilcoxon (non-parametric) because I don't assume the fitness distribution is normal. For comparing *three or more* algorithms I would use the **Friedman test** followed by a post-hoc (e.g., Nemenyi).

6. What stopping criteria did you establish and why?

- **Max iterations** `T = 100` as the hard budget cap.
- **Stagnation report:** I compute the last iteration with an improvement `eps = 1e-4`; it lands around iteration 79 in the traced run (74-90 across the 15 seeds), confirming `T` is sufficient and the swarm has plateaued (not still climbing at the buzzer).
- I deliberately run the *full* budget for all methods rather than early-stopping, so the baseline comparison stays budget-matched (see #7).

7. How do you ensure a perfectly fair baseline comparison? By matching the **number of fitness evaluations**, not iterations or wall-clock. PSO uses $P(T+1) = 3030$ evaluations; Random Search draws exactly 3030 samples; Grid Search enumerates $\binom{25}{3} = 2300$ combinations (it *cannot* use more without exceeding the budget — itself a finding). An `assert` in the code checks PSO's evaluation count equals the budget, so the fairness is enforced, not assumed.

---

2 Part 3 (continued) — Decision Making (Assignment 3, Part B)

Everything below is tied to the actual code: a **University of Dhaka hall water tank** modelled as a finite discounted **MDP** (`src/rl_water_tank.py`) and solved twice — once by **Value Iteration** with the transition table handed to it, once by **Q-learning** with nothing but a simulator doorway. |S| = **1584**, **3432** legal state-action pairs, γ = **0.99**. Headline: VI is exact (**−163.637**), a tuned reactive threshold rule loses **14.51 %**, the myopic greedy policy loses **120.62 %**, Q-learning

after **720,000** environment steps sits at **15.13 % ± 2.36**, and certainty-equivalence VI on Q-learning's *own samples* lands at **1.43 % ± 0.23**.

2.1 3.4 Sixty-second spoken opening for Part B (memorise this)

> "Part B is the same hall, one utility down: the water tank. An electric pump fills an overhead tank, students draw from it, and the grid goes down — hardest, and for longest, exactly in the evening hours when the hall wants water. There is a diesel generator, but the hall buys a fixed ration of two generator-hours each morning, and unused fuel does not roll over. Every hour the caretaker picks one of three things: leave the pump off, pump on grid power, or burn diesel. Pumping early on cheap grid power banks water against an outage that has not happened yet. Burning diesel early spends a ration you cannot get back. That is the whole game.
>
> I modelled it as a Markov Decision Process. The state is tank level, hour of day, is-the-grid-up, and diesel left — 1584 states, 3432 legal state-action pairs. The clock is in the state deliberately: without it, demand and outage rates are non-stationary and the process is not Markov at all.
>
> I solved it twice. Value Iteration is handed the transition table and computes the exact optimum — 2273 sweeps, a quarter of a second, and the Bellman residual decays at exactly 0.9900, which is gamma, precisely as the contraction theorem says it must. Q-learning is handed only a doorway: put in a state and an action, get back where you landed and what it cost. It never sees a probability.
>
> The result I want to defend is not 'which algorithm wins'. It is this: the myopic policy loses 121 percent and the best tuned threshold rule still loses 14.5 percent, so the lookahead is doing real work. And when I hand the planner a deliberately wrong outage model, it still beats my Q-learner — right up until the model claims the grid never fails. I ran the one baseline that could sink my own thesis, certainty-equivalence, and it did change what I am allowed to claim."

(Timing: ~60 s. If cut short, keep paragraphs 1 and 4.)

2.2 3.5 Question & Answer bank — Part B

2.2.1 (a) Conceptual & Theoretical

Q. What makes this an MDP? Name every component. A finite, stationary, *continuing* discounted MDP (S, A, P, R, γ). **States** $s = (L, t, g, F)$: tank level 0–10 (100 L units), hour 0–23, grid up/down, diesel-hours left 0–2 — $11 \times 24 \times 2 \times 3 = 1584$. **Actions** $A(s) \subseteq \{\text{IDLE}, \text{PUMP\_GRID}, \text{PUMP\_GEN}\}$, state-*dependent* — 3432 legal pairs of 4752. **Transitions** $P(s' \mid s, a)$, built explicitly in `build_model()` as a sparse $(S \cdot A) \times S$ matrix. **Reward** $R(s, a) = -\mathbb{E}[\text{pump cost} + 0.5 \cdot \text{overflow} + 20 \cdot \text{unmet}]$. **Discount** $\gamma = 0.99$. There is no terminal state — the hour wraps at 23, which is why the 24-hour "episode" is a reporting unit, not an absorbing boundary.

Q. State the Markov property and prove it holds for your state. $\Pr(s_{t+1}, r_t \mid s_t, a_t, s_{t-1}, a_{t-1}, \dots) = \Pr(s_{t+1}, r_t \mid s_t, a_t)$ — the future is conditionally independent of the past given the present. It holds by construction: the demand D_t is drawn from a truncated Poisson whose mean depends only on t, and t is *in the state*; the grid flips per a two-state Markov chain whose $p_{\text{fail}}, p_{\text{restore}}$ depend only on (g, t), both in the state; the fuel update is a deterministic function of $(t, \text{fuel left})$. Nothing outside (L, t, g, F) is consulted. You can read this straight off `simulate_hour` — it decodes s, draws two independent uniforms, and returns.

Q. Why is the hour of day in the state? Isn't that just bookkeeping? It is the load-bearing design decision. Drop t and the process on (L, g, F) is **not Markov** — demand and outage probabilities become non-stationary and unpredictable from the remaining state. Putting the clock in the state is what *makes* it Markov. It costs a 24× blow-up in $|S|$ and it is what lets the optimal policy anticipate the 18:00–22:00 outage window at all.

Q. Model-based vs model-free — draw the line exactly where your code draws it. The `HallWaterMDP` class has two faces and they never touch. `build_model()` returns the explicit P and R; Value Iteration consumes them and **computes** the answer — it never takes a sample. `step()` returns one (s', r) and no probability ever leaves that function; Q-learning is only allowed to call `step()`. That narrow doorway *is* the difference between the two algorithms in this report. Model-based means "I can evaluate $\sum_{s'} P(s' \mid s, a)V(s')$"; model-free means "I can only average over things that actually happened to me".

Q. Why discounted, and what does γ = 0.99 mean at one hour per step? γ is an **operational horizon**, not a psychological one. $1/(1-\gamma) = 100$ hours of effective lookahead — comfortably more than the 24-hour cycle the entire story depends on, so the agent at 14:00 can still see the evening outage window. Mathematically, discounting is what makes the Bellman operator a contraction and guarantees a bounded, unique V^* on a continuing task. The honest concession: the discounted-optimal policy is **not guaranteed to be gain-optimal** (average-reward optimal). I say so rather than pretending the two criteria coincide, and I never draw the discounted V^* line on an average-cost axis — `rollout_stats` reports cost/day and shortage/day separately, as translation, not as the optimisation target.

Q. Bellman optimality vs Bellman expectation — what actually differs? The **expectation** equation is *linear* in V for a fixed policy: $V^\pi = R_\pi + \gamma P_\pi V^\pi \Rightarrow (I - \gamma P_\pi)V^\pi = R_\pi$. So I do not iterate it — `policy_evaluation()` **solves** it with a sparse linear solve. $(I - \gamma P_\pi)$ is invertible for any $\gamma < 1$ because γP_π has spectral radius $\leq \gamma < 1$. The **optimality** equation carries a max: $V^*(s) = \max_{a \in A(s)}[R(s,a) + \gamma \sum_{s'} P(s' \mid s, a)V^*(s')]$ — nonlinear, no closed form, so it must be iterated (or turned into an LP). Both operators are γ-contractions in $\|\cdot\|_\infty$; only one of them is linear. That is exactly why VI exists and why policy evaluation does not need to.

2.2.2 (b) Algorithm Mechanics

Q. Why does Value Iteration converge? Give the argument, not the vibe. The Bellman optimality operator T is a γ-contraction in the sup norm: $\|TV - TU\|_\infty \leq \gamma \|V - U\|_\infty$. Two facts do the work — max is non-expansive ($|\max_a x_a - \max_a y_a| \leq \max_a |x_a - y_a|$), and each row of P is a probability distribution, so averaging cannot increase a sup norm; the γ out front does the shrinking. Banach's fixed-point theorem then gives a **unique** V^* and **geometric** convergence from *any* initialisation. My run: **2273 sweeps** to tol 10^{-9}, **0.23 s**.

Q. Your residual decays at exactly 0.9900. Contraction only gives you "≤ γ". Why is it tight? Right — the theorem is an upper bound, so measuring the *bound* is not the same as measuring the *rate*. The rate is tight for a specific reason. The greedy policy stops changing long before sweep 2273; after that, VI is a **linear** iteration $V_{k+1} = R_{\pi^*} + \gamma P_{\pi^*} V_k$, so the error $e_k = V_k - V^*$ obeys $e_{k+1} = \gamma P_{\pi^*} e_k$. The asymptotic rate is therefore $\gamma \cdot \rho(P_{\pi^*}) = \gamma \cdot 1 = \gamma$, because P_{π^*} is row-stochastic and its dominant eigenvalue is exactly 1. Measured: **0.9900**, which is γ to four decimals. This makes it a genuine unit test on the maths: a rate *above* γ would mean a bug in the backup; a rate meaningfully *below* it would mean the induced chain is degenerate. I also cross-check the answer independently — $\|V^* - V^{\pi^*}\|_\infty = 1.81 \times 10^{-8}$ against the exact linear solve, consistent with the standard stopping bound $\|V_k - V^*\|_\infty \leq \gamma\epsilon/(1-\gamma) \approx 10^{-7}$.

Q. Why is Q-learning off-policy? Point at the line. This line: `td_target = r + gamma * Q[s_next].max()`. The bootstrap uses $\max_{a'} Q(s', a')$ — the **greedy** action — not $Q(s', a_{\text{actually taken next}})$. So the update does not depend on how the behaviour policy chooses to act. The agent behaves ϵ-greedily (and starts in a uniformly random state) forever, and still converges to Q^*: it *learns about* the greedy target policy while *behaving under* an exploratory one. Swap that `max` for the action you actually took and you have **SARSA**, which is on-policy and converges to Q^{π_ϵ} — a more conservative policy that hedges against its own future exploration mistakes (it would burn diesel earlier here, to avoid getting caught dry by a random exploratory IDLE). No importance sampling is needed, and that is worth saying: the target is defined by a *max*, not by an expectation under the behaviour policy, so there is nothing to reweight.

Q. State the Robbins-Monro conditions and prove a constant α cannot converge to Q*. Stochastic approximation needs $\sum_n \alpha_n = \infty$ (the steps must retain enough total mass to travel from any initialisation to Q^*) and $\sum_n \alpha_n^2 < \infty$ (the injected sampling noise must be annealed

away). My schedule $\alpha_n = 1/(1+n(s,a))^{0.7}$ satisfies both: $\sum n^{-0.7}$ diverges, $\sum n^{-1.4}$ converges. Note n is the **per-pair** visit count `visits[s,a]`, not a global step counter — the theory requires that. A **constant** α fails the second condition: $\sum \alpha^2 = \infty$. Each update keeps injecting a fresh $\alpha\cdot$ (TD noise) term that is never damped, so Q does not converge to a point at all — it converges *in distribution* to a random variable jittering in a ball of radius $O(\alpha)$ around Q^*. Bigger α, bigger ball. I do not just assert this; the sweep shows exactly the predicted ordering: polynomial $^{0.7}$ → **25.07 % ± 2.10**, constant 0.10 → **50.43 % ± 2.85**, constant 0.50 → **57.86 % ± 3.75**. The constant-α runs flatten out short of the optimum while the polynomial one keeps closing.

Q. What do exploring starts actually buy you? Q-learning's convergence proof *assumes* every (s,a) is visited infinitely often — it does not provide it. Under a fixed start (half tank, midnight, grid up, full ration) and a half-decent policy, states like "tank full at 08:00 with no diesel" are essentially never reached; their Q entries stay at whatever I initialised them to, and the argmax over them is noise. That matters because `policy_evaluation` scores from a **uniform** start — those unreachable states are in the score. Numbers: exploring starts ON → **100 % coverage of the 3432 legal pairs** and **25.07 %** regret; OFF → **76.73 % ± 7.04**. It is the **single biggest lever in the whole sweep**, by a factor of three, bigger than every learning-rate and epsilon choice combined. And the concession, before you make it: exploring starts are a **simulator privilege**. You cannot teleport a real hall's tank to a random level at midnight. In deployment I would need a different coverage story — restarts from logged historical states, or an intrinsic exploration bonus. My best result leans on something the real world would not give me for free, and I would rather name that than have it found.

Q. Q initialised at 0 — you call that optimistic. Then your ablation says pessimism wins. Explain. Every reward in this MDP is ≤ 0, so $Q \equiv 0$ is **optimistic**: untried actions look better than anything real, and the greedy argmax is pulled toward them. That is doing genuine exploration work, so I ablated it rather than quietly taking the credit — and the result was a surprise: `q_init = -200` (pessimistic) scores **21.05 % ± 1.30** versus optimistic **25.07 % ± 2.10**. Better, and with lower variance. The explanation: exploring starts already guarantee 100 % coverage, so the optimism has no coverage left to buy. What is left is a pure *scale* effect — V^* averages **−163.6**, so −200 starts near the correct magnitude, while 0 is wildly off-scale and the learner must spend thousands of updates walking every entry down before the argmax means anything. Optimism is a real mechanism; it is just not what is carrying this run. The exploring starts are.

2.2.3 (c) Problem Design

Q. Why is the diesel ration FINITE? What breaks without it? Because it is the only **irreversibility** in the problem, and irreversibility is what forces lookahead. F only ever decreases within a day; it resets to 2 at hour 23 and does **not** roll over. So burning a generator-hour at 07:00 on a hunch is a decision you cannot take back at 21:00 when it actually matters — it has an opportunity cost that is invisible in this hour's reward and only visible in the value function. Remove the ration (infinite fuel) and the whole thing collapses. F drops out of the state, there is no longer any cost to acting *now* rather than later, and the problem degenerates to a reactive rule — "if the tank is low, pump; if the grid is down, use the generator" — which a myopic policy can find in one step. There would be nothing for Value Iteration to be better *at*. The use-it-or-lose-it reset matters too: without it the agent could hoard fuel forever and never pump, which is a different degenerate optimum.

Q. Why is PUMP_GRID masked out during an outage? Isn't disallowing an action a bit convenient? It is the opposite of convenient — look at `_physics`. If `action == PUMP_GRID` but `grid == 0`, the branch falls through to the `else`: inflow 0, pump cost 0, fuel unchanged. That is **bit-for-bit identical to IDLE**. Offering it would not be "realistic", it would be a bug with three consequences: (1) Q-learning would spend a third of its exploration budget in those 792 states on a provably no-op duplicate; (2) the argmax between two exactly-equal actions is a coin flip, so any VI-vs-QL *policy agreement* number would be measuring the coin, not the policies; (3) it would inflate my legal-pair count and flatter my coverage statistic. The masking accounts for exactly the missing pairs: 4752 total, minus 792 (PUMP_GRID with the grid down) minus 528 (PUMP_GEN with no diesel) = **3432 legal**. And V^* is unchanged by masking — a duplicate action cannot change a max. Masking costs nothing in optimality and buys everything in sample

efficiency and metric honesty.

Q. Your reward is not a function of (s, a, s'). Isn't that outside the definition of an MDP? It is outside the *textbook's most common notation*, not outside the definition. The reward depends on the realised demand D, and D is **not recoverable from** s': once the tank hits zero, s' cannot tell you whether the students wanted one more unit or five more. So $r \neq r(s, a, s')$. What it *is* is the **disturbance form**: $r = r(s, a, w)$ with $w = (D, g')$ a fresh noise draw, independent of everything given (s, a). That is a completely standard and legitimate MDP — the requirement is that (s', r) jointly depend only on (s, a) plus fresh noise, which it does. Neither algorithm cares. Value Iteration only ever needs the **expected** reward $R(s, a) = \mathbb{E}_w[r]$, which is what `build_model` tabulates, and the Bellman operator is still a γ-contraction. Q-learning consumes the **realised** r, which is an unbiased sample of $R(s, a)$ — and that is all stochastic approximation ever asked for.

2.3 3.6 The traps — answered head-on

1. "Your Q-learner loses to a planner running a four-times-wrong model. Why did you bother?"

I concede it completely, and it is in the report as a finding, not buried. A planner who believes outages are **four times rarer than they are** (`outage_scale = 0.25`) scores **2.66 %** regret. My Q-learner, after **720,000** environment steps, scores **15.13 % ± 2.36**. VI on a wrong model beats it at **every** mis-specification I tested — 2.00× (1.54 %), 1.50× (0.43 %), 1.25× (0.11 %), 0.75× (0.24 %), 0.50× (1.54 %), 0.35× (2.10 %), 0.25× (2.66 %), 0.10× (6.66 %) — and only loses when the model claims the grid **never fails** (`scale = 0.0`, **21.39 %**).

And I will go further than you were going to. At this budget Q-learning also loses to the **tuned caretaker rule** (**14.51 %**) — a two-parameter reactive threshold with no learning in it at all. That is the honest headline and I would rather say it than have it extracted.

So why bother? Because the experiment answers a question that "which is better" does not. Q-learning needs **no** P **whatsoever** — only a doorway. The correct reading of my numbers is: *for a 1584-state MDP whose transition structure you can actually write down, planning wins, and it wins even when your parameters are badly wrong. Model-free earns its keep only where you cannot write the structure down at all.* The failure regime is not "a bit wrong" — a roughly wrong model is extremely robust here. It is **structurally** wrong (scale 0.0: the model has no outage mechanism at all) that finally makes 720,000 steps of experience worth buying. That is a sharper and more useful conclusion than "Q-learning is cool", and I only get to state it because I ran the sweep that could have embarrassed me.

2. "Where is your certainty-equivalence baseline?"

I have it, it wins, and it is the reason my thesis statement is narrower than it was when I started. Same training run, same snapshots, same **720,000** samples: I build the maximum-likelihood MDP from the counts Q-learning collected — $\hat{P}(s' \mid s, a) = N(s, a, s')/N(s, a)$, $\hat{R}(s, a) = \sum r/N(s, a)$ — and plan on it with value iteration. Result: **1.43 % ± 0.23**, against Q-learning's **15.13 % ± 2.36** on the identical data. Both curves come from one run, at the same instants, so the comparison is not arguable.

What that does to my thesis: it **kills** the naive claim "model-free beats model-based when the model is wrong". That claim is false and my own baseline falsifies it. The claim I am left with, and the one I will defend, is: **a learned model beats a wrong prior model, and it beats model-free at the same sample budget.**

The mechanism is not mysterious. A Q-learning update spends one transition on **one** (s, a) cell and then throws it away. Certainty-equivalence stores it and then, in planning, **re-uses** it through every Bellman backup until convergence — the sample gets propagated across the whole state space, not just into the cell it landed in. Q-learning is $O(1)$ memory per sample and wastes the information; CE is $O(S \cdot A \cdot S)$ memory and extracts it. That trade is exactly why model-free exists: not because it learns better, but because when S is large enough (or unenumerable), you *cannot* store or plan on $\hat{P}$ at all. My problem is 1584 states. It is not that world, and pretending otherwise would be a lie I could have easily gotten away with.

3. "Print Q(s, IDLE) and Q(s, PUMP_GRID) in an outage state. Why are they identical?"

They are not identical in my tables — `PUMP_GRID` is set to $-\infty$ in every $g = 0$ state, by `_masked()` for VI and by `Q[~avail] = -inf` for Q-learning. But you are asking about the *physics underneath the mask*, and there the answer is yes: they would be **bit-for-bit identical**. `_physics` falls through to `else: inflow = 0, pump_cost = 0.0` — flipping a switch on a dead line moves no water and costs nothing.

Had I not masked it, three things happen, and only the first is harmless:

- V^* and the optimal *value* are **unchanged** — a duplicate action cannot change a max. So no, this is not me hiding a modelling error; the optimum is the same either way.
- Q-learning would burn roughly a third of its exploration in those **792** states repeatedly learning the value of a no-op it already knows under a different name. Straight sample-efficiency loss.
- Every argmax in those 792 states becomes a **coin flip between two exactly-equal actions**. Any policy-agreement metric would then be reporting the coin. That is why I never report a label match against π^* — I report `action_optimality`, which asks whether the chosen action is *as good as* the best available one (value gap $\leq$ tol). It comes to **86.6 %** of states. A raw argmax comparison would have been louder, cheaper, and meaningless.

4. "You compare VI iterations with Q-learning episodes. Those have no common unit."

Correct, they have none, and I do not put them on the same axis. Defining the honest axis:

- One VI **sweep** is 3432 *model* backups, each touching up to $(D_{\max} + 1) \times 2 = 14$ successors. One Q-learning **step** is a single sampled update to a single cell. There is no exchange rate between them and I will not invent one.
- The axis I actually use in `plot_learning` is **environment steps** — simulated hours of hall operation. Q-learning consumes **720,000** of them. Certainty-equivalence consumes *exactly the same ones*, so it is plotted as a curve on the same axis. Value Iteration consumes **zero** — it never touches the environment — so it appears as a **horizontal reference line**. That is not a formatting shortcut, it is the entire point of the figure: the model is what lets you buy a policy without buying experience.
- Where compute is the question, I quote **wall-clock**: VI is **0.23 s**, Q-learning is minutes.
- And the outcome axis is neither. Every policy in the report — VI's, Q-learning's, CE's, the baselines' — is scored by the **same** function: exact policy evaluation under the **true** model, averaged over a uniform start, reported as **regret %**. No rollout noise, no lucky seeds. Different algorithms get different resource axes; they all get one scoreboard.

5. "Your simulator returns the realised shortage, not E[U]. Prove it. And if it returned E[U], what exactly would be wrong?"

Proof, three lines from `simulate_hour`:

```
demand = min(int(np.searchsorted(self._demand_cdf[hour], rng.random())), cfg.max_demand)
unmet  = max(0, demand - after_pump)
reward = -(pump_cost + cfg.cost_overflow * overflow + cfg.cost_shortage * unmet)
```

A uniform is drawn, a demand is realised, the shortage is that realisation. The only place an expectation over demand appears anywhere in the file is `exp_unmet = np.dot(p_demand, np.maximum(demand_vals - after_pump, 0))` — and that lives inside `build_model`, which Q-learning never calls. `step()` is the sole doorway and it contains no `np.dot`, no `p_demand`, no expectation of any kind. Empirically: call `step(s, a)` twice with the same arguments and you get different rewards. And the parity test in the suite Monte-Carlo-estimates P and R back out of `step()` and asserts they match `build_model()` within sampling error — one of the **23 RL tests** (37 total with PSO).

What would be wrong if it returned $\mathbb{E}[U]$: the reward would become deterministic and exactly equal to $R(s, a)$. That means I would have handed the *demand distribution* — half the model — to an agent I am calling model-free. The comparison would silently become "VI with P and R" versus "Q-learning with R", which is not the experiment I claim to have run. And here is the nasty part: **it would still converge to the same** Q^*, because the TD update only needs an unbiased sample of $R(s, a)$ and a zero-variance sample is trivially unbiased. It would just converge *faster* and look *better*. So the bug would be completely invisible in the results and

visible only in the code. That is exactly the class of error worth pre-empting, which is why it has a test rather than a sentence.

6. "γ = 0.99 was your principled choice, but γ = 0.90 learns a BETTER policy in your own table."

It does: **19.54 %** at $\gamma = 0.90$ versus **25.07 %** at $\gamma = 0.99$, at 8000 episodes. I am not going to explain that away, because there are two different gammas and conflating them is the actual error.

- The **evaluation** γ is **0.99**, fixed, and it *defines the objective*. Every regret number in the report — including the 19.54 % — is scored at $\gamma = 0.99$ under the true model. I did not move the goalposts to make a number look good.
- The **training** γ is a **hyperparameter of the learner**, and it is allowed to differ. Training at 0.90 shortens the bootstrap horizon from 100 hours to 10, which shrinks both the magnitude and the variance of the TD target and dramatically speeds up credit assignment. The resulting policy is *biased* for the 0.99 objective — but at finite samples the variance reduction more than pays for the bias.

This is the known effective-planning-horizon result: when your value estimates are noisy, a lower discount acts as **regularisation**. Asymptotically $\gamma = 0.99$ training must win — it is the only one whose fixed point is the right Q^*. At 8000 episodes it does not, because 8000 episodes is nowhere near asymptotic.

And the reason 0.90 gets away with it *here specifically*: $1/(1 - 0.90) = 10$ hours of lookahead. The anticipation this problem needs happens at around 14:00 for an 18:00–22:00 outage window — four to eight hours out. A 10-hour horizon still sees it. Had the critical structure been 30 hours away, $\gamma = 0.90$ would have been blind to it and the table would look very different. That is the caveat that makes the observation a finding rather than a fluke.

7. "Prove this problem needs lookahead and isn't just a threshold rule in disguise."

Three independent pieces of evidence, in ascending order of how hard they are to argue with.

(i) The greedy policy is catastrophic. `policy_myopic` is VI with $\gamma = 0$ — not a strawman, the **formally correct** one-step-optimal policy, $\arg\max_a R(s, a)$. It scores **−361.012**, a regret of **120.62 %**: **78.1** cost/day against the optimum's **32.8**, and **3.08** shortage units/day against **0.78**. If this problem had no long-horizon structure, the greedy policy would land near the optimum. It loses by more than a factor of two.

(ii) The best possible threshold rule still loses. I did not compare against a crippled heuristic. `tune_caretaker` grid-searches **both** thresholds over all 11×11 combinations and scores each by exact policy evaluation. The winner is (pump on grid if $L < 9$; burn diesel if $L < 3$) — it has the **same action set** as the optimal policy, generator included. It scores **−187.384**, regret **14.51 %**, **36.9** cost/day. That is the strongest version of "just a threshold rule" that exists, tuned to its own best configuration, and it is still **12 % worse per day** than the optimum. What it cannot do is **read a clock**: its action is a function of (L, g, F) only. The optimal policy's action at fixed (L, g, F) **changes with** t — you can see it in the policy map, pre-filling into the shaded evening window. No time-blind threshold can express that, no matter how you tune it.

(iii) A witness state. At **14:00, tank 4/10, grid up, full ration**: pumping on grid costs **0.97 MORE** this hour in expected reward than idling. It is nevertheless the optimal action — $Q^*(s, \text{PUMP\_GRID}) - Q^*(s, \text{IDLE}) = +\mathbf{4.69}$. So $+4.69 - (-0.97) = +\mathbf{5.66}$ of that advantage comes **purely from the future**. That is the signature of lookahead, in one state, computed from Q^*, not asserted. The agent takes a certain loss now to hold water it will need in four hours' time.

8. "Is your state Markov with respect to REALITY, or only to your model?"

Only to my model. I will say it plainly rather than be walked into it.

Inside the model, (L, t, g, F) is Markov by construction, and the parity test confirms the simulator and the transition table agree. But that test proves **internal consistency, not fidelity to Dhaka** — the simulator *is* the model. Verifying them against each other is a tautology dressed as a check, and it is worth being clear about which of the two things it does.

Where reality breaks it:

- **Load-shedding is scheduled, not geometric.** I model outage duration with a memoryless two-state chain (constant p_{restore}, so duration is geometric with mean 6.7 h in the evening). Real rotational shedding runs in **fixed slots**. That makes the hazard rate non-constant: an outage that started 90 minutes ago is *more* likely to end in the next hour than one that started five minutes ago. My state cannot represent that, because it does not know how long the current outage has been running.
- **Demand is autocorrelated** beyond hour-of-day — exam week, weekends, weather. My Poisson mean is a pure function of t.
- **The pump is not single-speed** and the tank has hydraulics; inflow rate depends on head.

The remedy is not a different algorithm — this is the point I want to land. If the state is not Markov, tabular **Q-learning is broken too**: its convergence guarantee assumes a Markov state, and on a non-Markov one it converges to the fixed point of a backup that has no optimality meaning. Neither planning nor learning survives a bad state space. **You fix the state, not the solver.** The concrete fix here is to augment the state with time-since-outage-start (or the published schedule slot), which restores the Markov property at the cost of multiplying $|S|$ by the number of duration bins. I would do exactly that if I had the real schedule data, and the model-mismatch sweep in E4 is precisely the instrument for asking how much that unmodelled structure would cost me.

Code Defense

Line by line, and the screen questions

15 July 2026

Contents

1 Part 5 — Code Defense: line-by-line mapping and screen questions

Companion to `src/pso_wifi_placement.py`. This document (a) maps concrete code to the theory of Part 1 / Part 3, and (b) pre-loads sharp answers to the questions an examiner asks while pointing at the screen.

1.1 5.1 Code → theory mapping

Code (in `src/pso_wifi_placement.py`)	Theoretical principle	Where discussed
`X = rng.uniform(p.lower, p.upper, (P, D))` (swarm init)	Population-based sampling: cover the search space, not a single start point	Part 1 §1
`V = w*V + c1*r1*(pbest-X) + c2*r2*(gbest-X)`	The PSO velocity update; `cognitive` term = personal memory (exploitation of own best), `social` term = swarm attraction (information sharing)	Part 1 (PSO); slide ”How a particle moves”
`w = w_max - (w_max-w_min)*t/(T-1)`	Exploration→exploitation schedule: high inertia early (roam), low inertia late (refine)	Part 1 (explore/exploit trade-off); Part 3 Q
`r1, r2 = rng.random(...)` fresh each step	Stochasticity that keeps the search from collapsing deterministically; the ”random wing-flutter”	Part 1 (diversity)
`V = np.clip(V, -v_max, v_max)`	Velocity clamping $V_{\max}$: stability guarantee against swarm explosion	Part 1 (PSO pathologies)

Code (in `src/pso_wifi_placement.py`)	Theoretical principle	Where discussed
`X = np.clip(X, lower, upper); V[hit]=0`	**Constraint repair** for the boundary (box) constraint; absorbing walls	Part 1 (constraint handling); Part 3 Q
`breach += max(0, d_min-‖a_j-a_k‖)**2`	**Penalty method** for the inequality (separation) constraint	Part 1; Part 3 "constraint" trap
`pbest/pbest_fit` update on `improved`	Elitist personal memory: never forget your best (monotone `pbest`)	Part 1 (convergence)
`if pbest_fit[best] > gbest_fit: gbest=…`	Global-best elitism → guarantees the returned `history` is monotone non-decreasing	Part 1 (convergence definition)
`_diversity()` = mean distance to centroid	Quantitative diversity metric to *detect* premature convergence	Part 1 (premature convergence); `diversity.png`
`_stagnation_iter()`	Stopping-criterion analysis: last iteration with > `eps` improvement	Part 3 (stopping criteria)
`random_search(... n_evals ...)` and the `assert n_fitness_calls == n_evals`	**Fair baseline**: identical fitness-evaluation budget (not identical iterations)	Part 3 (fair comparison trap)
`grid_search()` with `C(G,K) ≤ budget`	Honest grid baseline; its inability to scale = live curse-of-dimensionality demo	Part 3
`stats.wilcoxon(pso_best, rand_best, alternative='greater')`	Non-parametric paired significance test over independent runs	Part 3 (statistical significance trap)
multi-seed `run_experiments` (mean ± std)	Metaheuristics are stochastic → report distributions, not a single lucky run	Part 3

1.2 5.2 Biological-analogy annotations (already inline in the code)

- **Particle** = a bird in a flock hunting the best feeding spot; its position is a whole candidate AP layout.
- **`pbest`** = the bird's private memory / nostalgia for the best spot it found.
- **`gbest`** = the socially broadcast best spot found by any bird.
- **inertia `w`** = the bird's momentum; **`r1,r2`** = the random flutter of wings.
- **velocity clamp** = a physical speed limit so a bird cannot teleport across the whole hall in one wingbeat.

1.3 5.3 Three high-probability "pointing-at-the-screen" questions

1.3.1 Q1 — "On the velocity line, what happens if the inertia weight `w` goes to 0? What if it stays at 0.9 the whole run?"

Answer. `w` scales the particle's *momentum* (its previous velocity).

- **`w → 0`:** the velocity loses all memory of prior motion; each step becomes a pure random-weighted pull toward `pbest`/`gbest`. The swarm contracts onto the current global best almost immediately — strong exploitation, but **premature convergence**: if `gbest` is a mediocre local optimum, the swarm is trapped there with no momentum to carry it out.
- **`w = 0.9` fixed:** the opposite failure — particles keep barrelling past good regions (over-exploration), the swarm never settles, and final-iteration accuracy is poor.
- **Why my schedule:** I decay `w` linearly $0.9 \to 0.4$ so the swarm explores broadly early and refines late — the standard, stable Shi–Eberhart (1998) regime. The `diversity.png` curve is the empirical proof this transition happens.

1.3.2 Q2 — "Delete the `np.clip(V, -v_max, v_max)` line — or set `v_max` huge. What breaks?"

Answer. That line is the **velocity clamp**, the classic PSO stability mechanism. Without it, the term $w\mathbf{v} + c_1 r_1(\cdot) + c_2 r_2(\cdot)$ can grow without bound when a particle is far from `pbest`/`gbest`: velocities **explode**, positions overshoot the floor every step, and the swarm diverges instead of converging (the well-documented "swarm explosion"). The subsequent position clamp would then just pin particles to the walls with zeroed velocity, destroying the search. `v_max` is set to 20 % of each axis range so a particle moves in meaningful sub-floor steps. (The constriction-factor variant of Clerc & Kennedy 2002 is the alternative fix; I use inertia + `v_max` to match the course slide's update rule.)

1.3.3 Q3 — "Why `np.maximum(dist, 1.0)` before the `log10`? Remove it — what happens?"

Answer. The radio model is **log-distance path loss** referenced to $d_0 = 1$ m: $\text{PL} = L_0 + 10n \log_{10}(d/d_0)$. Two reasons for the floor:

1. **Numerics:** if an AP sits exactly on a room, $d = 0$ and $\log_{10} 0 = -\infty$, giving RSSI $= +\infty$ and `NaN` fitness that poisons the whole run.
2. **Physics:** the log-distance model is only valid in the far field ($d \geq d_0$); below 1 m it is meaningless. Flooring at 1 m keeps every evaluation inside the model's domain. Removing it makes "stack an AP on the busiest room" look infinitely good — a degenerate optimum the optimizer would happily exploit.

1.4 5.4 What the final printout proves (for the viva)

Running `./run.sh swarm` prints, in order:

- the **best solution vector** (the K AP coordinates),
- its **fitness** (weighted coverage %) and **hard room coverage %**,
- the **iteration at which convergence stagnated**,
- a **side-by-side matrix** (PSO vs Random vs Grid) at the *identical* budget,
- the **Wilcoxon** statistic and p-value with a significance verdict.

This is the complete evidentiary chain: *what* was found, *how good* it is, *when* it converged, and *whether the advantage over the baseline is real*.

---

2 Part 5B — Code Defense: the decision-making code

Companion to `src/rl_water_tank.py` (the MDP, Value Iteration, Q-learning, certainty equivalence) and `src/rl_experiments.py` (the experiments that produce every number and figure). Same job as above: map the code onto the theory, then pre-load the answers to the questions an examiner asks while pointing at a line.

2.1 5.5 Code → theory mapping

Code	Theoretical principle	Where discussed
`value_iteration()`: `Q = R + gamma*(P@V).reshape(S,A)` then `V_new = _masked(Q,avail).max(axis=1)`	The **Bellman optimality operator** $(\mathcal{T}V)(s) = \max_{a \in \mathcal{A}(s)} \left[R(s,a) + \gamma \sum_{s'} P(s' \mid s,a) V(s')\right]$, applied synchronously. The `max` is what makes it *optimality* rather than *expectation*	Part 1 (Bellman equations)
`residuals.append(delta)`, 'delta =	V_new - V	_inf'
`if delta < tol: break`, `tol=1e-9`	Stopping rule justified by the standard bound $\|V_k - V^*\|_\infty \le \gamma\varepsilon/(1-\gamma)$ — not picked by feel	Part 1 (VI error bound)
`policy_evaluation()`: `spla.spsolve(sp.eye(S) - gamma*P_pi, R_pi)`	The **Bellman expectation equation** for a *fixed* policy, $V^\pi = R_\pi + \gamma P_\pi V^\pi$, is **linear** — so we solve $(I - \gamma P_\pi)V^\pi = R_\pi$ exactly rather than iterating it. $(I - \gamma P_\pi)$ is invertible because $\rho(\gamma P_\pi) \le \gamma < 1$	Part 1; §5.7 Q6
`build_model()` → `R[s,a] = -(pump_cost + c_ov*overflow + c_short*exp_unmet)` where `exp_unmet = p_demand @ max(d - after_pump, 0)`	**Expected reward** $R(s,a) = \mathbb{E}_w[r(s,a,w)]$, $w = (D, g')$. The reward is *not* a function of (s,a,s') — once the tank hits zero you cannot recover D from s' — so we are in the disturbance form of an MDP. VI only ever needs the expectation, and the operator is still a contraction	Part 1 (MDP formalism)
`simulate_hour()` → `unmet = max(0, demand - after_pump); reward = -(...)`	The **realised** reward, from a *drawn* demand. Q-learning must consume this. Feeding it $\mathbb{E}[U]$ would leak the demand distribution into the "model-free" agent and collapse the entire point of the assignment	Part 1 (model-free)
`step()` — the only method Q-learning is allowed to call	The **sampling oracle**. No probability ever leaves this function. This narrow doorway *is* the model-based / model-free distinction, made mechanical rather than rhetorical	Part 1
`build_model()` returning a sparse `csr_matrix` of shape `(S*A, S)`	The explicit $P(s' \mid s,a)$ that makes VI *model-based* in the most literal sense: the sum over s' is only computable because somebody handed us P. Each row has ≤ 14 non-zeros; a dense $S \times A \times S$ array would be 99.9% zeros	Part 1
`q_learning()`: `td_target = r + gamma*Q[s_next].max()`; `Q[s,a] += alpha*(td_target - Q[s,a])`	The **temporal-difference update** $Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$. The `.max()` is the **off-policy** part: we bootstrap off the *best* next action even though ε-greedy may not take it. That is precisely what lets the agent converge to Q^* while behaving sub-optimally throughout	Part 1 (Q-learning); §5.7 Q4

Code	Theoretical principle	Where discussed
`alpha = 1.0/(1.0 + visits[s,a])**qcfg.alpha_omega, alpha_omega=0.7`	**Robbins–Monro** conditions: $\sum_n \alpha_n = \infty$, $\sum_n \alpha_n^2 < \infty$. $\omega \in (0.5, 1]$ delivers both. A *constant* α satisfies neither and provably converges only to a *neighbourhood* of Q^* — we do not just assert this, the sweep shows constant $\alpha = 0.1$ stalling at **50.43%** regret where the polynomial schedule reaches **25.07%**	Part 1 (stochastic approximation)
`eps = max(eps_end, eps_start - (…)*ep/anneal_over)`	ε-greedy **exploration–exploitation** schedule: explore early, exploit late. The mirror of PSO's inertia decay in §5.1	Part 1 (explore/exploit)
`if qcfg.exploring_starts: s = rng.integers(S)`	**Exploring starts**: buys the coverage that Q-learning's convergence proof simply *assumes* (every (s, a) visited infinitely often). Turning it off costs **76.73%** regret against 25.07% — the single most consequential knob in the sweep	Part 1 (convergence conditions)
`_build_availability()` → boolean `(S,3)` mask; `Q[~avail] = -inf`; `_masked()`	**State-dependent action sets** $\mathcal{A}(s)$. `PUMP_GRID` needs the grid, `PUMP_GEN` needs diesel. 3432 legal (s, a) of 4752. Not realism-decoration — see §5.7 Q5	Part 4 §B1.2
`certainty_equivalence()`: `P_hat = counts/n`, `R_hat = reward_sums/n`, then `value_iteration(P_hat, …)`	**Certainty equivalence** / model-based RL: form the maximum-likelihood MDP from experience and plan on it as if it were true. Run on the *same samples* Q-learning saw, at the same instant. It is the arm that could have sunk the report's thesis, and it very nearly did — **1.43%** regret against Q-learning's **15.13%**	Part 1 (model-based RL)
`worst = -(cost_gen + c_ov*pump_rate + c_short*max_demand)` for unvisited (s, a), plus a self-loop	Pessimism under uncertainty: an action never tried cannot look attractive by accident. The alternative (optimism) would make CE-VI fall in love with unexplored actions	Part 1
`policy_myopic() = _masked(R, avail).argmax(axis=1)`	Value iteration at $\gamma = 0$: the **formally correct greedy policy**. Not a strawman — if it landed near the optimum, the problem would have no long-horizon structure and the whole assignment would be vacuous. It loses **120.62%**	Part 4 §B1.3
`tune_caretaker()` grid-searching both thresholds	**Fair baseline**: we report the *best* version of the reactive rule, not a hand-crippled one. Beating a crippled baseline proves nothing	Part 3 (fair-comparison trap)

Code	Theoretical principle	Where discussed
`score_policy()` / `regret_percent()` — exact V^π under the *true* P	Every policy in the report is ranked by **exact expected discounted return**, never by whichever run drew a luckier rollout. Monte-Carlo noise is removed from the comparison entirely	Part 3 (statistical rigour)
`action_optimality()`: `(Q*.max(1) - Q*[range, policy]) <= tol`	Deliberately *not* a label match against π^*. Many states are near-ties; a raw `argmax` comparison would report loud "disagreement" where the two choices differ by 0.001. We ask the question that matters: is the chosen action *as good as* the best available one?	Part 3
`e4_mismatch()`: `mdp.build_model(outage_scale=k)` then score in the *true* model	**Model mis-specification**. `outage_scale` multiplies p_{fail} only — "twice as often" and "twice as long" are different errors with different consequences, so we vary exactly one	Part 4 §B1.4
multi-seed `seeds=[1..5]`, mean ± std everywhere	RL is stochastic → report distributions, not a single lucky run. Same discipline as the swarm half	Part 3

2.2 5.6 The one invariant the whole report rests on

`HallWaterMDP` has two faces, and keeping them honest is the entire trick:

- **`build_model()`** hands out P and R. This is the *only* thing Value Iteration touches. It averages over the randomness.
- **`step()`** hands out one sampled (s', r). This is the *only* thing Q-learning touches. It samples the randomness.

Both call the same `_physics()` helper, so the two faces cannot disagree about what a pump-hour *does* — only about whether they average over the disturbance or draw it. And `tests/test_rl_water_tank.py` Monte-Carlo-estimates P and R back out of `step()` and asserts they match `build_model()` within sampling error. That parity test is the gate: if it fails, every VI-vs-Q-learning number in the report is meaningless, because the two algorithms would be solving different problems.

2.3 5.7 Three high-probability "pointing-at-the-screen" questions

2.3.1 Q4 — "In `q_learning`, the TD target uses `Q[s_next].max()`. Change it to `Q[s_next][a_next]`, where `a_next` is the ε-greedy action you will actually take next. What algorithm have you just written, and what changes?"

Answer. That is **SARSA** — the on-policy TD control algorithm (Rummery & Niranjan 1994). The change is one character wide and it changes what the algorithm converges to.

- **Q-learning is off-policy.** The `max` bootstraps off the *greedy* action regardless of what the behaviour policy actually does next. The target is therefore an estimate of Q^*, and under the Robbins–Monro conditions plus infinite visitation, $Q \to Q^*$ — **the optimal action-values, even though the agent never once behaves optimally during training**. That decoupling is the whole selling point.

- **SARSA is on-policy.** The target $r + \gamma Q(s', a')$ uses the action the ε-greedy policy *will actually take*, so it converges to Q^{π_ε} — the value of the **exploring** policy, not the optimal one. It reaches Q^* only if you also anneal $\varepsilon \to 0$ (a GLIE schedule).
- **What it would look like here.** SARSA's values are pessimistic about states where exploration is expensive, because it *charges itself* for the random actions it is going to take. In this MDP the expensive random action is burning diesel — with ε floored at 0.05 the agent randomly torches its ration in 5% of steps, and SARSA, unlike Q-learning, prices that in. So the SARSA policy would be more conservative about states near the edge of a shortage. The classic cliff-walking result in miniature: SARSA learns the safe path, Q-learning learns the optimal one.
- **Why we chose Q-learning.** We are comparing against Value Iteration, which returns π^*. Comparing π^* against π^ε would be comparing two different optima and the regret numbers would be uninterpretable. Q-learning is the model-free algorithm that targets the *same* fixed point VI does, which is the only reason the head-to-head is fair.

2.3.2 Q5 — "Delete the line `Q[~avail] = -np.inf`. What breaks?"

Answer. Three things break, and — the interesting part — the *optimal value* is not one of them.

1. **The agent can now select actions that do nothing.** With the mask gone, the ε-greedy `argmax` can pick `PUMP_GRID` in a state where the grid is down, or `PUMP_GEN` with an empty diesel drum. Follow it into `_physics()`: neither branch matches, so it falls through to `else: inflow, pump_cost = 0, 0.0`. The action is a **bit-for-bit no-op — an exact duplicate of `IDLE`**.
2. **A third of exploration is wasted.** Exploration would sample uniformly from all three actions instead of from $\mathcal{A}(s)$. In the **792 outage states**, `PUMP_GRID` is a duplicate of `IDLE`, so a third of the exploratory steps taken there teach us nothing we were not already learning. Same story in the 528 zero-diesel states for `PUMP_GEN`. That is why we pre-compute `legal = [np.flatnonzero(avail[s]) …]` and sample from *that*.
3. **Every `argmax` in those 792 states becomes a coin flip.** $Q(s, \text{idle})$ and $Q(s, \text{pump/grid})$ converge to the *identical* value, so the argmax between them is decided by floating-point noise. Any policy-agreement metric between VI and Q-learning would then be measuring **how the two implementations happen to break ties**, not how good their policies are. The reported agreement number would be garbage, and it would be garbage in a way that looks like a real result.

But V^* is unchanged. Adding a duplicate of an action already in $\mathcal{A}(s)$ cannot change $\max_a Q(s, a)$ — the max of a set is unaffected by repeating an element of it. So Value Iteration would still compute exactly the same optimal value function and (up to tie-breaking) the same policy. The mask is not there to protect the mathematics. It is there to protect the **experiment**: it keeps the action set honest so that exploration is not diluted and so that policy comparisons measure policies. This is also why `action_optimality()` scores "is the chosen action as good as the best one?" instead of "does the label match π^*?" — the same near-tie problem, one level up.

2.3.3 Q6 — "Why is `policy_evaluation` a linear `spsolve`, but `value_iteration` a loop? Solve them both."

Answer. Because one of them is linear and the other one is not, and the difference is exactly the `max`.

- **Fixed policy.** With π fixed, every state has exactly one action, so $R_\pi \in \mathbb{R}^S$ and $P_\pi \in \mathbb{R}^{S \times S}$ are just matrices you can slice out (`rows = arange(S)*A + policy`). The Bellman *expectation* equation is
$$V^\pi = R_\pi + \gamma P_\pi V^\pi \quad \Longleftrightarrow \quad (I - \gamma P_\pi) V^\pi = R_\pi,$$
which is **affine in** V — an $S \times S$ linear system. P_π is row-stochastic so $\rho(P_\pi) = 1$, hence $\rho(\gamma P_\pi) \le \gamma < 1$, hence $(I - \gamma P_\pi)$ is non-singular and the solution is *unique*. One sparse LU solve returns it **exactly** — no tolerance, no iteration count, no truncation error.
- **Optimal policy.** The Bellman *optimality* equation is
$$V^*(s) = \max_{a \in \mathcal{A}(s)} \Big[R(s, a) + \gamma \sum_{s'} P(s' \mid s, a) V^*(s') \Big].$$

The max is piecewise-linear and convex, but it is **not linear**. There is no matrix to invert. The only handle we have is that the operator $\mathcal{T}$ is a γ-contraction in $\|\cdot\|_\infty$, so Banach's fixed-point theorem guarantees a unique V^* and guarantees that iterating $\mathcal{T}$ converges to it geometrically. Geometrically, not finitely: at $\gamma = 0.99$ it takes **2273 sweeps** to drive the residual under 10^{-9} (0.23 s), and the measured decay rate is **0.9900** — exactly γ, as the theory says.

- **The two together are policy iteration.** Alternating "solve exactly for V^π" with "take the greedy policy w.r.t. it" is Howard's policy-iteration algorithm. We have both halves in the file; we run VI because it needs no policy-improvement bookkeeping and 0.23 s is not a runtime we need to optimise.
- **Why the exactness matters to the report.** `policy_evaluation` is the **scoring function for every policy we report** — VI's, Q-learning's, certainty-equivalence's, the tuned caretaker's. Because it is a solve and not a rollout, the ranking between policies carries **zero Monte-Carlo noise**: when we say Q-learning lands at 15.13% regret and CE-VI at 1.43%, that gap is a property of the policies, not of the random seeds we happened to draw.

2.4 5.8 What the RL printout proves (for the viva)

Running `./run.sh rl` prints, in order:

- **The MDP, stated honestly.** $|\mathcal{S}| = 1584$ (level 11 × hour 24 × grid 2 × diesel 3), 3432 legal (s, a) of 4752, the sparsity of P, and $\gamma = 0.99 \Rightarrow$ a 100-hour effective horizon. The examiner can check the arithmetic on the spot.
- **E1 — Value Iteration does what the theory promised.** 2273 sweeps, 0.23 s, empirical contraction rate **0.9900** against a predicted $\gamma = 0.99$; and a cross-check that $\|V^* - V^{\pi^*}\|_\infty \approx 0$, i.e. the iterative operator and the exact linear solve agree. $V^* = -163.637$. This is the number everything else is measured against.
- **E2 — the problem is not vacuous.** The formally correct greedy policy loses **120.62%**; the *tuned* reactive threshold rule — same actions as the optimum, both thresholds grid-searched — still loses **14.51%**. Lookahead is doing real work.
- **E3 — what experience is worth.** After **720,000 environment steps** (82 simulated years of hall operation), Q-learning reaches **15.13% ± 2.36** regret. Certainty-equivalence VI, planning on a model estimated from *exactly the same samples at the same instant*, reaches **1.43% ± 0.23**. Same data, two ways of spending it, an order of magnitude apart.
- **E4 — how wrong a model can afford to be.** A planner who believes outages are four times rarer than they are loses **2.66%**. One who is 25% off loses **0.11%**. One who believes the grid never fails loses **21.39%**. So a *roughly* right prior model beats 82 years of experience, and a *badly* wrong one does not. That is the finding, and it is not the one we expected.
- **E5 — which knobs matter.** Exploring starts OFF: **76.73%** regret, against **25.07%** with them ON. Constant $\alpha = 0.1$: **50.43%**, against **25.07%** for the polynomial Robbins–Monro schedule. Both ablations confirm a theoretical prediction rather than merely tuning a number.
- **E6 — the anticipation witness.** At 14:00, tank 4/10, grid up: pumping costs **0.97 more** this hour than idling, yet Q^* says its advantage is **+4.69**. So **+5.66 of value comes purely from the future**. That single line is the entire argument for lookahead, in one state, with a number attached.

Together: *the model is stated, the exact optimum is computed and verified against theory, the greedy and reactive alternatives are shown to fail, the model-free agent is given a fair and generous budget, the honest baseline that could have refuted us is run anyway,* and *the one thing lookahead buys is exhibited in a single state.*

Literature Applied

Which paper changed which line

15 July 2026

Contents

1 Part 7 — What we took from the literature, and where it shows up

The honest question is not ”did you cite papers”, it is ”would this project be different if you had not read them”. This file answers that, one paper at a time, with the line of code or the experiment that changed. Where a paper made a claim we could test, we tested it, and we say what happened — including the two places where we got it wrong first.

1.1 7.1 The short version

Paper	What we took from it	Where it lives	Did we test its claim?
Kennedy & Eberhart 1995	the velocity update itself	`pso_wifi_placement.py` §3	— (it is the algorithm)
Shi & Eberhart 1998	inertia weight `w`: 0.9 → 0.4	`optimize()`	yes, via the diversity curve
Kennedy & Mendes 2002	**swarm topology is a variable, not a given**	`topology` / `_social_attractor()`	**yes — and it corrected us**
Wolpert & Macready 1997 (NFL)	you must *demonstrate* fit to the landscape, not assert it	the entire ablation	it is why the ablation exists
Rappaport 2002	log-distance path loss + wall attenuation	`WifiFloorProblem.fitness()`	(it is the objective)
Bellman 1957 / Howard 1960	value iteration; the contraction bound	`value_iteration()`	yes — measured rate = 0.9900 = γ
Watkins & Dayan 1992	the TD update; the `max` that makes it off-policy	`q_learning()`	— (it is the algorithm)
Jaakkola et al. 1994 / Tsitsiklis 1994	”every (s,a) infinitely often”	exploring starts	yes — off = 76.7% vs on = 25.1%
Even-Dar & Mansour 2003	α = 1/(1+n)^0.7, and *why constant α cannot work*	`alpha_omega`	**yes — constant α stalls at 50.4%**

Paper	What we took from it	Where it lives	Did we test its claim?
Agarwal et al. 2020 + Li et al. 2024	**the sample-complexity gap that explains our headline result**	`certainty_equivalence()`	yes — predicted gap, observed gap
Sutton 1990 (Dyna)	the model-free/model-based boundary is a spectrum	framing of Part B	noted as future work

Two of these did not merely inform the project. They **changed a result we had already written down.** Those two are worth the rest of this document.

1.2 7.2 Kennedy & Mendes (2002) — the paper that caught us overclaiming

What we had. Our first ablation was binary: communication on (every particle pulled towards the single global best) or off (`c2 = 0`). On, 89.91. Off, 87.30 — indistinguishable from random search. Communication matters. Done.

What the paper says. Kennedy & Mendes systematically compare 70 different swarm communication topologies, and their finding is narrower and more useful than the folklore:

> *"greater connectivity speeds up convergence, though it does not tend to improve the population's ability to discover global optima"*

Concretely, they measure the fully-connected `gbest` swarm hitting their criterion in a median of **404.8 iterations but only 85% of the time**, against a ring `lbest` swarm needing **755.5 iterations but succeeding 94% of the time**.

They also warn about the other end, which is exactly our `c2 = 0` row:

> *"inhibiting communication too much results in inefficient allocation of trials, as individual particles wander cluelessly through the search space"*

What we did with it. We implemented a ring topology (`ring_k` neighbours each side, information walks around the circle one hop per iteration instead of teleporting) and re-ran the ablation at the same 3,030-evaluation budget.

What we got, and how we got it wrong first. Our first pass reported that the ring was simply *better* — mean 89.99 against gbest's 89.91. A unit test asserting that failed on a different set of seeds, which was the correct outcome. We then tested it properly, on 30 paired seeds:

	mean	sd	stagnates at
fully connected (gbest)	89.831	0.317	iteration **83** / 100
ring, k = 1	89.961	**0.129**	iteration **100** / 100

- **Wilcoxon on the mean: p = 0.92.** Not significant. The ring does **not** find a better optimum.
- **F-test on the variance: 6× smaller, $p = 6 \times 10^{-6}$.** Highly significant.
- The fully-connected swarm **gives up at iteration 83**. The ring is still improving when the budget runs out.

That is the paper's claim, reproduced: more connectivity converges *faster*, not *better*. The gain from the ring is **reliability**, not peak performance. Our first write-up said "better" and the statistics did not support it.

The bit we would have got wrong if we had only skimmed it. The paper does *not* recommend the ring. It ranks ring-with-self **64th of the 70** topologies tested ("slow and inaccurate") and recommends the **von Neumann** lattice instead. So we must not say "the literature says use a ring". We say the narrower true thing: the literature predicts a speed-versus-reliability trade between dense and sparse topologies, and on our landscape that trade appears exactly as described. Trying von Neumann is the obvious next experiment and we have not run it.

1.3 7.3 Even-Dar & Mansour (2003) — a prediction we could falsify

The Robbins–Monro conditions require $\Sigma\alpha = \infty$ and $\Sigma\alpha^2 < \infty$. A **constant** α satisfies the first and fails the second, so it provably converges only to a *neighbourhood* of Q*, never to Q* itself. A polynomial rate $\alpha_n = 1/(1+n)^{\omega}$ with $\omega \in (0.5, 1]$ satisfies both.

This is a falsifiable prediction about *our* problem, so we put both in the sweep rather than just citing the theorem:

α schedule	regret
polynomial, ω = 0.7	**25.1%**
polynomial, ω = 0.5	28.2%
constant α = 0.10	50.4%
constant α = 0.50	57.9%

The constant-α runs stall exactly as the theory says they must. We did not have to take the theorem on faith, and neither does the reader.

1.4 7.4 Agarwal et al. (2020) and Li et al. (2024) — why our headline result happens

This is the one that turned an empirical curiosity into an explanation.

Our result. Take the 720,000 transitions Q-learning consumed, estimate the maximum-likelihood model $\hat{P}$ from them, and plan on it. That *certainty-equivalence* planner reaches **1.43%** regret against the Q-learner's **15.13%** — a factor of ten, on identical data. We could describe this but not explain it.

The near-miss. Our first instinct was to cite Azar, Munos & Kappen (2013), which proves a minimax bound on model-based value iteration. When we checked what it actually proves, it does **not** support our claim: its lower bound is information-theoretic and binds *every* algorithm, model-free included. Citing it for a model-based-beats-model-free claim would have been wrong, and an examiner who knows the paper would have said so.

The papers that do support it. The separation is real, and it is sharper than we expected:

- **Agarwal, Kakade & Yang (2020)** prove the *plug-in* (certainty-equivalence) estimator — build $\hat{P}$, plan optimally in the empirical MDP — is **minimax optimal**, achieving $\tilde{\Theta}(|S||A| / ((1-\gamma)^3\varepsilon^2))$.
- **Li, Cai, Chen, Wei & Chi (2024)** settle vanilla synchronous Q-learning at $\tilde{\Theta}(|S||A| / ((1-\gamma)^4\varepsilon^2))$ and state plainly that this "unveils the strict sub-optimality of Q-learning when $|A| \geq 2$".

So certainty-equivalence sits at $1/(1-\gamma)^3$ and vanilla tabular Q-learning is stuck at $1/(1-\gamma)^4$ — **a full factor of $1/(1-\gamma)$ worse in the effective horizon.**

Now put our numbers in. We chose $\gamma = 0.99$, so $1/(1-\gamma) = \mathbf{100}$. The theory predicts our Q-learner should be roughly two orders of magnitude behind in sample efficiency, purely because of the horizon. That is not a curiosity about our hall. It is a known, proved separation, and our 1.43%-versus-15.13% is what it looks like from the inside.

The caveat we must not drop. The separation is against *vanilla* Q-learning, not model-free RL in general. Variance-reduced Q-learning variants do reach the optimal rate. So the correct sentence is "tabular Q-learning is provably sample-inefficient here", **not** "model-free learning is worse". We say the first.

The mechanism, in one line. A Q-learning update spends a transition on one cell of the table and discards it. A learned model *stores* it, and every subsequent Bellman backup re-reads it — so one sample propagates across the whole state space instead of into the single cell where it landed.

1.5 7.5 Wolpert & Macready (1997) — the paper that shaped the method, not the code

No Free Lunch says that averaged over all objective functions, every optimiser is identical. The practical consequence is that ”PSO is good” is not a claim you can make; only ”PSO is good *on this landscape*, and here is the evidence” is.

That single idea is why Part A is built the way it is. It is why we do not report ”PSO found a good layout” and stop; it is why there is a matched-budget random search, a matched-budget grid search, a swarm-size sweep, a communication ablation and a topology comparison. Every one of those exists to answer ”compared to what?”.

It is also why the Part B result survives. We went looking for evidence that model-free learning wins, ran the experiment that could embarrass us, and it did. Reporting that is the same discipline.

1.6 7.6 What we read and did not use

Being straight about this is part of the point.

- **The twelve applications in PART2** are a genuine literature survey with verified citations, but not one of them changed a design decision here. They are context, not input. If we were doing it again we would mine them for *encoding* tricks (several of those papers are interesting precisely because of how they encode a constraint into the representation rather than penalising it) and see whether our AP-position encoding could absorb the separation constraint structurally instead of as a penalty term.
- **The pump-scheduling RL papers** (Hajgató et al. 2020; Hu et al. 2023; Pei et al. 2025) all use deep RL on large water-distribution networks. We deliberately went the other way — small enough that value iteration gives exact ground truth, so every other method can be scored against the true optimum rather than against another approximation. That is a design decision *informed* by them, but it is a decision to do the opposite.
- **Clerc & Kennedy (2002)** — the constriction factor $\chi = 0.7298$ is an alternative to the inertia weight, and Eberhart & Shi (2000) found it performs better on their benchmarks (while still keeping velocity clamping, contrary to a common misreading). We use the inertia weight because it is what the course teaches and what the slide shows. Comparing the two on our landscape is a clean experiment we did not run.
- **Sutton (1990), Dyna** — Dyna interleaves real experience with simulated experience from a learned model, and sits exactly on the boundary our Part B is about. Our certainty-equivalence arm is the batch, one-shot version of the same idea. Dyna-Q is the incremental version and would be the natural third arm.

1.7 7.7 If he asks ”which paper actually changed your mind?”

Two, and say both.

Kennedy & Mendes (2002) turned our communication ablation from a yes/no question into a ”how much, and in what structure” question — and then corrected us when we over-read our own numbers. The mean difference between the ring and the fully connected swarm is not significant. The variance difference is, by six times. We had written ”better” and had to change it to ”steadier”.

Li et al. (2024) with **Agarwal et al. (2020)** gave us the reason our headline result happens at all. We had the measurement — a learned model beats Q-learning by ten times on identical samples — and no explanation. The explanation is a proved sample-complexity separation of $1/(1-\gamma)$, which at our $\gamma = 0.99$ is a factor of a hundred. We also learned, checking it, that the paper we *nearly* cited for this does not support the claim, which is its own lesson about reading past the abstract.

Foundations: Population Methods

Swarm theory

15 July 2026

Contents

1 Part 1 — Deep Foundation & Core Swarm Pillars

Self-contained theory of population-based metaheuristics, written to be defended line-by-line in a viva and to seed the arXiv "Learning Journey" report. Notation is fixed once in §1.1 and reused throughout. Every update equation is attributed to its seminal source.

1.1 1.1 What a population-based metaheuristic *is* (formal template)

1.1.1 1.1.1 The optimization problem

We are given a search space $\mathcal{S}$ and an objective (fitness) function

$$f : \mathcal{S} \to \mathbb{R}, \qquad \text{find } x^\star \in \arg\min_{x \in \mathcal{S}} f(x)$$

(maximization is the sign-flip $f \mapsto -f$; our Lab-3 code maximizes coverage, so we negate mentally where needed). $\mathcal{S}$ may be continuous ($\mathcal{S} \subseteq \mathbb{R}^n$, bounded by a box $[\ell, u]$), combinatorial (permutations, binary strings $\{0,1\}^n$), or mixed. We assume only that f can be **evaluated pointwise** — an *oracle* or *black box*. We do **not** assume f is convex, differentiable, continuous, or even given in closed form.

A **metaheuristic** is a problem-independent, stochastic, iterative strategy that samples $\mathcal{S}$, guided only by the fitness values it has observed, to drive the best-so-far sample toward $x^\star$. A **population-based** metaheuristic maintains not one incumbent but a *set* (population) of μ candidate solutions simultaneously.

1.1.2 1.1.2 State, operators, and the generic template

Let the **population** at iteration t be

$$P_t = \left(x_t^{(1)}, x_t^{(2)}, \ldots, x_t^{(\mu)}\right) \in \mathcal{S}^\mu.$$

Every algorithm in this dossier (GA, PSO, ACO, ABC, FA, GWO, CS, WOA) is an instance of the same three-operator loop:

1. **Evaluate** — compute $f(x_t^{(i)})$ for all i (the only place f is touched; the *budget* is counted here).
2. **Vary** — produce candidate offspring/moves by a stochastic operator $\mathcal{V}$ that reads the current population (and possibly external memory M_t such as pheromone or personal bests):

$$\tilde{P}_t \sim \mathcal{V}(\cdot \mid P_t, M_t).$$

   $\mathcal{V}$ is where the algorithms differ: recombination + mutation (GA), velocity update (PSO/FA), pheromone-biased construction (ACO), neighbour perturbation (ABC), leader-guided steps (GWO), Lévy flights (CS), spiral/encircling (WOA).
3. **Select** — choose the survivors P_{t+1} from $P_t \cup \tilde{P}_t$ by a rule $\mathcal{Q}$ biased toward lower f (fitter):

$$P_{t+1} = \mathcal{Q}(P_t, \tilde{P}_t).$$

```
initialise P_0 ~ Uniform(S),  M_0 = empty
evaluate f(P_0)
for t = 0, 1, 2, ... until stop:
    P~ ← Vary(P_t, M_t)          # stochastic reproduction / movement
    evaluate f(P~)               # <-- one "generation" of budget
    P_{t+1} ← Select(P_t, P~)    # fitness-biased survival
    M_{t+1} ← UpdateMemory(...)  # pheromone / pbest / gbest / leaders
return best-so-far x*
```

The engine is **variation ⊕ selection**: variation *injects entropy* (proposes unseen points), selection *removes entropy* (concentrates probability mass on good regions). Optimization is the controlled tension between the two — this is the exploration/exploitation axis formalized in §1.4.

1.1.3 1.1.3 The Markov-chain view (why this is analyzable)

Because P_{t+1} depends only on (P_t, M_t) and fresh randomness, the sequence of populations is a **Markov chain** on the (finite or measurable) state space $\Omega = \mathcal{S}^\mu \times \mathcal{M}$:

$$\Pr\left(P_{t+1} = q \mid P_t = p, P_{t-1}, \dots\right) = \Pr\left(P_{t+1} = q \mid P_t = p\right) =: K(p, q),$$

where K is the transition kernel induced by $\mathcal{V}$ and $\mathcal{Q}$. For a finite $\mathcal{S}$ (e.g. bitstrings), K is a stochastic matrix and the whole apparatus of Markov-chain theory (irreducibility, ergodicity, absorbing states) applies — this is exactly how convergence is *proved* (§1.6). Two structural facts follow immediately:

- If mutation assigns **strictly positive probability** to every point of $\mathcal{S}$ from every state (global reachability), the chain can never be permanently trapped: $\Pr(\exists\, t : x_t^{(i)} = x^\star) \to 1$. This is the *global-search property*.
- If selection is **elitist** (the best-so-far is never discarded), the best-so-far fitness is a monotone, bounded process and therefore converges (Rudolph 1994; §1.6.2).

1.1.4 1.1.4 Generational vs. steady-state

- **Generational**: the entire population is replaced each iteration (P_{t+1} built from $\tilde{P}_t$). Canonical GA, PSO, WOA, GWO, FA are generational.
- **Steady-state**: only one/few individuals are replaced per step (e.g. replace-worst). ABC is effectively steady-state per food source; ACO updates pheromone incrementally.
- **Elitism** is an orthogonal switch: retain the top-k verbatim. It converts a merely *exploratory* chain into a *convergent* one (§1.6.2) at the cost of selection pressure (§1.5). Our PSO keeps `gbest`, an implicit elitism of size 1.

1.2 1.2 Why and when they beat gradient-based and exact solvers

1.2.1 1.2.1 What gradient methods assume — and when it breaks

Gradient descent, Newton, quasi-Newton (L-BFGS), and conjugate-gradient all update

$$x_{k+1} = x_k - \eta_k \, H_k^{-1} \, \nabla f(x_k)$$

and therefore **require** (i) f differentiable with a computable (or finite-difference-approximable) ∇f, and (ii) local gradient information to be *informative* about the global optimum. They are *local* methods with (super)linear convergence **on convex or locally convex, smooth** landscapes. They fail — provably or practically — when:

- **No gradient exists**: f is discontinuous, piecewise-constant, or defined by a simulation/`max`/threshold. *Our Lab-3 objective* uses $\max_j \mathrm{RSSI}_{ij}$ (a kink), a logistic soft-threshold, and integer **wall-crossing counts** — the last makes f literally piecewise-constant in AP position, so $\nabla f = 0$ almost everywhere and undefined on the kinks. A gradient step has *nothing to descend*.
- **Non-convex / multimodal**: many local minima; gradient descent converges to whichever basin it starts in. Restarts help only $\propto$ (number of basins)$^{-1}$.
- **Ill-conditioned or deceptive**: the gradient points *away* from the global optimum (deceptive functions, §1.7.4).
- **Black-box, expensive, noisy**: finite-difference gradients cost $n{+}1$ evaluations per step and amplify noise.

1.2.2 1.2.2 What exact/deterministic solvers assume — and when it breaks

Exact methods (branch-and-bound, MILP, dynamic programming, exhaustive enumeration, A*/backtracking as in Labs 1–2) return a *provably* optimal solution but with worst-case cost that **explodes**:

- Combinatorial spaces grow factorially/exponentially: k-of-G placement has $\binom{G}{k}$ options; a permutation of n has $n!$. Our Grid-Search baseline is exactly this — a $\binom{25}{3} = 2300$-way enumeration of a *coarse* grid; a grid fine enough to rival a continuous optimizer needs $\binom{G}{k} \to \infty$. This is the **curse of dimensionality** made concrete.
- Continuous global optimization is NP-hard in general; exact solvers need special structure (linearity, convexity, submodularity) that black-box f does not provide.

1.2.3 1.2.3 The regime where metaheuristics win — and where they lose (honest)

Population metaheuristics are the rational choice when **all** of these hold: no reliable gradient; non-convex/multimodal or combinatorial; black-box or simulation-based f; a "good enough, fast enough" near-optimum is acceptable in place of a certificate of optimality; and the per-evaluation cost permits thousands of samples.

They **lose** — and one must say so in a viva — when:

- f is **convex and smooth**: gradient/Newton converge (super)linearly to the *global* optimum; a swarm is slower and gives no certificate.
- f is **low-dimensional and cheap**: dense grid or multi-start local search dominates.
- The problem has **exploitable structure** (LP/MILP/convex/DP/submodular): an exact solver returns a *provably* optimal answer with a bound; a metaheuristic cannot.
- Evaluations are **extremely expensive** ($\sim$minutes each): Bayesian optimization / surrogate models use the budget better than a 30-particle swarm.

Metaheuristics buy **robustness to landscape pathology** at the price of **optimality guarantees**. That trade is the thesis of this lab.

1.3 1.3 The No Free Lunch (NFL) theorem

Source. D. H. Wolpert & W. G. Macready, "No Free Lunch Theorems for Optimization," *IEEE Transactions on Evolutionary Computation* 1(1):67–82, 1997, DOI 10.1109/4235.585893.

Setup. Fix finite $\mathcal{S}$ and finite value set $\mathcal{Y}$. An algorithm a generates a time-ordered sample (trace) $d_m = ((x_1, y_1), \dots, (x_m, y_m))$ of m distinct visited points; let d_m^y be the sequence of *fitness values* seen. Performance is any functional $\Phi(d_m^y)$ (e.g. best value found).

Statement (NFL-1). For any two algorithms a_1, a_2, averaged **uniformly over all possible objective functions** $f : \mathcal{S} \to \mathcal{Y}$,

$$\sum_f \Pr\left(d_m^y \mid f, m, a_1\right) \;=\; \sum_f \Pr\left(d_m^y \mid f, m, a_2\right).$$

Consequently $\sum_f \Phi(d_m^y \mid f, m, a_1) = \sum_f \Phi(d_m^y \mid f, m, a_2)$ for **every** performance measure Φ: *summed over all functions, all non-revisiting algorithms are identical.* Any above-average performance on one class of f is paid for by exactly equal below-average performance on the complementary class. (An analogous NFL holds for supervised learning.)

Why it is true, in one line. The uniform average over all f makes the histogram of unseen fitness values independent of *how* points were chosen; an algorithm's cleverness is about *which point to sample next*, but under the uniform prior the value at any unseen point is equally likely to be anything, so ordering gains nothing.

Implications for algorithm selection.

1. **There is no universally best optimizer.** Claims that "algorithm X dominates" are meaningful only *relative to a function class*. Benchmarking on a biased suite (e.g. only separable, smooth functions) and generalizing is an NFL fallacy.
2. **Performance is bought with matched priors.** An algorithm beats another *iff* its inductive bias matches the structure of the target functions. GWO's leader-hierarchy exploitation wins on smooth low-D redundant landscapes (robot IK, Part 2); PSO's continuous drift wins on our coverage surface. *Match the operator to the landscape.*
3. **Real problems are not uniformly random.** NFL's uniform prior includes mostly incompressible, structureless functions. Real objectives have exploitable regularity (locality, gradient-of-fitness correlation, decomposability). Metaheuristics work *in practice* precisely because they encode weak, broadly-correct priors (nearby solutions have correlated fitness — §1.7.2).
4. **The engineering job is choosing a bias that fits the problem; no algorithm wins everywhere.** This is what justifies the "choose the algorithm to fit the problem" methodology of Part 4.

Caveat (sharpened NFL). Later work (e.g. Igel & Toussaint 2003; Auger & Teytaud 2010) shows NFL holds exactly only for sets of functions **closed under permutation** of the domain; over structured subsets some algorithms *do* dominate. This does not rescue a "best algorithm," but it explains why structure-exploiting heuristics beat blind search on real data.

1.4 1.4 Exploration vs. exploitation (diversification vs. intensification)

1.4.1 1.4.1 Definitions

- **Exploration / diversification**: sampling far from current incumbents to discover new basins of attraction; increases population **diversity** and the chance of escaping local optima; slows refinement.
- **Exploitation / intensification**: concentrating samples around the best-known solutions to refine them; accelerates convergence; risks **premature convergence** to a local optimum if diversity dies first.

Formally, let $D(P_t)$ be a diversity measure, e.g. mean distance to the population centroid $\bar{x}_t$:

$$D(P_t) = \frac{1}{\mu} \sum_{i=1}^{\mu} \left\| x_t^{(i)} - \bar{x}_t \right\|_2 .$$

Healthy search shows $D(P_t)$ **high early** (exploration) decaying **monotonically** to ≈ 0 (exploitation/convergence). *Our Lab-3* `diversity.png` *plots exactly this collapse* — it is empirical evidence the balance was tuned correctly, not left at library defaults.

1.4.2 1.4.2 The balance is a schedule, and every algorithm has a knob

The state of the art controls the balance by **annealing** a single parameter from "explorative" to "exploitative" over the run:

Algorithm	Balance mechanism	Explorative → exploitative
PSO	inertia weight w	w linearly $0.9 \to 0.4$ (Shi & Eberhart 1998); large w = momentum/roaming, small w = local damping
GA	mutation rate p_m, selection pressure	high p_m/low pressure early; anneal p_m down
GWO	coefficient a	a linearly $2 \to 0$; $\lvert A\rvert>1$ diverge (explore), $\lvert A\rvert<1$ converge (exploit)
WOA	a (same) + spiral/encircle switch p	$a : 2 \to 0$; probabilistic switch keeps both modes
ACO	evaporation ρ	high ρ forgets, keeps exploring; low ρ locks in trails
FA	randomization α, absorption γ	anneal α; γ sets attraction range ($\gamma \to 0$ global, $\gamma \to \infty$ local)
CS	discovery prob. p_a, Lévy scale α	Lévy heavy tails give rare long explorative jumps throughout
ABC	`limit` (abandonment)	scouts re-inject exploration when a source stagnates

Our PSO uses the inertia schedule $w_t = w_{\max} - (w_{\max} - w_{\min})\, t/(T-1)$ — the direct answer to "how did you avoid premature convergence?"

1.5 1.5 Diversity maintenance & premature convergence

Premature convergence = the population collapses onto a single (sub-optimal) basin while budget remains; $D(P_t) \to 0$ *before* $x^\star$ is found, after which variation can no longer escape (crossover of near-identical parents is a no-op; PSO velocities $\to 0$). Signs (from the course slides): population becomes uniform; no best-fitness improvement over many generations. Mechanisms that counter it:

- **Mutation / immigration.** Non-zero mutation guarantees global reachability (§1.1.3). Random *immigration* injects fresh individuals when stagnation is detected — a hard reset of diversity.
- **Controlled elitism.** Keep only a *small* elite fraction; retaining too many clones the population.
- **Selection-pressure control.** Tournament size, rank-based instead of fitness-proportional selection (bounds the takeover rate so one super-individual cannot dominate in a few generations).
- **Niching / fitness sharing** (Goldberg & Richardson 1987). Penalize crowding so multiple optima coexist. Shared fitness:

$$f_i' = \frac{f_i}{\sum_{j=1}^{\mu} \mathrm{sh}(d_{ij})}, \qquad \mathrm{sh}(d) = \begin{cases} 1 - (d/\sigma_{\text{share}})^{\alpha}, & d < \sigma_{\text{share}} \\ 0, & d \geq \sigma_{\text{share}} \end{cases}$$

where $d_{ij} = \|x_i - x_j\|$ and σ_{share} is the niche radius. Individuals in a crowd share (dilute) their fitness, so selection spreads them across niches.
- **Crowding / restricted replacement** (De Jong; Deb & Goldberg 1989): offspring replace the *most similar* parent, preserving spatial spread.
- **Velocity clamping / re-init** (PSO): clamp $|v| \leq v_{\max}$ to stop explosion (over-exploration) *and* GCPSO-style re-randomization of `gbest` to stop stagnation (over-exploitation).

Our optimizer relies on: velocity clamping ($v_{\max} = 0.2\times$range), the inertia schedule, and the swarm's own stochastic r_1, r_2; the `diversity.png` curve is the evidence that these keep $D(P_t) > 0$ through the exploration phase.

1.6 1.6 Mathematical definitions of convergence

1.6.1 1.6.1 What "convergence" means

Let $x_t^{\text{best}} = \arg\min_{i,\, s\leq t} f(x_s^{(i)})$ be the best-so-far and $f^\star = f(x^\star)$.

- **Convergence in value (in probability):** $\forall \varepsilon > 0,\ \Pr\left(f(x_t^{\text{best}}) - f^\star > \varepsilon\right) \xrightarrow{t\to\infty} 0.$
- **Almost-sure convergence:** $\Pr\left(\lim_{t\to\infty} f(x_t^{\text{best}}) = f^\star\right) = 1.$
- **Global-search guarantee:** the (weaker, prerequisite) property that every region of positive measure is sampled infinitely often, so $x^\star$ is *reached* with probability 1. Global reachability + elitism $\Rightarrow$ a.s. convergence.

Note the distinction between converging to a **fixed point** (the swarm agrees — *stagnation*, which may be sub-optimal) and converging to the **global optimum** (the object we actually want). Conflating them is a classic viva trap (§Part 3).

1.6.2 1.6.2 Elitist GA converges; canonical GA does not

Source. G. Rudolph, "Convergence Analysis of Canonical Genetic Algorithms," *IEEE Transactions on Neural Networks* 5(1):96–101, 1994, DOI 10.1109/72.265964.

Modeling the GA as a homogeneous Markov chain on $\{0,1\}^{\ell\mu}$, Rudolph proves: the **canonical GA** (fitness-proportional selection + crossover + bit mutation, *no elitism*) does **not** converge to the global optimum — its best-so-far can oscillate because selection can lose the optimum once found. Adding **elitism** (maintain the best individual across generations) yields a chain whose best-so-far is monotone and **converges to the global optimum with probability 1**. Takeaway: *the convergence guarantee comes from elitism + positive mutation, not from crossover.*

1.6.3 1.6.3 PSO can stagnate at a non-optimum

Source. F. van den Bergh, "An Analysis of Particle Swarm Optimizers," PhD thesis, Univ. of Pretoria, 2001; and F. van den Bergh & A. P. Engelbrecht, "A study of particle swarm optimization particle trajectories," *Information Sciences* 176(8):937–971, 2006, DOI 10.1016/j.ins.2005.02.003.

Standard `gbest` PSO is **not** a global (nor even local) optimizer: once all particles coincide with `gbest`, the cognitive and social terms vanish and velocity decays as $v \to 0$, so the swarm halts at `gbest` **regardless of whether it is a local minimum** — pure *stagnation*. The **Guaranteed Convergence PSO (GCPSO)** fix forces the global-best particle to sample a shrinking random neighbourhood, restoring convergence to a local minimizer. Clerc & Kennedy (2002, DOI 10.1109/4235.985692) give the *constriction* analysis that yields stable trajectories via $\chi \approx 0.7298,\ c_1{=}c_2{=}2.05$; the inertia form $w{\in}[0.4, 0.9],\ c_1{=}c_2{\approx}1.49$ we use is its practical equivalent.

The honest position for the viva: PSO offers **no** guarantee of global optimality; we mitigate stagnation empirically (diversity monitoring, multiple seeds, matched-budget baselines, Wilcoxon test) rather than claim a proof we do not have.

1.7 1.7 Topology of the fitness landscape

A landscape is the triple $(\mathcal{S}, \mathcal{N}, f)$ where $\mathcal{N}$ is a neighbourhood/move structure. Its geometry dictates which operator wins (NFL, §1.3).

1.7.1 1.7.1 Modality

The number of local optima. **Unimodal**: one optimum (gradient/hill-climbing suffices). **Multimodal**: many (population search + niching needed). Formally x is a local minimum if $f(x) \leq f(y)\ \forall y \in \mathcal{N}(x)$. Our coverage surface is multimodal: distinct AP configurations (which AP covers which wing) are separate basins of comparable quality.

1.7.2 1.7.2 Ruggedness (fitness correlation)

How fast fitness de-correlates along a walk. Take a random walk $x_0, x_1, \ldots$ over $\mathcal{N}$ and the series $f_t = f(x_t)$; its **autocorrelation** at lag s is

$$\rho(s) = \frac{\mathbb{E}[(f_t - \bar{f})(f_{t+s} - \bar{f})]}{\mathrm{Var}(f)}, \qquad \ell = -\frac{1}{\ln|\rho(1)|}$$

(Weinberger 1990, *Biological Cybernetics* 63:325–336). Large **correlation length** $\ell \Rightarrow$ smooth, "nearby solutions have similar fitness" $\Rightarrow$ local search is informative. Small $\ell \Rightarrow$ rugged, needle-like $\Rightarrow$ near-random, population diversity is essential. Ruggedness is *why* metaheuristics encode the locality prior NFL says you need.

1.7.3 1.7.3 Neutrality

Extended regions of **equal** fitness — *neutral networks* — over which selection is blind and only drift moves the population (Kimura's neutral theory, ported to EC by Barnett, Reidys & Stadler). A landscape with plateaus starves gradient and fitness-proportional selection of signal; the search must *drift* (mutation) across the plateau to its edge. Our integer wall-crossing term creates exactly such plateaus (moving an AP within a shadow-free region does not change the crossing count), so f is locally flat in patches — another reason gradient methods stall and population drift helps.

1.7.4 1.7.4 Deceptiveness

A landscape is **deceptive** (Goldberg 1989, *Genetic Algorithms in Search, Optimization, and Machine Learning*) when low-order statistics point *away* from the global optimum: the average fitness of the schemata (building blocks) containing $x^\star$ is *lower* than that of competing schemata, so selection actively drives the population away from $x^\star$. Deceptive problems break gradient methods *and* naive GAs; they motivate linkage learning and diversity preservation. Most real landscapes are partially deceptive in places.

1.7.5 1.7.5 Why population metaheuristics fit non-convex / non-diff / multimodal / high-D / black-box

- **Non-differentiable**: they use only f values, never ∇f — immune to kinks, `max`, thresholds, integer counts (our objective has all four).
- **Non-convex / multimodal**: a *population* samples many basins in parallel; selection + variation perform implicit basin-hopping; niching keeps optima co-resident.
- **High-dimensional**: no $\binom{G}{k}$ enumeration; the swarm follows fitness-correlation structure (§1.7.2) rather than gridding, so cost scales with iterations, not with $\dim(\mathcal{S})$ exponentially. (This is precisely why our PSO beats Grid Search on the 6-D placement problem under a matched budget.)
- **Black-box**: they need only the oracle; no model, gradient, or convexity.
- **Robust to noise/neutrality**: population averaging and drift tolerate flat/noisy regions that stall single-point deterministic methods.

1.8 1.8 The eight algorithms — mechanics, one by one

Common notation: x = a solution/position; f = objective; t = iteration; $r, r_1, r_2 \sim U(0,1)$ fresh randoms; boldface = vectors.

1.8.1 1.8.1 Genetic Algorithm (GA)

- **Analogy.** Darwinian evolution: a population of chromosomes; fitter individuals reproduce more; crossover recombines parental "genes," mutation introduces novelty; over generations, good building blocks (schemata) proliferate.
- **Core equations.** Fitness-proportional (roulette) selection probability

$$p_i = \frac{f_i}{\sum_{j=1}^{\mu} f_j}.$$

  Single-point crossover exchanges the tail substrings of two parents past a random cut; bit-flip mutation flips each gene independently with probability p_m. The **Schema Theorem** bounds building-block growth:

$$\mathbb{E}[m(H, t{+}1)] \;\geq\; m(H,t)\,\frac{f(H)}{\bar{f}}\Big[1 - p_c\,\frac{\delta(H)}{\ell - 1} - o(H)\,p_m\Big],$$

  where $m(H,t)$ = instances of schema H at gen t, $f(H)$ = mean fitness of H, $\bar{f}$ = population mean fitness, $\delta(H)$ = defining length, $o(H)$ = order, ℓ = string length, p_c = crossover rate. Short, low-order, above-average schemata grow exponentially.
- **Parameters & sensitivity.** Population $\mu \in [30, 100]$; crossover $p_c \in [0.6, 0.95]$; mutation $p_m \in [0.001, 0.01]$ (values > 0.05 usually harmful — devolves to random walk). Selection scheme (tournament/rank) controls takeover speed; encoding choice dominates everything.
- **Best-fit landscape.** Combinatorial / discrete / mixed spaces with meaningful recombination structure (feature selection, scheduling, permutations).
- **Signature strength.** Extreme representational flexibility — encode almost anything (binary, permutation, tree, real vector) and design matched operators.
- **Main limitation.** Premature convergence via diversity loss; performance highly sensitive to encoding and operator design.
- **Seminal cite.** J. H. Holland, *Adaptation in Natural and Artificial Systems*, Univ. of Michigan Press, 1975 (2nd ed. MIT Press 1992, DOI 10.7551/mitpress/1090.001.0001).

1.8.2 1.8.2 Particle Swarm Optimization (PSO)

- **Analogy.** A flock/school foraging: each bird (particle) balances its own memory of the best food it found (`pbest`, nostalgia) against the flock's best (`gbest`, social influence), with momentum (inertia) carrying it forward.
- **Core equations** (inertia form; the course slide's rule):

$$\mathbf{v}_i^{t+1} = w\,\mathbf{v}_i^t + c_1 r_1(\mathbf{p}_i - \mathbf{x}_i^t) + c_2 r_2(\mathbf{g} - \mathbf{x}_i^t), \qquad \mathbf{x}_i^{t+1} = \mathbf{x}_i^t + \mathbf{v}_i^{t+1}.$$

  $\mathbf{v}_i$ = velocity, $\mathbf{x}_i$ = position, $\mathbf{p}_i$ = personal best, $\mathbf{g}$ = global best, w = inertia, c_1 = cognitive (personal) coefficient, c_2 = social coefficient, $r_1, r_2 \sim U(0,1)$. Inertia w was added by Shi & Eberhart 1998 (DOI 10.1109/ICEC.1998.699146); it is absent from the 1995 original.
- **Parameters & sensitivity.** $w \in [0.4, 0.9]$ (often linearly decreasing); c_1,$c_2 \approx 2.0$ (range 0.5–2.5); swarm 20–50; velocity clamp $v_{\max}$. Balance is very sensitive to w and to the c_1:c_2 ratio; w too large $\Rightarrow$ divergence, too small $\Rightarrow$ premature convergence. Clerc–Kennedy constriction (χ=0.7298) is the stability-guaranteed alternative.
- **Best-fit landscape.** Continuous, real-valued, moderately multimodal optimization — *our Wi-Fi placement is a textbook fit*.
- **Signature strength.** Few operators, few parameters, fast empirical convergence on continuous problems; trivially parallel.
- **Main limitation.** Stagnation/velocity collapse at a non-guaranteed point (van den Bergh 2001); weaker on combinatorial spaces without re-encoding.
- **Seminal cite.** J. Kennedy & R. C. Eberhart, "Particle Swarm Optimization," *Proc. IEEE ICNN'95*, vol. 4, pp. 1942–1948, DOI 10.1109/ICNN.1995.488968.

1.8.3 1.8.3 Ant Colony Optimization (ACO)

- **Analogy.** Ants deposit pheromone on trails; shorter routes are traversed faster and reinforced more; evaporation erases stale trails; the colony collectively converges on short paths — *stigmergy* (indirect coordination via the environment).
- **Core equations.** Probabilistic edge choice by ant k at node i:

$$p_{ij}^{k} = \frac{\tau_{ij}^{\alpha}\,\eta_{ij}^{\beta}}{\sum_{l\in\mathcal{N}_i^k}\tau_{il}^{\alpha}\,\eta_{il}^{\beta}}, \qquad \tau_{ij} \leftarrow (1-\rho)\,\tau_{ij} + \sum_{k}\Delta\tau_{ij}^{k}, \quad \Delta\tau_{ij}^{k} = \tfrac{Q}{L_k}.$$

  τ_{ij} = pheromone on edge (i,j), η_{ij} = heuristic desirability ($\approx 1/d_{ij}$), α,β = weight exponents, $\mathcal{N}_i^k$ = feasible next nodes, ρ = evaporation rate, Q = constant, L_k = tour length of ant k.
- **Parameters & sensitivity.** $\alpha \approx 1$, $\beta \in [2,5]$, $\rho \approx 0.5$, ants $m \approx n$, Q. Over-emphasizing τ (large α, small ρ) causes early lock-in; MAX–MIN Ant System (Stützle & Hoos 2000) bounds $\tau \in [\tau_{\min}, \tau_{\max}]$ to prevent it.
- **Best-fit landscape.** Discrete graph/combinatorial construction problems: routing, TSP, scheduling, sequencing.
- **Signature strength.** Constructs solutions incrementally with an explicit *learned* memory (pheromone) — excellent on shortest-path/graph problems and dynamic re-routing.
- **Main limitation.** Pheromone stagnation / premature convergence; sensitive to ρ, α, β; mostly discrete.
- **Seminal cite.** M. Dorigo, V. Maniezzo, A. Colorni, "Ant System: Optimization by a Colony of Cooperating Agents," *IEEE Trans. SMC-B* 26(1):29–41, 1996, DOI 10.1109/3477.484436 (concept: Dorigo PhD thesis, Politecnico di Milano, 1992).

1.8.4 1.8.4 Artificial Bee Colony (ABC)

- **Analogy.** Honeybee foraging with a division of labour: *employed* bees exploit known food sources, *onlookers* probabilistically pick richer sources to exploit further, *scouts* abandon exhausted sources and explore randomly.
- **Core equations.** Employed/onlooker neighbour step and onlooker selection:

$$v_{ij} = x_{ij} + \phi_{ij}\,(x_{ij} - x_{kj}), \quad \phi_{ij} \in [-1,1], \qquad p_i = \frac{\text{fit}_i}{\sum_{n=1}^{SN}\text{fit}_n}.$$

  x_{ij} = dimension j of source i; x_{kj} = a random other source $k \neq i$; ϕ_{ij} = uniform random; SN = number of food sources. A source not improved within `limit` trials is abandoned and re-initialized (scout): $x_{ij} = x_j^{\min} + r\,(x_j^{\max} - x_j^{\min})$.
- **Parameters & sensitivity.** Colony/source count SN; abandonment `limit` (governs explore/exploit — small `limit` over-explores); employed=onlookers=$SN/2$.
- **Best-fit landscape.** Continuous, multimodal numerical optimization.
- **Signature strength.** Strong global exploration via scouts; only one real control parameter (`limit`) beyond size.
- **Main limitation.** Good exploration but *poor exploitation* — slow final convergence near optima (motivating GABC and hybrids).
- **Seminal cite.** D. Karaboga, "An Idea Based on Honey Bee Swarm for Numerical Optimization," Tech. Rep. TR06, Erciyes Univ., 2005; Karaboga & Basturk, *J. Global Optimization* 39(3):459–471, 2007, DOI 10.1007/s10898-007-9149-x.

1.8.5 1.8.5 Firefly Algorithm (FA)

- **Analogy.** Fireflies attract mates by bioluminescence; a dimmer firefly moves toward a brighter one; perceived brightness falls with distance (light absorption), so attraction is local — producing automatic subswarming around multiple optima.
- **Core equations.** Attractiveness and movement:

$$\beta(r) = \beta_0\, e^{-\gamma r^2}, \qquad \mathbf{x}_i \leftarrow \mathbf{x}_i + \beta_0\, e^{-\gamma r_{ij}^2}(\mathbf{x}_j - \mathbf{x}_i) + \alpha\,\varepsilon_i.$$

  β_0 = attractiveness at r=0; γ = light-absorption coefficient; $r_{ij} = \|\mathbf{x}_i - \mathbf{x}_j\|$; α = randomization scale; ε_i = random vector.

- **Parameters & sensitivity.** $\alpha \approx 0.2$ (often annealed); $\beta_0 = 1$; $\gamma \in [0.1, 10]$: $\gamma \to 0$ makes it PSO-like (global), $\gamma \to \infty$ makes it a random walk. γ is the critical knob.
- **Best-fit landscape.** Highly **multimodal** continuous problems where finding *many* optima matters (the local-attraction property partitions the swarm into niches).
- **Signature strength.** Automatic multimodal subdivision — locates multiple optima simultaneously without explicit niching.
- **Main limitation.** Premature convergence/stagnation on high-D problems; $O(\mu^2)$ pairwise comparisons per iteration.
- **Seminal cite.** X.-S. Yang, "Firefly Algorithms for Multimodal Optimization," *SAGA 2009*, LNCS 5792:169–178, DOI 10.1007/978-3-642-04944-6_14 (origin: *Nature-Inspired Metaheuristic Algorithms*, Luniver Press, 2008).

1.8.6 1.8.6 Grey Wolf Optimizer (GWO)

- **Analogy.** Grey-wolf pack hierarchy and hunting: the three best solutions are the α, β, δ wolves; the rest (ω) update their positions by encircling the estimated prey (optimum) as directed by the three leaders.
- **Core equations.**

$$\mathbf{D} = |\mathbf{C} \cdot \mathbf{X}_p - \mathbf{X}|, \qquad \mathbf{X}(t+1) = \mathbf{X}_p - \mathbf{A} \cdot \mathbf{D},$$

  with leader aggregation $\mathbf{X}_1 = \mathbf{X}_\alpha - \mathbf{A}_1\mathbf{D}_\alpha$, $\mathbf{X}_2 = \mathbf{X}_\beta - \mathbf{A}_2\mathbf{D}_\beta$, $\mathbf{X}_3 = \mathbf{X}_\delta - \mathbf{A}_3\mathbf{D}_\delta$, and $\mathbf{X}(t+1) = \frac{1}{3}(\mathbf{X}_1 + \mathbf{X}_2 + \mathbf{X}_3)$. Coefficients $\mathbf{A} = 2\mathbf{a} \cdot \mathbf{r}_1 - \mathbf{a}$, $\mathbf{C} = 2\mathbf{r}_2$, where $\mathbf{a}$ decreases **linearly from 2 to 0** across iterations; $|\mathbf{A}| > 1$ = explore, $|\mathbf{A}| < 1$ = exploit.
- **Parameters & sensitivity.** Population ($\approx$30), max-iterations, and the $\mathbf{a}$ schedule (the sole explore/exploit control). Few parameters $\Rightarrow$ easy to tune, but the linear $\mathbf{a}$ schedule is a rigid prior.
- **Best-fit landscape.** Smooth-ish, low-to-moderate-dimensional, *redundant* continuous problems (e.g. robot inverse kinematics — Part 2) where strong late-stage exploitation pays.
- **Signature strength.** Excellent exploitation with almost no parameters; the three-leader consensus damps premature lock to a single incumbent.
- **Main limitation.** Premature convergence on high-D multimodal problems (no mutation/diversity operator; driven by the leaders).
- **Seminal cite.** S. Mirjalili, S. M. Mirjalili, A. Lewis, "Grey Wolf Optimizer," *Advances in Engineering Software* 69:46–61, 2014, DOI 10.1016/j.advengsoft.2013.12.007.

1.8.7 1.8.7 Cuckoo Search (CS)

- **Analogy.** Brood parasitism: cuckoos lay eggs in host nests; the best eggs (solutions) survive; hosts discover and discard a fraction p_a of alien eggs (abandonment); new nests are sought by **Lévy flights** — heavy-tailed random walks that mix frequent short hops with rare long jumps.
- **Core equations.**

$$\mathbf{x}_i^{t+1} = \mathbf{x}_i^t + \alpha \oplus \text{Lévy}(\lambda), \qquad \text{Lévy}(\lambda) \sim u = s^{-\lambda},\ 1 < \lambda \leq 3,$$

  where α = step-size scaling, $\oplus$ = entrywise product, and a fraction p_a of worst nests is replaced by a biased random walk $\mathbf{x}_i^{t+1} = \mathbf{x}_i^t + r(\mathbf{x}_j^t - \mathbf{x}_k^t)$.
- **Parameters & sensitivity.** Discovery probability $p_a \approx 0.25$ (authors report low sensitivity), Lévy exponent $\lambda \approx 1.5$, nests $n \in [15, 40]$. Notably *few* parameters and robust defaults.
- **Best-fit landscape.** Multimodal continuous global optimization; the Lévy heavy tail gives persistent long-range exploration that resists premature convergence.
- **Signature strength.** Lévy-flight global exploration escapes local optima that Gaussian-step methods miss; very few parameters.
- **Main limitation.** Fixed Lévy step size $\Rightarrow$ slow late convergence on complex/high-D problems.
- **Seminal cite.** X.-S. Yang & S. Deb, "Cuckoo Search via Lévy Flights," *World Congress on Nature & Biologically Inspired Computing (NaBIC 2009)*, IEEE, pp. 210–214, DOI 10.1109/NABIC.2009.5393690.

1.8.8 1.8.8 Whale Optimization Algorithm (WOA)

- **Analogy.** Humpback-whale *bubble-net* feeding: whales encircle prey and swim along a shrinking spiral while blowing bubbles; the algorithm alternates between shrinking-encircle and logarithmic-spiral moves toward the best solution.
- **Core equations.** Encircle vs. spiral, chosen by a coin flip p:

$$\mathbf{X}(t{+}1) = \begin{cases} \mathbf{X}^* - \mathbf{A} \cdot |\mathbf{C}\,\mathbf{X}^* - \mathbf{X}|, & p < 0.5 \\ \mathbf{D}'\, e^{bl} \cos(2\pi l) + \mathbf{X}^*, & p \geq 0.5 \end{cases}$$

  where $\mathbf{X}^*$ = best solution so far, $\mathbf{D}' = |\mathbf{X}^* - \mathbf{X}|$, b = spiral shape constant, $l \in [-1, 1]$, $\mathbf{A} = 2\mathbf{a}\mathbf{r} - \mathbf{a}$, $\mathbf{C} = 2\mathbf{r}$, and $\mathbf{a}$ decreases linearly $2 \to 0$. When $|\mathbf{A}| \geq 1$ the encircle term targets a *random* whale instead of $\mathbf{X}^*$ (exploration).
- **Parameters & sensitivity.** Population, max-iterations, spiral constant b, and the $\mathbf{a}$ schedule; like GWO, very few knobs.
- **Best-fit landscape.** Continuous global optimization; the spiral gives a distinctive exploitation trajectory useful on engineering design problems.
- **Signature strength.** The dual encircle/spiral operator balances explore/exploit with minimal parameters; strong on many engineering benchmarks.
- **Main limitation.** Slow convergence and local-optima trapping on complex high-D problems; diversity loss in late iterations.
- **Seminal cite.** S. Mirjalili & A. Lewis, "The Whale Optimization Algorithm," *Advances in Engineering Software* 95:51–67, 2016, DOI 10.1016/j.advengsoft.2016.01.008.

1.9 1.9 Side-by-side summary table

Algorithm	Inspiration	Core update (essence)	Key parameters	Best-fit landscape	Signature strength	Main limitation	Seminal cite
GA	Darwinian evolution	roulette $p_i{=}f_i/\sum f$; crossover; bit-flip mutation p_m	μ 30–100, p_c 0.6–0.95, p_m 0.001–0.01	discrete/combinatorial/mixed w/ recombination structure	representation flexibility (encode anything)	premature convergence; encoding-sensitive	Holland 1975
PSO	bird flock / fish school	$\mathbf{v}{=}w\mathbf{v}{+}c_1 r_1(\mathbf{p}{-}\mathbf{x}){+}c_2 r_2(\mathbf{g}{-}\mathbf{x})$	$w{\approx}0.4$–0.9, $c_1{,}c_2{\approx}2$, swarm 20–50	continuous, moderately multimodal	few params, fast on continuous, parallel	stagnation, no global guarantee	Kennedy & Eberhart 1995
ACO	ant stigmergy (pheromone)	$p{\propto}\tau^\alpha\eta^\beta$; $\tau{\leftarrow}(1{-}\rho)\tau{+}\sum_k Q/L_k$	$\alpha{\approx}1$, $\beta{\approx}2$–5, $\rho{\approx}0.5$	discrete graph/routing/sequencing	incremental construction w/ learned memory	pheromone stagnation; mostly discrete	Dorigo et al. 1996
ABC	honeybee foraging roles	$v{=}x{+}\phi(x{-}x_k)$; scouts after `limit`	SN, `limit`	continuous multimodal	strong exploration; one main knob	weak exploitation (slow finish)	Karaboga 2005
FA	firefly bioluminescence	$\mathbf{x}{+}{=}\beta_0 e^{-\gamma r^2}(\mathbf{x}_j{-}\mathbf{x}_i){+}\alpha\varepsilon$	$\alpha{\approx}0.2$, $\beta_0{=}1$, γ 0.1–10	highly multimodal (many optima)	automatic multimodal niching	premature convergence; $O(\mu^2)$/iter	Yang 2008/2009
GWO	grey-wolf pack hierarchy	3 leaders $\alpha\beta\delta$; $\mathbf{X}{=}\mathbf{X}_p{-}\mathbf{A}\mathbf{D}$; $a{:}2{\to}0$	pop, iters, a schedule	smooth/reduncant low–mid-D continuous	strong exploitation, few params	stagnation in high-D multimodal premature convergence	Mirjalili et al. 2014

Algorithm	Inspiration	Core update (essence)	Key parameters	Best-fit landscape	Signature strength	Main limitation	Seminal cite
CS	cuckoo brood parasitism	$\mathbf{x} \mathrel{+}= \alpha \oplus$ Lévy(λ); abandon p_a	$p_a \approx 0.25$, $\lambda \approx 1.5$, n 15–40	multimodal continuous global	Lévy long-jumps escape local optima	fixed step $\Rightarrow$ slow finish	Yang & Deb 2009
WOA	humpback bubble-net	encircle/spiral switch; a:2$\rightarrow$0	pop, iters, b, a schedule	continuous engineering-design global	dual operator, minimal params	slow, local-optima trap on high-D	Mirjalili & Lewis 2016

Cross-cutting reading. All eight are the §1.1.2 template with different $(\mathcal{V}, \mathcal{Q}, M)$: GA/ABC use explicit selection + recombination/neighbour variation; PSO/FA/GWO/WOA use *attraction toward memory* (pbest/gbest/brighter/leaders/best) as implicit selection; ACO uses environmental memory (pheromone). All expose exactly one dominant explore/exploit knob (§1.4.2). None escapes NFL (§1.3): each is a *bet* that the target landscape matches its operator's locality/attraction prior — the bet we place, and justify, for our Wi-Fi problem in Part 4.

Foundations: Decision Making

MDP theory

15 July 2026

Contents

1 Part 6 — Deep Foundation & Core Decision-Making Pillars

Self-contained theory of sequential decision-making under uncertainty, written to be defended line-by-line in a viva and to seed the arXiv "Learning Journey" report. Notation is fixed once in §6.1 and reused throughout. Every update equation is attributed to its seminal source. This is the mirror of Part 1: where Part 1 searched a continuous space with a population, this searches a policy space with dynamic programming and with samples.

All numbers quoted are from `src/rl_water_tank.py` and the experiment driver; nothing here is illustrative.

1.1 6.1 What a Markov Decision Process *is* (formal template)

1.1.1 6.1.1 The five-tuple

A finite, discounted, infinite-horizon MDP is

$$\mathcal{M} = (\mathcal{S},\ \mathcal{A},\ P,\ R,\ \gamma)$$

with

- $\mathcal{S}$ — a finite set of **states**. Ours is $s = (L, t, g, F)$: tank level $L \in \{0, \dots, 10\}$ (one unit = 100 L), hour of day $t \in \{0, \dots, 23\}$, grid up/down $g \in \{0, 1\}$, diesel generator-hours remaining $F \in \{0, 1, 2\}$. So $|\mathcal{S}| = 11 \times 24 \times 2 \times 3 = 1584$.
- $\mathcal{A}$ — a finite set of **actions**, here $\{\mathsf{IDLE}, \mathsf{PUMP\_GRID}, \mathsf{PUMP\_GEN}\}$, $|\mathcal{A}| = 3$. Availability is state-dependent: we write $\mathcal{A}(s) \subseteq \mathcal{A}$ and enforce it with a boolean mask (`_build_availability`). `PUMP_GRID` is not offered when $g = 0$ and `PUMP_GEN` is not offered when $F = 0$. That leaves **3432 legal** (s, a) **pairs out of** $1584 \times 3 = 4752$.

- P — the **transition kernel** $P(s' \mid s, a) = \Pr(S_{t+1} = s' \mid S_t = s, A_t = a)$, a conditional distribution over $\mathcal{S}$ for each legal (s, a). We store it as a sparse $(|\mathcal{S}||\mathcal{A}|) \times |\mathcal{S}|$ CSR matrix, row index $s|\mathcal{A}| + a$; each row has at most $(D_{\max}+1) \times 2 = 14$ non-zeros, so the dense array would be about 99.9% zeros.
- R — the **expected reward** $R(s, a) = \mathbb{E}[r_t \mid S_t = s, A_t = a] \in \mathbb{R}$. Ours is a cost, so $R \le 0$ everywhere.
- $\gamma \in [0, 1)$ — the **discount factor**, here 0.99.

The **return** from time t is $G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$, and a (stationary, deterministic) **policy** is a map $\pi : \mathcal{S} \to \mathcal{A}$ with $\pi(s) \in \mathcal{A}(s)$. Its **value function** is

$$V^\pi(s) = \mathbb{E}_\pi\left[\sum_{k=0}^{\infty} \gamma^k r_{t+k} \;\middle|\; S_t = s\right], \qquad Q^\pi(s, a) = \mathbb{E}_\pi\left[\sum_{k=0}^{\infty} \gamma^k r_{t+k} \;\middle|\; S_t = s, A_t = a\right].$$

Because $|r| \le r_{\max}$ (bounded below by $-(c_{\text{gen}} + c_{\text{ovf}} \cdot \text{pump} + c_{\text{short}} \cdot D_{\max}) = -127.5$ in our costs) and $\gamma < 1$, the series converges absolutely: $|V^\pi| \le r_{\max}/(1 - \gamma)$. Boundedness is not decoration; it is what makes every operator below well-defined.

1.1.2 6.1.2 The disturbance form, and why our reward is not $r(s, a, s')$

The textbook writes $r = r(s, a, s')$. Ours cannot be written that way, and saying so is not a defect — it is the more general **disturbance form**

$$s_{t+1} = f(s_t, a_t, w_t), \qquad r_t = r(s_t, a_t, w_t), \qquad w_t \sim W(\cdot \mid s_t),$$

with disturbance $w_t = (D_t, g_{t+1})$: realised hourly demand and the next grid state. The realised shortage is $U_t = \max(0, D_t - \text{level after pumping})$, and once the tank hits zero the successor state s' no longer records *how much* demand went unmet — a demand of 4 and a demand of 6 against an empty tank both land in $L' = 0$. So D_t is not recoverable from s' and r is genuinely a function of the disturbance, not of the successor.

Nothing breaks. Value iteration only ever needs the **expected** reward

$$R(s, a) = \mathbb{E}_w[r(s, a, w)] = -\Big(c_{\text{pump}}(a) + c_{\text{ovf}} \cdot \text{overflow} + c_{\text{short}} \cdot \textstyle\sum_d p_t(d)\ \max(0, d - \ell_a)\Big),$$

which is exactly what `build_model` tabulates (ℓ_a = level after pumping, p_t = the hour-t truncated-Poisson demand pmf). The Bellman operator is still a γ-contraction (§6.3), because the contraction argument never touches the *shape* of the reward, only its boundedness.

Q-learning, by contrast, consumes the **realised** r from `simulate_hour`. It must. If we fed it $\mathbb{E}_w[r]$ we would have leaked the demand distribution into the "model-free" agent and the whole comparison would be a fiction. `simulate_hour` never takes an expectation; `build_model` never takes a sample. Keeping those two faces honest is the single most load-bearing design decision in the code, and there is a Monte-Carlo parity test that estimates P and R back out of the simulator and asserts they match the tabulated model within sampling error.

1.1.3 6.1.3 The Markov property, stated precisely

The state is **Markov** if, for all t and all histories $h_t = (s_0, a_0, r_0, \dots, s_t)$,

$$\Pr\left(S_{t+1} = s',\, r_t = r \;\middle|\; S_t = s_t,\, A_t = a_t,\, S_{t-1}, A_{t-1}, \dots, S_0\right) = \Pr\left(S_{t+1} = s',\, r_t = r \;\middle|\; S_t = s_t,\, A_t = a_t\right).$$

In words: the current state screens off the entire past. Given (s, a), the distribution of what happens next carries no extra information about how you arrived.

Our state satisfies this **by construction, and only because we built it to**:

- Demand D_t depends on the hour, so the hour t is *in* the state. Drop it and D becomes serially informative about itself through the diurnal cycle, and the process is no longer Markov in the remaining variables.
- Load-shedding is a two-state Markov chain $g_t \to g_{t+1}$ with hour-dependent $p_{\text{fail}}, p_{\text{restore}}$. Evening outages are both likely ($p_{\text{fail}} = 0.35$) and long ($p_{\text{restore}} = 0.15$, so $1/0.15 \approx 6.7$ h expected). The *duration* memory is encoded entirely in the binary g — which is exactly what a two-state chain buys you: "the power has been out for three hours" tells you nothing beyond "the power is out".
- The diesel ration F must be in the state because burning fuel at 7am changes what is possible at 9pm. Without F the process has hidden memory and the whole "spend the ration wisely" story is unrepresentable.

This is the **only** thing that licenses both algorithms in this half of the lab. The Bellman equations are a statement about a one-step recursion; they are valid iff the one-step recursion is *complete*, which is precisely the Markov property. Value iteration inherits it directly (the backup conditions on (s, a) and nothing else). Q-learning inherits it too, and less obviously: the TD target $r + \gamma \max_{a'} Q(s', a')$ treats s' as a sufficient statistic for the future, so if s' is *not* Markov, the target is bootstrapping off a value that does not exist and the fixed point Q-learning converges to is not the value of anything. Non-Markov states break model-free learning quietly, which is worse than breaking it loudly.

Adding a variable to the state to restore Markovianity is always possible in principle (take the whole history as the state) and almost never possible in practice (the state space explodes). The engineering skill here is finding the *smallest* sufficient statistic. Ours is (L, t, g, F) at 1584 states.

1.1.4 6.1.4 Continuing, not episodic

The hour wraps: $t' = (t + 1) \bmod 24$. There is no terminal state. The 24-hour "episode" in `q_learning` is a **reporting unit**, not an episode in the MDP sense — the last update of each simulated day bootstraps normally off $\max_{a'} Q(s', a')$ and we never zero the value at the cut. Zeroing it would silently convert the problem into a finite-horizon one, and the Q that converged would not be the infinite-horizon $Q^\star$ that VI computes. The two curves in the report would then be comparing different objects and the comparison would mean nothing.

1.2 6.2 The two Bellman equations, and the difference that actually matters

Source. R. E. Bellman, *Dynamic Programming*, Princeton Univ. Press, 1957; R. A. Howard, *Dynamic Programming and Markov Processes*, MIT Press, 1960. The modern reference treatment is M. L. Puterman, *Markov Decision Processes: Discrete Stochastic Dynamic Programming*, Wiley, 1994, DOI 10.1002/9780470316887.

1.2.1 6.2.1 The expectation equation — linear, so solve it

Fix a policy π. Condition on the first step and use the Markov property:

$$V^\pi(s) = R\big(s, \pi(s)\big) + \gamma \sum_{s'} P\big(s' \mid s, \pi(s)\big)\, V^\pi(s').$$

Collect the $|\mathcal{S}|$ equations into vectors: let $R_\pi \in \mathbb{R}^{|\mathcal{S}|}$ with $(R_\pi)_s = R(s, \pi(s))$, and $P_\pi \in \mathbb{R}^{|\mathcal{S}| \times |\mathcal{S}|}$ with $(P_\pi)_{s,s'} = P(s' \mid s, \pi(s))$. Then

$$V^\pi = R_\pi + \gamma P_\pi V^\pi \qquad \Longleftrightarrow \qquad (I - \gamma P_\pi)\, V^\pi = R_\pi.$$

This is a **linear system in** V^π. P_π is row-stochastic, so its spectral radius is exactly 1, so $\rho(\gamma P_\pi) = \gamma < 1$, so 1 is not an eigenvalue of γP_π, so $(I - \gamma P_\pi)$ is invertible and

$$V^\pi = (I - \gamma P_\pi)^{-1} R_\pi = \sum_{k=0}^{\infty} \gamma^k P_\pi^k R_\pi$$

(the Neumann series, which converges for the same reason). The point that gets missed: because the equation is linear, **you do not have to iterate it**. Iterating $V \leftarrow R_\pi + \gamma P_\pi V$ converges geometrically at rate γ and is what most student code does; solving the sparse system is exact and, at $|\mathcal{S}| = 1584$, faster.

Our `policy_evaluation` does exactly this:

```
P_pi = P[np.arange(S) * A + policy]                    # (S, S) sparse
R_pi = R[np.arange(S), policy]
return spla.spsolve(sp.eye(S, format="csr") - gamma * P_pi, R_pi)
```

One `spsolve`, no loop, no tolerance, no residual. This is the scoring function for **every** policy in the report — value iteration's, Q-learning's, certainty-equivalence's, the myopic control's, and the tuned caretaker heuristic's — and always under the **true** P. That removes Monte-Carlo noise from the comparison completely: policies are ranked by exact expected discounted return, not by which one drew a luckier rollout. Every regret number below is an exact quantity, not an estimate, apart from the seed-to-seed spread of the *learning* runs themselves.

1.2.2 6.2.2 The optimality equation — non-linear, so iterate it

Now do not fix a policy; ask for the best one.

$$V^\star(s) = \max_{a \in \mathcal{A}(s)} \left[R(s,a) + \gamma \sum_{s'} P(s' \mid s,a)\, V^\star(s') \right], \qquad Q^\star(s,a) = R(s,a) + \gamma \sum_{s'} P(s' \mid s,a) \max_{a' \in \mathcal{A}(s')} Q^\star($$

The max is what changes everything. $V \mapsto \max_a[\cdots]$ is not a linear map — it is piecewise-linear and convex — so there is no matrix to invert. There is no closed form. You cannot solve it; you can only find its fixed point, and the standard way is to apply the operator over and over. (There is an exact route: the optimality equation can be written as a linear program with $|\mathcal{S}|$ variables and $|\mathcal{S}||\mathcal{A}|$ constraints, minimise $\sum_s V(s)$ subject to $V(s) \geq R(s,a) + \gamma \sum_{s'} PV(s')$ for all (s,a). It is exact and it is slower than VI at this size, so we do not use it. Mentioning it matters because "non-linear, therefore no exact method" would be false.)

The two equations differ in exactly one symbol and it is the whole difference between **evaluation** and **control**. We use both, for different jobs: the optimality equation to *find* a policy, the expectation equation to *score* every policy on a level field.

1.3 6.3 Why value iteration converges

1.3.1 6.3.1 The Bellman optimality operator and its contraction

Define $T : \mathbb{R}^{|\mathcal{S}|} \to \mathbb{R}^{|\mathcal{S}|}$ by

$$(TV)(s) = \max_{a \in \mathcal{A}(s)} \Big[R(s,a) + \gamma \sum_{s'} P(s' \mid s,a)\, V(s') \Big].$$

Claim. T is a γ-contraction in the sup-norm $\|V\|_\infty = \max_s |V(s)|$:

$$\|TV - TU\|_\infty \;\leq\; \gamma \|V - U\|_\infty \qquad \forall\, V, U \in \mathbb{R}^{|\mathcal{S}|}.$$

Proof. Fix s and let $a^\dagger = \arg\max_a [R(s,a) + \gamma \sum_{s'} P(s' \mid s,a) V(s')]$. Then

$$(TV)(s) - (TU)(s) \;\le\; \gamma \sum_{s'} P(s' \mid s, a^\dagger)\,[V(s') - U(s')] \;\le\; \gamma \sum_{s'} P(s' \mid s, a^\dagger)\,\|V - U\|_\infty \;=\; \gamma \|V - U\|_\infty,$$

where the first inequality is because $(TU)(s) \ge R(s,a^\dagger) + \gamma \sum P(s'|s,a^\dagger)U(s')$ (the max is at least the value at $a^\dagger$), and the last equality uses $\sum_{s'} P(s' \mid s, a^\dagger) = 1$. The argument is symmetric in V, U, so it bounds $|(TV)(s) - (TU)(s)|$; taking the max over s gives the claim. The two ingredients are the *non-expansiveness of* max ($|\max_a f(a) - \max_a g(a)| \le \max_a |f(a) - g(a)|$) and the fact that P is a **probability** distribution — the row sums to one, which is why the discount γ survives undiluted. ■

1.3.2 6.3.2 Banach

$\mathbb{R}^{|\mathcal{S}|}$ with $\|\cdot\|_\infty$ is a complete metric space (it is finite-dimensional). The **Banach fixed-point theorem** (S. Banach, *Fundamenta Mathematicae* 3:133–181, 1922) then gives, in one shot:

1. T has a **unique** fixed point $V^\star$ with $TV^\star = V^\star$.
2. From **any** initialisation V_0, the iterates $V_{k+1} = TV_k$ converge to $V^\star$.
3. The convergence is geometric: $\|V_k - V^\star\|_\infty \le \gamma^k \|V_0 - V^\star\|_\infty$.

That fixed point is the optimal value function, and its greedy policy $\pi^\star(s) = \arg\max_{a \in \mathcal{A}(s)} [R(s,a) + \gamma \sum_{s'} PV^\star]$ is optimal among **all** policies, including history-dependent and randomised ones (Puterman 1994, Thm 6.2.10). That last clause is worth pausing on: we searched only over the $\prod_s |\mathcal{A}(s)|$ stationary deterministic policies, and we get optimality over a vastly larger class for free. It is a consequence of the Markov property, not of the algorithm.

Three things follow that are useful and that people forget:

- **Initialisation does not matter for correctness.** We start at $V_0 = 0$. Any starting point converges. It matters only for speed.
- **Convergence is unconditional in P.** VI converges just as happily on a *wrong* P — it will confidently return the exact optimum of the wrong MDP. §6.9 is about what that costs.
- **$\gamma \to 1$ is the hard regime.** The rate is γ and the error constants below all carry $1/(1-\gamma)$. At $\gamma = 0.99$ that factor is 100.

1.3.3 6.3.3 The stopping bounds — how we *choose* a tolerance instead of guessing

VI cannot see $\|V_k - V^\star\|$ (it does not know $V^\star$). It can see the **Bellman residual** $\varepsilon_k = \|V_{k+1} - V_k\|_\infty$. The two are linked. Write $V^\star - V_{k+1} = \sum_{j \ge 1} (V_{k+j+1} - V_{k+j})$ and apply the contraction to each term:

$$\boxed{\|V_{k+1} - V^\star\|_\infty \;\le\; \frac{\gamma\,\varepsilon}{1-\gamma}} \qquad \text{whenever} \quad \|V_{k+1} - V_k\|_\infty < \varepsilon.$$

And the quantity we actually care about is not the value error but the **loss of the greedy policy** extracted from an inexact V. If $\|V - V^\star\|_\infty \le \delta$ and π_V is greedy w.r.t. V, then (Williams & Baird 1993; S. P. Singh & R. C. Yee, "An Upper Bound on the Loss from Approximate Optimal-Value Functions," *Machine Learning* 16:227–233, 1994, DOI 10.1007/BF00993308):

$$\boxed{\|V^{\pi_V} - V^\star\|_\infty \;\le\; \frac{2\gamma\,\delta}{1-\gamma}}$$

Chaining the two — substituting $\delta = \gamma\varepsilon/(1-\gamma)$ — gives the residual-to-policy-loss bound $\|V^{\pi_k} - V^\star\|_\infty \le 2\gamma^2\varepsilon/(1-\gamma)^2$. (Both boxed forms appear in the literature and they are *not* the same statement: the first has the residual on the right, the second has the value error. Confusing them costs you a factor of $1/(1-\gamma)$, which at $\gamma = 0.99$ is a factor of 100. We keep them separate.)

This is how the tolerance gets chosen, and it is the whole reason to state the bounds. We want the returned policy to be optimal to well inside the precision anyone could care about. We run `value_iteration` at tol $= 10^{-9}$. Then:

$$\|V_k - V^\star\|_\infty \le \frac{0.99 \times 10^{-9}}{0.01} = 9.9 \times 10^{-8}, \qquad \|V^{\pi_k} - V^\star\|_\infty \le \frac{2 \times 0.99^2 \times 10^{-9}}{10^{-4}} \approx 2.0 \times 10^{-5}.$$

Against $|V^\star| = 163.637$ that is a relative policy loss below 1.2×10^{-7} percent. So when we call VI's policy "the exact optimum" and use it as the denominator of every regret number, that is a claim with a certificate behind it, not a hope. Had we picked tol $= 10^{-3}$ "because it looked converged", the greedy-loss bound would only guarantee 2×10^{1} — worthless, on a $V^\star$ of -163.6. The lesson is that at γ near 1, eyeballing the residual curve is not enough; the $1/(1-\gamma)^2$ amplification is real.

1.3.4 6.3.4 What we measured

VI converged in **2273 sweeps in 0.23 s**. The measured contraction rate — the geometric decay rate fitted to the residual sequence ε_k — is **0.9900**, i.e. γ to four decimal places.

That is not a coincidence and it is not a tautology either; it is the theory being tight. The contraction bound says the rate is *at most* γ; whether it is *achieved* depends on the MDP. The rate is asymptotically governed by the sub-dominant structure of $P_{\pi^\star}$, and for a chain that mixes slowly relative to the horizon — ours has a hard 24-hour periodicity and a slow evening outage process — the worst case is essentially attained. A rate strictly below γ would mean the chain forgets faster than the discount does; ours does not. So the observed 0.9900 is a small piece of empirical confirmation that the operator we implemented is the operator we derived, and we report it as such.

The sweep count is also predictable: from $V_0 = 0$ the first residual is $\varepsilon_0 = \max_s \max_a |R(s,a)| \approx 50$ (an empty tank at the evening demand peak with no grid and no fuel), so a rate-γ decay to 10^{-9} needs about $\ln(10^{-9}/50)/\ln(0.99) \approx 2.4 \times 10^3$ sweeps. We observed 2273. The gap is because the early sweeps contract slightly faster than the worst case, which is exactly what one would expect.

1.3.5 6.3.5 Policy iteration, for contrast

Source. Howard 1960 (as above).

Policy iteration alternates: (i) evaluate π exactly by solving $(I - \gamma P_\pi)V^\pi = R_\pi$ — the linear system of §6.2.1 — and (ii) improve, $\pi'(s) = \arg\max_a [R(s,a) + \gamma \sum_{s'} PV^\pi]$. The **policy improvement theorem** says $V^{\pi'} \ge V^\pi$ componentwise, with equality iff π is already optimal. Since there are finitely many deterministic policies and each iteration strictly improves at least one state's value or halts, PI **terminates in finitely many iterations** with the exact optimum — typically a handful, often under ten. VI, by contrast, only converges asymptotically.

We did not use PI as the headline solver, and the honest reason is that VI at 0.23 s was already fast enough that the engineering effort had no payoff. Every ingredient PI needs is in the code anyway (`policy_evaluation` *is* PI's evaluation step). The trade is the usual one: PI does few, expensive iterations ($O(|\mathcal{S}|^3)$ dense, much less sparse); VI does many, cheap ones ($O(|\mathcal{S}||\mathcal{A}| \cdot$ nnz-per-row) per sweep). At γ close to 1, VI's sweep count blows up as $1/(1-\gamma)$ and PI's iteration count barely moves — so at $\gamma = 0.999$ the ranking would likely flip.

1.4 6.4 Model-based vs model-free, stated sharply

The distinction is about **what the algorithm is allowed to read**, and nothing else.

- **Model-based** requires $P(s' \mid s,a)$ and $R(s,a)$ **written down** — an object you can sum over. Value iteration's backup contains the literal term $\sum_{s'} P(s' \mid s,a)V(s')$. That sum is

not computable unless somebody hands you the distribution. In our code that "somebody" is `build_model`, and VI's entire interface to the world is the pair `(P, R)`.
- **Model-free** requires only **samples**: put in (s, a), get back one (s', r). Our `step()` is a four-line function that returns a sampled successor and a realised reward and nothing else. Q-learning calls it and never sees a probability.

That is the whole definition. It is a statement about **information access**, not about algorithm quality, and the sloppy version of this distinction ("model-free is more modern / more general / better") is the single most common error in undergraduate RL writing.

Three consequences that follow immediately and that our experiments demonstrate:

1. **With a correct model, planning wins by construction.** VI returns the exact optimum. No amount of sampling can beat exact. Asking "does Q-learning beat VI?" with a correct P is asking whether an approximation beats the thing it approximates. It does not, and if it appeared to, we would have a bug.
2. The interesting question is therefore not *which wins* but **how wrong the model has to be, and how much experience you have to buy, before learning beats planning.** Dhaka publishes a load-shedding schedule that is famously optimistic. This is not a hypothetical framing device.
3. **Sample access is strictly weaker than model access, but strictly stronger than nothing** — and crucially, samples can be *converted into* a model (§6.9). The moment you notice that, the model-free/model-based line stops being a fence and starts being a dial.

We enforce the fence in code rather than in prose. `HallWaterMDP` has two faces, `build_model()` and `step()`, and the parity test Monte-Carlo-estimates $\hat{P}$ and $\hat{R}$ back out of `step()` and asserts they agree with `build_model()` within sampling error. If that test fails, every VI-vs-QL number in the report is meaningless, so it is the gate the whole experiment hangs on.

1.5 6.5 Q-learning

Source. C. J. C. H. Watkins, *Learning from Delayed Rewards*, PhD thesis, King's College, Cambridge, 1989; C. J. C. H. Watkins & P. Dayan, "Q-learning," *Machine Learning* 8(3–4):279–292, 1992, DOI 10.1007/BF00992698. The TD idea it rests on is R. S. Sutton, "Learning to Predict by the Methods of Temporal Differences," *Machine Learning* 3:9–44, 1988, DOI 10.1007/BF00115009.

1.5.1 6.5.1 The update

$$Q(s,a) \;\leftarrow\; Q(s,a) \;+\; \alpha_n(s,a)\,\Big[\underbrace{r + \gamma \max_{a' \in \mathcal{A}(s')} Q(s',a')}_{\text{TD target}} \;-\; Q(s,a)\Big],$$

where $\alpha_n(s,a) \in (0,1]$ is the step size on the n-th visit to (s,a), and the bracketed quantity is the **TD error** δ_t. Read the update as stochastic approximation: the TD target is a **one-sample unbiased estimate** of the Bellman optimality backup,

$$\mathbb{E}_{s' \sim P(\cdot|s,a),\, r}\Big[r + \gamma \max_{a'} Q(s',a')\Big] = R(s,a) + \gamma \sum_{s'} P(s' \mid s,a) \max_{a'} Q(s',a') = (TQ)(s,a).$$

So Q-learning is Robbins–Monro applied to the fixed-point equation $Q = TQ$ — the same operator whose contraction we proved in §6.3, approached with noisy samples instead of exact expectations. That is the entire idea, and it is why the convergence conditions in §6.6 look like they do: they are the conditions under which the noise averages out fast enough to let the contraction do its work.

1.5.2 6.5.2 Why it is off-policy

The agent **behaves** with ε-greedy: with probability ε it picks uniformly from $\mathcal{A}(s)$, otherwise $\arg\max_a Q(s,a)$. Call that the **behaviour policy** μ. But the target contains $\max_{a'} Q(s',a')$ — the value of the **greedy** action at s', which is the action the *target policy* $\pi = \text{greedy}(Q)$ would take, and which μ may well not take on the next step.

The learner bootstraps off an action it did not (necessarily) execute. That mismatch — target policy $\neq$ behaviour policy — is the definition of **off-policy**, and it is exactly what lets Q-learning converge to $Q^\star$ (the value of the optimal policy) while behaving sub-optimally for the entire duration of training. The exploration is charged to the behaviour policy; the learning is credited to the target policy; the two are decoupled. You could not do this on-policy: you would learn the value of the exploratory policy, which is not what you want.

1.5.3 6.5.3 SARSA, in two sentences

Source. G. A. Rummery & M. Niranjan, "On-line Q-learning using Connectionist Systems," Tech. Rep. CUED/F-INFENG/TR 166, Cambridge Univ., 1994; convergence in S. Singh, T. Jaakkola, M. Littman, C. Szepesvári, "Convergence Results for Single-Step On-Policy Reinforcement-Learning Algorithms," *Machine Learning* 38:287–308, 2000, DOI 10.1023/A:1007678930559.

SARSA replaces the max with the action actually taken: $Q(s,a) \leftarrow Q(s,a) + \alpha[\, r + \gamma\, Q(s',a') - Q(s,a)\,]$ with $a' \sim \mu(\cdot \mid s')$. It is therefore **on-policy** — it learns Q^μ, the value of the ε-greedy policy it is actually running, and converges to $Q^\star$ only if μ is annealed to greedy (GLIE). The practical difference: SARSA accounts for the cost of its own exploration, so on problems where exploring is dangerous (the cliff-walk being the canonical example) it learns a *safer* policy than Q-learning, which happily learns the value of an optimal path it never dares walk. In our hall, exploration costs money but nothing irreversible happens, so the off-policy choice is free and we take it.

1.6 6.6 Convergence conditions for Q-learning

1.6.1 6.6.1 The two conditions

Tabular Q-learning converges to $Q^\star$ with probability 1 provided:

(C1) Robbins–Monro step sizes. For every (s,a),

$$\sum_{n=1}^{\infty} \alpha_n(s,a) = \infty \qquad \text{and} \qquad \sum_{n=1}^{\infty} \alpha_n^2(s,a) < \infty.$$

The first condition says the steps are large enough, in aggregate, to travel any finite distance — the estimate can still move no matter how far it has to go, so an unlucky initialisation is recoverable. The second says they shrink fast enough for the injected noise (variance $\propto \alpha_n^2$) to be summable — so the estimate eventually stops jittering. Both must hold. (H. Robbins & S. Monro, "A Stochastic Approximation Method," *Annals of Mathematical Statistics* 22(3):400–407, 1951, DOI 10.1214/aoms/1177729586.)

(C2) Infinite visitation. Every legal (s,a) pair is visited infinitely often. Not "every state" — every *pair*. The max in the target is an argmax over $\mathcal{A}(s')$, and an action never tried has whatever value we initialised it to; the argmax over a set containing an unexamined element is not an estimate of anything.

Given (C1) and (C2), plus bounded rewards and $\gamma < 1$: $Q_t \to Q^\star$ almost surely.

Sources.

- C. J. C. H. Watkins & P. Dayan, "Q-learning," *Machine Learning* 8:279–292, 1992, DOI 10.1007/BF00992698 — the original proof, via an "action-replay process" construction.
- T. Jaakkola, M. I. Jordan, S. P. Singh, "On the Convergence of Stochastic Iterative Dynamic Programming Algorithms," *Neural Computation* 6(6):1185–1201, 1994, DOI

10.1162/neco.1994.6.6.1185 — the clean stochastic-approximation proof; Q-learning and TD(λ) both fall out of one lemma about contractive stochastic iterations.
- J. N. Tsitsiklis, "Asynchronous Stochastic Approximation and Q-learning," *Machine Learning* 16:185–202, 1994, DOI 10.1007/BF00993306 — handles the **asynchronous** case, which is the one that actually applies: real Q-learning updates one (s, a) per step, not all of them, and different pairs get wildly different numbers of updates. This is the citation that licenses what our code does.
- E. Even-Dar & Y. Mansour, "Learning Rates for Q-Learning," *Journal of Machine Learning Research* 5:1–25, 2003 — moves from "it converges" to **how fast**, as a function of the step-size schedule. They show a linear rate $\alpha_n = 1/n$ can be exponentially slow in $1/(1-\gamma)$, while a **polynomial** rate $\alpha_n = 1/n^\omega$ with $\omega \in (1/2, 1)$ gives polynomial-time convergence. This paper is the reason our default is $\omega = 0.7$ and not $1/n$.

1.6.2 6.6.2 Why a constant α cannot converge to $Q^\star$

Take $\alpha_n \equiv \alpha$. Then $\sum \alpha_n = \infty$ (C1a holds) but $\sum \alpha_n^2 = \sum \alpha^2 = \infty$ (C1b fails). The failure is not a technicality; it is visible in one line of algebra. Unrolling the update,

$$Q_{t+1}(s,a) = (1-\alpha)\, Q_t(s,a) + \alpha\big[r_t + \gamma \max_{a'} Q_t(s',a')\big],$$

the estimate is an **exponentially-weighted moving average** of its recent targets with effective window $\approx 1/\alpha$. It never stops averaging over a *fixed-size* window, so the variance of the noise in the targets never gets averaged away — it is asymptotically driven down only to a floor proportional to α. Formally, Q_t converges in distribution to a random variable concentrated in a **neighbourhood** of $Q^\star$ whose radius is $O\big(\sqrt{\alpha}\,\sigma/(1-\gamma)\big)$ for target-noise scale σ, and it keeps rattling around inside that neighbourhood forever. It does *not* converge to $Q^\star$. It cannot: an estimator that permanently gives weight α to a single fresh noisy sample cannot have vanishing variance.

The trade is real, not a defect: a constant α never forgets how to move, which is precisely what you want in a **non-stationary** environment. Our hall is stationary, so we pay the cost of a constant α for none of its benefit.

We do not merely assert this. The hyperparameter sweep shows it:

step size	final regret vs $V^\star$
polynomial, $\alpha_n = 1/(1+n)^{0.7}$	**25.1%**
constant, $\alpha = 0.1$	**50.4%**

The constant-α run flattens out short of the optimum and stays there; the polynomial one keeps closing. (These sweep runs use a smaller training budget than the headline run in §6.9, so the absolute regrets are higher than the 15.13% quoted there. Only the ordering is the claim.) Our implementation counts visits per (s, a) and sets $\alpha_n = 1/(1 + n(s,a))^{0.7}$, which satisfies (C1) with $\omega = 0.7 \in (0.5, 1]$ and lands in Even-Dar & Mansour's polynomial regime.

1.7 6.7 Exploration vs exploitation in RL

This is the same axis as Part 1 §1.4, transposed. In a metaheuristic, exploration is *sampling far from the incumbent*; in RL it is *taking an action whose value you are not yet sure of*. The tension is identical: you cannot improve an estimate you never sample, and you cannot exploit an estimate you never trust.

The difference is that in RL, exploration has a **cost that is paid in the same currency as the objective** (every exploratory hour of diesel is real money) and, more subtly, exploration is what condition (C2) demands. Exploration in RL is not a heuristic nicety; it is a hypothesis of the convergence theorem.

1.7.1 6.7.1 ε-greedy

With probability ε take a uniformly random legal action; otherwise take $\arg\max_a Q(s,a)$. We anneal ε linearly from 1.00 to 0.05 over the first 60% of training and hold it there. Annealing to *zero* would be wrong for two reasons: it violates (C2) exactly (an action stops being visited), and a small floor is cheap insurance against a Q estimate that got unlucky early. Keeping $\varepsilon \geq 0.05$ is what GLIE conditions formalise for on-policy methods; off-policy Q-learning needs only (C2), which the floor delivers.

ε-greedy is not clever. It explores *uniformly* — as willing to re-try an action it has tried ten thousand times as one it has tried twice. Count-based and optimism-under-uncertainty methods (UCB-style bonuses, R-MAX) explore in proportion to *uncertainty* and have far better sample-complexity bounds. We use ε-greedy because it is what the syllabus specifies and because our state space is small enough that the difference does not decide the experiment. On a larger MDP it would.

1.7.2 6.7.2 Optimistic initialisation — and why $Q_0 = 0$ is optimistic *here*

Initialise $Q_0(s,a)$ above any achievable value. Then every untried action looks better than it is, the greedy step *itself* is exploratory, and the agent systematically works through untried actions until their estimates fall to reality. Optimism converts exploration from a random perturbation into a directed one, for free.

In our MDP every reward is ≤ 0 (they are costs: pump cost, overflow, shortage). So the true $Q^\star(s,a) \leq 0$ everywhere, and initialising $Q_0 = 0$ is **optimistic by construction** — an untried action is quietly credited with the best value physically possible. That is doing real exploration work for us, and pretending otherwise would be taking credit that belongs to the sign of the reward function rather than to the algorithm. So we say so, and we expose `q_init` as a config knob so it can be ablated rather than silently benefiting from it.

1.7.3 6.7.3 Exploring starts

Each episode begins in a state drawn uniformly from $\mathcal{S}$. This is the crudest and most effective route to (C2). Without it, the agent starts every day at (half-full tank, midnight, grid up, full ration) and follows a near-greedy policy; states like *"tank full, 8am, no diesel left"* are then almost unreachable, their Q values never move off their initialisation, and the max over them in the TD target is bootstrapping off a number that means nothing. That poison propagates backwards through every state that can reach them.

Exploring starts buy the coverage that the convergence proof *assumes* and does not provide. The sweep:

exploring starts	final regret
on	**25.1%**
off	**76.7%**

A factor of three. This is the largest single effect in the entire hyperparameter sweep, larger than the step-size effect, and it is not about learning at all — it is about whether the algorithm's stated precondition is satisfied. Our headline run reaches **100% coverage of the 3432 legal** (s,a) **pairs** with a strictly positive minimum visit count; without exploring starts it does not.

The honest caveat: exploring starts require the ability to *reset the simulator into an arbitrary state*, which a real hall would not give you. It is a legitimate use of a simulator and an illegitimate assumption about the world, and any deployment story would have to replace it with something like an ε-floor plus a long enough run, or with directed exploration.

1.8 6.8 The discount factor as an effective horizon

1.8.1 6.8.1 $1/(1-\gamma)$

In $G_t = \sum_k \gamma^k r_{t+k}$ the weight on a reward k steps ahead is γ^k. The weights sum to $\sum_{k\geq 0} \gamma^k = 1/(1-\gamma)$, and their mean lag is $\sum_k k\gamma^k(1-\gamma) = \gamma/(1-\gamma)$. Either way the number $1/(1-\gamma)$ is the natural unit: it is the **effective horizon**, the number of steps the agent meaningfully sees. (A cleaner statement: a discounted infinite-horizon MDP is equivalent to an undiscounted one with a $(1-\gamma)$ per-step termination probability, whose expected length is exactly $1/(1-\gamma)$.)

Our step is **one hour**. So:

γ	$1/(1-\gamma)$	effective lookahead
0	1	1 hour — this hour only
0.9	10	10 hours — under half a day
0.99	**100**	**100 hours — about four days**
0.999	1000	six weeks

We chose $\gamma = 0.99$, and the reason is structural rather than aesthetic: the entire story of this MDP is *"bank cheap grid water in the afternoon against an evening outage you cannot see yet, and do not burn the diesel ration before you need it"*. That reasoning chain spans the 24-hour cycle. An agent with a lookahead shorter than one day literally cannot represent the trade-off, because the consequence it is trading against falls outside its horizon. At $\gamma = 0.99$ the lookahead is 100 hours, comfortably past the 24-hour cycle with margin to spare, and the value of a decision made at 2pm still carries measurable weight at midnight ($0.99^{10} = 0.90$).

The myopic control proves this is not decoration. `policy_myopic` is value iteration at $\gamma = 0$: it maximises this hour's expected reward and ignores the future entirely. That is not a strawman — it is the formally correct greedy policy, the exact optimum of a one-hour horizon. It scores **120.62% regret**: more than twice as costly as optimal. If it had landed near $V^\star$, the problem would have had no long-horizon structure and the whole assignment would have been vacuous. Reporting that gap is how we prove it is not.

For contrast, the tuned two-threshold caretaker rule (pump on grid below θ_g, burn diesel below θ_f, both thresholds grid-searched to give the heuristic its best possible shot) reaches **14.51% regret**. It has the same action set as $\pi^\star$, including the generator; what it cannot do is **read a clock**. It does not know that 6pm is coming. That single missing feature — temporal anticipation, which is precisely what a non-zero γ buys — is worth 14.5% of the objective.

1.8.2 6.8.2 Discounted vs average-reward, honestly

Our task is **continuing**: the hour wraps, there is no terminal state, the hall runs forever. For a continuing task there are two defensible optimality criteria, and they are not the same one:

- **Discounted:** maximise $\mathbb{E}[\sum_k \gamma^k r_k]$. Well-posed for any bounded r; the contraction argument works; the optimal stationary policy exists and is computable. This is what we do.
- **Average-reward (gain-optimal):** maximise $g^\pi = \lim_{T\to\infty} \frac{1}{T}\mathbb{E}[\sum_{k<T} r_k]$. This is arguably the *more natural* objective for a hall that will still be a hall next year. Nobody running a residential building genuinely believes an outage in 2027 matters 0.99^{17520} times less than one today. The discount is a mathematical convenience that we are importing into a problem that did not ask for it.

The honest position, which we state up front in the code and repeat here: **the discounted-optimal policy is not guaranteed to be gain-optimal.** It is guaranteed to be gain-optimal only in the *Blackwell* limit — there exists $\gamma^\star < 1$ such that for all $\gamma \in (\gamma^\star, 1)$ the discounted-optimal policy is also gain-optimal (D. Blackwell, "Discrete Dynamic Programming," *Annals of Mathematical Statistics* 33(2):719–726, 1962) — but $\gamma^\star$ is not known for our MDP and we did not compute it. Average-reward methods (R-learning; see S. Mahadevan, "Average Reward Reinforcement Learning: Foundations, Algorithms, and Empirical Results," *Machine Learning* 22:159–195, 1996, DOI 10.1007/BF00114727) optimise g directly and would be the principled choice.

We chose discounted for three defensible reasons and one weak one. The defensible: (i) it is what the assignment specifies and what the contraction theory in §6.3 requires; (ii) $1/(1-\gamma) = 100$ h genuinely does exceed the horizon on which the problem's structure lives, so we are not truncating anything that matters; (iii) it gives a unique optimal stationary policy without any of the unichain/multichain assumptions that average-reward DP needs. The weak one: it is easier. We sweep γ in the experiments and report the resulting policies rather than asserting insensitivity.

One consequence we are careful about in the plots: we never draw the discounted $V^\star$ line on an average-cost axis. `rollout_stats` reports taka-per-day and dry-hours-per-day because that is what a hall warden asks about, but those are *translations* of the policy into consequences, not the objective the policy was optimised for. They are different optima for different criteria, and overlaying them would be a category error.

1.9 6.9 Certainty equivalence: the bridge, and the headline result

1.9.1 6.9.1 The method

Estimate the model from data by maximum likelihood, then plan on the estimate:

$$\hat{P}(s' \mid s, a) = \frac{N(s, a, s')}{N(s, a)}, \qquad \hat{R}(s, a) = \frac{1}{N(s, a)} \sum_{i=1}^{N(s,a)} r_i,$$

then run value iteration on $(\hat{P}, \hat{R})$ and return its greedy policy. This is **certainty equivalence** — so named because you plan as if your point estimate were certain. (See L. P. Kaelbling, M. L. Littman, A. W. Moore, "Reinforcement Learning: A Survey," *JAIR* 4:237–285, 1996, DOI 10.1613/jair.301, §"model-based methods"; the ideas trace to adaptive control.)

Our `certainty_equivalence` builds $\hat{P}$ and $\hat{R}$ from **the counts Q-learning itself accumulated**, on the same training run, from the same samples, in the same order. Nothing extra is collected. Unvisited (s, a) pairs — of which there are none at full budget, but there are early in training — get a **pessimistic** treatment: a self-loop at the worst reward the MDP can physically produce, $-(c_{\text{gen}} + c_{\text{ovf}} \cdot \text{pump} + c_{\text{short}} \cdot D_{\max}) = -127.5$. That is deliberately the opposite choice from Q-learning's optimistic $Q_0 = 0$: an optimistic *planner* would fall in love with an action it never tried and confidently recommend it, which is the classic way certainty equivalence goes wrong.

1.9.2 6.9.2 The result

On the **same 720,000 samples**:

method	regret vs $V^\star$ (mean ± s.d. over seeds)
Q-learning	**15.13% ± 2.36**
Certainty-equivalence VI	**1.43% ± 0.23**

A **factor of ten**, on identical data. This is the headline finding of the decision-making half, and the explanation is not subtle once you look at where a sample *goes*.

1.9.3 6.9.3 Why the learned model is so much more sample-efficient

Think about what each algorithm does with one transition (s, a, r, s').

Q-learning performs one update, to one cell: $Q(s,a) \mathrel{+}= \alpha[\, r + \gamma \max_{a'} Q(s', a') - Q(s, a)\,]$. Then it throws the sample away. The information in that transition reaches the rest of the table only *later*, and only by diffusion: it must be carried backward one bootstrap hop per visit, through states that themselves have to be re-visited to move. Value propagates through the table at roughly one step of horizon per pass over the data. With an effective horizon of 100 and a γ of 0.99, information has to make a great many hops before it is felt where it matters — and

every hop is re-scaled by a learning rate that is itself decaying. This is why the standard sample-complexity bound for tabular Q-learning carries $1/(1-\gamma)^4$: one factor from the horizon in the value scale, one from the variance, and two more from the sequential propagation.

Certainty equivalence takes the same transition and increments a counter, $N(s,a,s') \mathrel{+}= 1$. That counter is then **read by every single Bellman backup, of every sweep, forever**. VI on $\widehat{P}$ performed 2273 sweeps; each sweep touches every (s,a) row of $\widehat{P}$; so a single observed transition is *re-used thousands of times*, and its implications are propagated to every state that can reach (s,a) — not by diffusion through the data, but by the planner, which is allowed to do arbitrarily much computation on a fixed dataset.

That is the entire asymmetry, and it is a **compute-for-samples trade**. A Q-learning update spends a sample on one cell and discards it. A learned model *stores* the sample as a sufficient statistic and lets planning amortise it across every backup. Samples are expensive (they are hours of a real hall); backups are free (0.23 s). We are trading the cheap resource for the expensive one, and the factor of ten is the exchange rate.

The theory agrees. The minimax sample complexity of learning an ε-optimal policy with a generative model is $\tilde{O}(|\mathcal{S}||\mathcal{A}|/((1-\gamma)^3\varepsilon^2))$, and it is achieved by the **model-based** estimator (M. G. Azar, R. Munos, H. J. Kappen, "Minimax PAC Bounds on the Sample Complexity of Reinforcement Learning with a Generative Model," *Machine Learning* 91:325–349, 2013, DOI 10.1007/s10994-013-5368-1). The corresponding bounds for tabular Q-learning are worse by a factor of $1/(1-\gamma)$ (Even-Dar & Mansour 2003; and see C. Jin, Z. Allen-Zhu, S. Bubeck, M. I. Jordan, "Is Q-Learning Provably Efficient?", *NeurIPS* 2018, for the regret-minimising analogue). At $\gamma = 0.99$ that missing factor is 100. We did not tune our way to a factor of ten; the factor of ten was there in the bounds all along.

1.9.4 6.9.4 What this forces us to conclude — and what it forbids

This baseline is in the experiment precisely because **it could have sunk the thesis**, and it partly did.

Had we run only VI-with-the-true-model against Q-learning, we could have told a clean story: *"the model is wrong in the real world, so model-free wins."* The certainty-equivalence arm makes that story unavailable. If a model **learned from** n **samples** beats Q-learning **trained on the same** n **samples** by a factor of ten, then "model-free beats model-based" is simply the wrong lesson. The right one is narrower and truer:

> A **wrong prior model** loses to a **learned model**. It does not lose to model-free learning. You reach for model-free methods when you cannot even write down the state-transition *structure* to estimate — not merely when your numbers are wrong.

That is a weaker claim than the one we would have liked to make, and it is the one the data supports. Writing the stronger one would have been the easier report and the dishonest one.

1.9.5 6.9.5 How wrong does a prior model have to be?

The mis-specification sweep gives the planner a wrong `outage_scale` (a multiplier on p_{fail} only — "outages happen twice as often" and "outages last twice as long" are different errors with different consequences, so we vary exactly one) and then scores the resulting policy under the **true** model:

prior model	regret vs $V^\star$
outages 1.25× rarer than reality	**0.11%**
outages 4× rarer than reality	**2.66%**
"the grid never fails" ($p_{\text{fail}} = 0$)	**21.39%**
Q-learning, 720k samples	15.13%
certainty-equivalence VI, same 720k samples	1.43%

Read that table carefully, because it is the answer to the question the assignment actually asks.

Planning with a model that is **wrong by a factor of four** still beats Q-learning trained on 720,000 samples (2.66% vs 15.13%). Value iteration is remarkably tolerant of a mis-specified P — a mild

surprise to us, and the reason is that the *ordering* of actions is what determines the policy, and moderate perturbations of P do not reorder the argmax in most states. The greedy operator is a step function; it does not care about small changes in its input.

The prior model only loses when it is **structurally** wrong. A planner told "the grid never fails" does not merely mis-price the outage risk; it **never learns that diesel exists as a hedge**, because in its world the hedge protects against an event of probability zero. That policy scores 21.39% — worse than Q-learning, and worse than the hand-tuned threshold rule. The failure mode is qualitative, not quantitative.

So: how wrong does your model have to be before learning beats planning? Roughly — **wrong enough to have deleted a phenomenon, not merely wrong enough to have mis-measured one.** And even then, the right response is not to abandon models; it is to *learn* one (1.43%).

1.10 6.10 Side-by-side summary table

	Value Iteration	Policy Iteration	Q-learning	SARSA	Certainty Equivalence
What it needs	explicit $P(s' \mid s,a)$, $R(s,a)$	same as VI	samples (s,a,r,s') only	samples (s,a,r,s',a') only	samples, then builds $\hat{P}, \hat{R}$, then needs a planner
What it guarantees	$V \to V^\star$ geometrically at rate γ; unique fixed point (Banach); certified via $\|V_k - V^\star\| \le \gamma\varepsilon/(1-\gamma)$	exact $\pi^\star$ in **finitely many** iterations (monotone improvement)	$Q \to Q^\star$ a.s. under Robbins–Monro + infinite visitation; **off-policy**	$Q \to Q^\mu$; $\to Q^\star$ only if μ is GLIE; **on-policy**	$\hat{P} \to P$ by LLN, so $\pi \to \pi^\star$; no guarantee at finite n (optimism bias)
Sample complexity	zero samples; $O(\text{sweeps} \times \|\mathcal{S}\|\|\mathcal{A}\| \cdot \text{nnz})$ compute. Ours: 2273 sweeps, 0.23 s	zero samples; few iterations, each an $\|\mathcal{S}\|$-dim linear solve	worst among these; bounds carry $1/(1-\gamma)^4$. Ours: 720k steps → **15.13%**	same order as Q-learning, usually a little worse (learns the exploratory policy's value)	minimax-optimal, $\tilde{O}(\|\mathcal{S}\|\|\mathcal{A}\|/((1-\gamma)^3\varepsilon^2))$. Ours: same 720k → **1.43%**
When to use	you have a trustworthy model and $\|\mathcal{S}\|$ fits in memory	same, and γ is close to 1 (VI's sweep count blows up; PI's does not)	you have a simulator or a live system and **cannot write down the transition structure at all**	same, but exploration is *dangerous* and you want the policy to price its own risk	you have samples **and** you can write down the state space — i.e. almost always, in tabular problems
Main limitation	needs P; $O(\|\mathcal{S}\|\|\mathcal{A}\|)$ memory; converges to the exact optimum of **whatever model you gave it**, wrong or not	$O(\|\mathcal{S}\|^3)$ per evaluation if dense; still needs P	sample-hungry; every sample updates one cell then is discarded; needs (C2) coverage, which needs exploring starts or something like them	slower to $Q^\star$; conservative by construction (a feature on cliffs, a bug elsewhere)	needs $O(\|\mathcal{S}\|^2\|\mathcal{A}\|)$ counts (does **not** scale to large $\mathcal{S}$); overconfident on unvisited (s,a) unless you add pessimism, which we do

Cross-cutting reading. All five are the same fixed-point equation approached from different information positions. VI and PI apply the Bellman operator with **exact expectations** because they were handed P. Q-learning and SARSA apply it with **one-sample estimates** because they

were not. Certainty equivalence **reconstructs** enough of P from samples to go back to exact expectations, which is why it dominates in our data. The apparent dichotomy "model-based vs model-free" is really a dial marked *how much of the model do you have, and how much are you willing to estimate*.

1.11 6.11 Where this connects to the swarm half

The two halves of this lab look unrelated — a particle swarm placing Wi-Fi access points, a caretaker deciding when to run a pump — and they are the same problem seen from two angles.

Both are search under a budget. PSO searches a continuous space $\mathcal{S} \subseteq \mathbb{R}^6$ with a population of 30 candidate solutions and a budget counted in **objective-function evaluations**. Value iteration searches the policy space $\prod_s \mathcal{A}(s)$ — combinatorially enormous, 3^{1584} before the availability mask — with dynamic programming and a budget counted in **Bellman backups**. Q-learning searches the same policy space with a budget counted in **environment samples**, which is the most expensive currency of the three. In every case the algorithm is a strategy for spending a finite resource to reduce uncertainty about where the optimum is.

The exploration/exploitation axis is literally the same axis. Part 1 §1.4 defines it in terms of population diversity $D(P_t)$ collapsing over a run; §6.7 defines it in terms of ε annealing and optimistic Q_0. Both are schedules that start broad and narrow. Both fail the same way: PSO stagnates when velocities collapse before $x^\star$ is found; Q-learning's argmax freezes on a badly-estimated action when ε decays before every (s, a) has been sampled. Premature convergence is one phenomenon with two names.

They differ in what structure they exploit, and that difference is the whole reason both are on the syllabus. PSO assumes *fitness locality* — nearby points have correlated fitness (Part 1 §1.7.2) — and nothing else. It cannot use structure it does not have. Value iteration assumes the *Markov property* and, given it, extracts an exact answer with a certificate. When you have the Markov structure, throwing a swarm at the policy space would be absurd: you would be using a method designed for the absence of structure on a problem that has it in abundance. The converse holds too — our Wi-Fi objective has no Markov decomposition, no state, no transition kernel, and dynamic programming has nothing to bite on.

NFL applies to both, and in the same way. Wolpert & Macready (Part 1 §1.3) say that averaged over all objective functions, no optimiser beats another. The RL statement of the same fact: averaged over all MDPs, no learning algorithm beats another. Q-learning's dominance over random search is not a free lunch — it is bought with the assumption that the environment is Markov and stationary, exactly as PSO's dominance over random search is bought with the assumption that fitness is locally correlated. **Each algorithm is a bet on a structural prior, and it wins exactly to the extent that the bet is right.**

Our results are that principle made numerical. VI wins by construction when the Markov model is correct — the bet is exactly right. It degrades gracefully to 2.66% when the model is wrong by 4× — the bet is approximately right. It collapses to 21.39% when told "the grid never fails" — the bet is *structurally* wrong, a phenomenon has been deleted from the prior, and no amount of exact computation on a wrong model recovers it. Q-learning, which bets on almost nothing beyond Markovianity, is never catastrophic and never excellent: 15.13% at 720,000 samples. And certainty equivalence, which bets on the Markov structure but declines to bet on the *numbers*, gets 1.43% — the best of both, because it made the one assumption that is true and estimated the rest.

That is the same lesson Part 1 ends on, arrived at from the other side: **match the prior to the problem, and be honest about which prior you are actually assuming.**

Problem Design

The problems we considered and rejected

15 July 2026

Contents

1 Part 4 — Designing an Original Local Problem

Three original optimization problems drawn from the everyday infrastructure of the University of Dhaka (DU) campus and its residential halls. None is a re-skinned textbook benchmark: each is grounded in a concrete, observable local pain point, but is formalised to the standard required for a rigorous swarm / evolutionary treatment. After specifying all three, we formally select one as the primary student project (Part 5 implements it from scratch).

Notation used throughout: bold lowercase = vectors, $\|\cdot\|$ = Euclidean norm, $\mathbb{1}[\cdot]$ = indicator, $\sigma(z) = 1/(1+e^{-z})$ = logistic function.

1.1 Problem A — Dormitory-Floor Wi-Fi Access-Point Placement (PSO) ★ selected

1.1.1 A.1 Problem statement

Every DU hall resident knows the ”dead-zone” problem: a handful of Wi-Fi routers are mounted along a long hall wing, and rooms far from a router — or shadowed by a concrete partition — get unusable signal. Hall administration has a **fixed, tight budget of K access points (APs)** for a floor and must decide *where on the floor plan to mount them* so that the **occupancy-weighted fraction of rooms with a usable link is maximised**, while respecting a co-channel-interference planning rule (APs on the same channel must not sit too close) and the physical boundary of the floor.

Formally: given a floor $[0, W] \times [0, H]$ (metres), N rooms at fixed positions $\mathbf{r}_i$ with occupancy weights w_i, and interior walls that attenuate signal, choose the continuous coordinates of K APs to maximise weighted coverage.

1.1.2 A.2 Why the landscape is a natural, non-trivial fit for swarm intelligence

- **Continuous and moderate-to-high dimensional.** A solution is a point in $\mathbb{R}^{2K}$. PSO operates natively on continuous vectors — no encoding gymnastics.
- **Non-differentiable.** Coverage uses $\max_j \mathrm{RSSI}_{ij}$ (best-AP selection), a soft threshold, and **discrete wall-crossing counts**. The last makes the objective piecewise-defined with kinks: an AP sliding a few centimetres so that a wall enters/leaves its line-of-sight to a room causes a sudden step. No usable gradient exists.
- **Multimodal.** Many spatially different layouts achieve similar coverage (permuting which AP covers which region), so the objective has many local optima and broad symmetric basins — exactly the regime where a *population* that samples the whole floor beats a single hill-climber.
- **Black-box-friendly.** The objective is cheap to *evaluate* but has no closed form to *invert*; we can only sample it. That is the metaheuristic setting.

Gradient descent has no gradient and would need finite-difference surrogates that the wall kinks corrupt; an exact solver (e.g., MILP after discretising the floor) explodes combinatorially and throws away the continuous structure.

1.1.3 A.3 Recommended algorithm and structural justification

Particle Swarm Optimization (PSO), canonical inertia-weight form (Kennedy & Eberhart 1995; inertia by Shi & Eberhart 1998). Each *particle is literally a candidate AP layout* — a point in $\mathbb{R}^{2K}$ — so the velocity update $\mathbf{v} \leftarrow w\mathbf{v}+c_1 r_1(\mathbf{p}_{\text{best}}-\mathbf{x})+c_2 r_2(\mathbf{g}_{\text{best}}-\mathbf{x})$ moves layouts through the floor in a way that is geometrically meaningful. The swarm's collective spatial search matches the spatial nature of the problem, and inertia decay gives a clean exploration→exploitation schedule. (A real-valued GA is a viable alternative; PSO is preferred for its native continuous geometry and its uniquely *visualisable* swarm — see the selection in §4.4.)

1.1.4 A.4 Complete solution encoding

A particle is the flat vector

$$\mathbf{x} = [a_0^x,\, a_0^y,\, a_1^x,\, a_1^y,\, \ldots,\, a_{K-1}^x,\, a_{K-1}^y] \in \mathbb{R}^{2K},$$

where (a_j^x, a_j^y) are the metre-coordinates of AP j. Bounds: $a_j^x \in [0, W]$, $a_j^y \in [0, H]$. In the reference instance $K = 3$, so the search space is $\mathbb{R}^6$ (a 60 m × 40 m hall block, $N = 40$ rooms).

1.1.5 A.5 Exact fitness function

Signal from AP j to room i uses the **log-distance indoor path-loss model** (2.4 GHz) with discrete wall attenuation:

$$\mathrm{RSSI}_{ij} = P_{\text{tx}} - \big(L_0 + 10\, n \log_{10} \max(d_{ij}, 1)\big) - \gamma\, c_{ij}, \qquad d_{ij} = \|\mathbf{r}_i - \mathbf{a}_j\|,$$

where P_{tx} = AP transmit power (dBm), L_0 = reference loss at 1 m, n = path-loss exponent, γ = attenuation per wall, and $c_{ij} \in \{0, 1, \ldots\}$ is the number of partition walls the segment $\mathbf{r}_i \to \mathbf{a}_j$ crosses. Each room hears its **best** AP, and coverage is a **soft threshold**:

$$\rho_i(\mathbf{x}) = \sigma\!\left(\frac{\max_j \mathrm{RSSI}_{ij} - \tau}{s}\right) \in (0, 1),$$

with usable-link threshold τ (dBm) and softness s. The objective maximised is occupancy-weighted coverage minus a separation penalty:

$$\boxed{F(\mathbf{x}) = \underbrace{100 \cdot \frac{\sum_i w_i\, \rho_i(\mathbf{x})}{\sum_i w_i}}_{\text{weighted coverage \%}} - \lambda_{\text{sep}} \sum_{j<k} \big[\max(0,\; d_{\min} - \|\mathbf{a}_j - \mathbf{a}_k\|)\big]^2}$$

Symbols: w_i = room occupancy (students), τ, s radio constants, $d_{\min}$ = minimum AP separation (co-channel rule), λ_{sep} = penalty weight.

1.1.6 A.6 Real-world constraints and how they are handled

Constraint	Type	Handling
AP inside floor $[0, W] \times [0, H]$	boundary	**Repair**: clamp position, zero the offending velocity component (absorbing wall)
Min AP separation $d_{\min}$	inequality	**Soft penalty** (quadratic breach) inside F
Exactly K APs (budget)	cardinality	**Structural** (fixed vector dimension $2K$)

1.1.7 A.7 Data source / simulation paradigm

A deterministic, seeded synthetic instance built from realistic DU-hall dimensions: rooms on a jittered grid (real rooms are not perfectly aligned), occupancy weights sampled per room, and interior concrete partitions as line segments. Radio constants are standard 2.4 GHz indoor values. The instance is fully reproducible; it can be swapped for a surveyed floor plan + measured occupancy without changing the optimizer.

1.1.8 A.8 Strong baselines

- **Random Search** at the *identical* fitness-evaluation budget.
- **Grid Search**: exhaustive enumeration of AP combinations over a coarse candidate-site grid, within the same budget.

1.1.9 A.9 Concrete validation methodology

15 independent seeded runs; report mean ± std of best fitness; **Wilcoxon signed-rank test** (PSO vs Random, paired by seed) for statistical significance; convergence curve (fitness vs iteration) with baseline reference levels; spatial coverage heatmap with the swarm global-best trajectory; swarm-diversity curve as evidence of the exploration→exploitation transition.

1.2 Problem B — Load-Shedding-Aware Appliance & Study Scheduling in a Dorm Room (GA)

1.2.1 B.1 Problem statement

Dhaka load-shedding is a daily reality. A hall room runs off the grid when it is up and off a **shared IPS / mini-inverter with a hard instantaneous power cap and a limited daily energy budget** when it is down. The resident owns a set of M loads/tasks — ceiling fan, tube light, study lamp, laptop charge, phone charge, electric kettle, router — each with a power draw, a duration or duty requirement, a **utility/priority**, and preferred **time windows** (study lamp in the evening, fan overnight). Given the day's known load-shedding schedule, decide **which loads run in which time slots** to maximise total utility without ever exceeding the instantaneous power cap or the daily energy budget.

1.2.2 B.2 Why the landscape fits a Genetic Algorithm

The decision is a **binary schedule matrix** with coupled, non-linear constraints (a knapsack-over-time / constrained scheduling problem). The feasible region is combinatorial and non-convex; utility is separable but the power/energy caps couple all loads within and across slots. This is the textbook home for a **binary GA** — the exact representation in the course slides (binary chromosome, single-point/uniform crossover, bit-flip mutation).

1.2.3 B.3 Recommended algorithm and structural justification

Genetic Algorithm with binary encoding, tournament selection, uniform crossover, and bit-flip mutation (Holland 1975). Bit-flip mutation toggles a single (load, slot) decision — a semantically

meaningful move; crossover mixes two partial schedules. A constraint-repair operator restores feasibility after variation.

1.2.4 B.4 Complete solution encoding

A binary matrix $A \in \{0,1\}^{M\times T}$ over T time slots (e.g., 48 half-hour slots), $A_{m,t} = 1$ iff load m is energised in slot t; flattened to a chromosome of length MT.

1.2.5 B.5 Exact fitness function

$$F(A) = \sum_{m=1}^{M}\sum_{t=1}^{T} u_m\, A_{m,t}\, \alpha_{m,t} - \lambda_P \sum_t \big[\max(0, \textstyle\sum_m P_m A_{m,t} - P_{\text{cap}}(t))\big]^2 - \lambda_E \big[\max(0, \textstyle\sum_{m,t} P_m A_{m,t}\Delta t -$$

where u_m = per-slot utility of load m, $\alpha_{m,t} \in \{0,1\}$ = availability mask (time-window / grid-up indicator), P_m = power draw (W), $P_{\text{cap}}(t)$ = instantaneous cap in slot t (larger when grid is up), Δt = slot length, E_{day} = daily energy budget, λ_P, λ_E = penalty weights.

1.2.6 B.6 Constraints and handling

Instantaneous power ($\sum_m P_m A_{m,t} \le P_{\text{cap}}(t)$) and daily energy caps as **soft penalties** above; **repair** operator greedily switches off the lowest utility-per-watt load in any over-cap slot; time windows enforced by the mask $\alpha_{m,t}$; minimum contiguous runtime (e.g., kettle) via a penalty on fragmented activations.

1.2.7 B.7 Data source / simulation paradigm

Real appliance wattages + a representative DPDC/DESCO load-shedding block schedule; utilities elicited on a simple priority scale. Fully simulatable.

1.2.8 B.8 Strong baselines

Greedy by utility-per-watt; priority-ordered earliest-feasible scheduling; random search — all at matched evaluation budget.

1.2.9 B.9 Validation methodology

Total utility achieved, feasibility rate, and per-slot load profile (Gantt + power-envelope chart) vs baselines; multi-seed mean ± std; significance test.

1.3 Problem C — Fair Water-Tanker Resupply Routing Across DU Halls (ACO)

1.3.1 C.1 Problem statement

During a WASA supply disruption, a **water tanker of fixed capacity** must resupply the DU residential halls (e.g., Zia, S.M., F.H., Jagannath, Rokeya, Shamsun Nahar, …) from a pump station. Each hall has a demand (litres); the tanker makes one or more capacity-limited trips from the station. Route the tanker to **minimise total travel distance/time** (fuel + delay) while meeting demand and per-trip capacity — a **Capacitated Vehicle Routing Problem (CVRP)**.

1.3.2 C.2 Why the landscape fits Ant Colony Optimization

CVRP is a permutation/graph construction problem over hall-to-hall edges — the canonical home of **ACO**, whose artificial ants *build* routes edge-by-edge and lay pheromone on good edges (Dorigo et al. 1996). Shorter, feasible routes get reinforced; evaporation prevents lock-in. The construction-plus-reinforcement mechanic maps one-to-one onto route building.

1.3.3 C.3 Recommended algorithm and structural justification

Ant System / MAX-MIN Ant System: transition probability $p_{xy}^k = \dfrac{\tau_{xy}^\alpha \, \eta_{xy}^\beta}{\sum_{z \in \mathcal{N}^k} \tau_{xz}^\alpha \eta_{xz}^\beta}$ with $\eta_{xy} = 1/d_{xy}$; pheromone update $\tau_{xy} \leftarrow (1-\rho)\tau_{xy} + \sum_k \Delta\tau_{xy}^k$, $\Delta\tau_{xy}^k = Q/L_k$ for edges on ant k's tour of length L_k.

1.3.4 C.4 Complete solution encoding

An ant's solution is a **giant tour permutation** of halls, split into capacity-feasible sub-routes by a decoder that opens a new trip (returns to the station) whenever the next hall would breach tanker capacity. Pheromone lives on a $(|H|+1) \times (|H|+1)$ hall/station edge matrix.

1.3.5 C.5 Exact fitness / objective

$$\min \; L(\pi) = \sum_{\text{edges } (x,y) \in \text{routes}(\pi)} d_{xy} \; + \; \lambda_{\text{tw}} \sum_h \max(0,\, \text{arr}_h - \text{due}_h)$$

where d_{xy} = road distance, and the second term penalises late arrivals against a due time due_h (optional time windows).

1.3.6 C.6 Constraints and handling

Tanker capacity via **repair in the decoder** (trip closes when full); demand satisfaction is structural (every hall visited once); time windows as **soft penalty**.

1.3.7 C.7 Data source / simulation paradigm

Real DU hall coordinates → road distance matrix (OSM); synthetic per-hall demands. Simulatable end-to-end.

1.3.8 C.8 Strong baselines

Nearest-neighbour construction; Clarke-Wright savings heuristic; random feasible routing.

1.3.9 C.9 Validation methodology

Total distance vs baselines; route map overlay; convergence of best-tour length; multi-seed statistics.

1.4 4.4 Formal selection of the primary project

Selected: Problem A — Dormitory-Floor Wi-Fi AP Placement, solved with PSO.

We score all three on the three criteria the assignment names — Feasibility, Novelty, Demonstrability — plus fit to the course's flagship algorithms.

Criterion	A · Wi-Fi (PSO)	B · Scheduling (GA)	C · Water routing (ACO)
Feasibility (codeable from scratch)	★★★ PSO core ≈40 lines; objective is pure `numpy`; no external data	★★ needs constraint-repair + penalty tuning	★ needs distance matrix, trip decoder, pheromone bookkeeping
Novelty (not a textbook benchmark)	★★★ dorm Wi-Fi dead-zones as continuous max-coverage — fresh, relatable	★★ constrained scheduling is common	★ CVRP is a classic textbook problem

Criterion	A · Wi-Fi (PSO)	B · Scheduling (GA)	C · Water routing (ACO)
Demonstrability (visual, viva-impressive)	★★★ particles *are* 2-D coordinates → live spatial swarm convergence + coverage heatmap	★★ Gantt chart (static)	★★ route map (static)
Course fit	PSO (flagship, continuous)	GA (flagship, binary)	ACO (flagship, discrete)

1.4.1 Why A wins, in detail

- **Feasibility.** PSO from scratch is the smallest, cleanest optimizer of the three: an inertia-weight velocity update and a clamp. The fitness is a handful of vectorised `numpy` lines. There is no external dataset dependency (unlike C, which wants a real road matrix) and no elaborate repair machinery (unlike B). It is the one we are most confident we can implement correctly and defend line by line.
- **Novelty.** "Where do we bolt the routers so the far rooms stop dropping?" is a problem every examiner has personally suffered, yet it is *not* a canned benchmark (no Rastrigin, no plain TSP). It is a genuine continuous facility-location / max-coverage instance grounded in DU hall life. The academic lineage is real and current (e.g., PSO/Cuckoo base-station siting in 5G — see Part 2), which lends the toy the credibility of a research problem.
- **Demonstrability.** This is the decisive edge. Because a particle **is** an (x, y) layout, the swarm can be drawn *on the floor plan*: one watches the global-best APs migrate across iterations and settle into a spread that lights up a coverage heatmap. That directly satisfies the assignment's demand for a "spatial visualization of the swarm coordinates ... over time" in a way a Gantt chart or route map cannot match. Combined with a convergence curve overtaking both baselines and a Wilcoxon-significant margin, it makes an unusually legible, persuasive viva artifact.

Part 5 implements Problem A exactly as specified in §A.4–A.9, from scratch, with matched-budget Random and Grid Search baselines and full statistical validation.

---

2 Part B — Designing the Decision-Making Problem

The swarm half asked *where do we put things*. This half asks *what do we do now, given that we will have to act again in an hour*. Three original sequential decision problems, again drawn from DU campus life, again formalised properly — here as Markov Decision Processes rather than as objective functions. After specifying all three we formally select one, and Part 5 implements it with Value Iteration and Q-learning from scratch.

For each candidate we give: the informal statement; the MDP tuple $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$ with the actual $|\mathcal{S}|$; why the problem is *genuinely* sequential, i.e. an argument that a greedy rule must fail; whether an explicit transition model $P(s' \mid s, a)$ can honestly be written down (this is the question that decides whether Value Iteration is even applicable); baselines; and validation method.

One thing worth stating before the candidates, because it shaped all three: an MDP only rewards lookahead where an action is **irreversible** or a resource is **exhaustible**. If every mistake can be undone next step at a bounded price, the optimal policy collapses towards the greedy one and any planning-versus- learning experiment built on top of it measures nothing. We learned that the hard way; §4.9 is the post-mortem.

2.1 Problem B1 — Overhead Water-Tank Pump Control under Load-Shedding (MDP) ★ selected

2.1.1 B1.1 Informal statement

A DU residential hall stores its water in an overhead tank and refills it with an electric pump. Three things turn this from plumbing into a decision problem. The grid drops out, and it drops out precisely in the evening hours when the hall wants water — and evening outages are *long*: once the power goes it tends to stay gone. A diesel generator can drive the pump through an outage, but the hall buys a **fixed ration of diesel each morning**; burn it at 07:00 on a hunch and there is none left at 21:00 when it actually matters. And demand is not flat — it spikes when the hall bathes, morning and evening.

Every hour the caretaker picks one of three things: leave the pump off, run it on grid power, or burn diesel. Pumping early on cheap grid power banks water against an outage that has not happened yet. Pumping diesel early spends a ration you cannot get back. That is the whole game.

2.1.2 B1.2 The MDP tuple

State $s = (L, t, g, F)$:

Component	Range	Meaning
L	$0 \dots 10$	tank level in 100 L units (1000 L tank)
t	$0 \dots 23$	hour of day — *wraps*; this is a continuing task
g	$\{0, 1\}$	is the grid up?
F	$0, 1, 2$	generator-hours of diesel left today

$$|\mathcal{S}| = 11 \times 24 \times 2 \times 3 = \mathbf{1584}.$$

Actions $\mathcal{A} = \{\text{idle},\ \text{pump/grid},\ \text{pump/diesel}\}$, restricted per state: pump/grid requires $g = 1$, pump/diesel requires $F > 0$. That leaves **3432 legal** (s, a) **pairs out of 4752** — 792 states have the grid down and 528 have no diesel left. The masking is not cosmetic; see §B1.5 and Part 5 §5.7 Q5.

Transition $P(s' \mid s, a)$ factorises into two independent pieces:

- hourly demand $D \sim \text{Poisson}(\lambda_t)$, truncated at 6 and **renormalised** (not clipped — we do not pile the tail onto the top bin), with λ_t following a two-peak daily profile (morning bathing, evening bathing);
- the grid as a two-state Markov chain with hour-dependent $P(\text{fail} \mid \text{up})$ and $P(\text{restore} \mid \text{down})$. Evening is the worst of both: failures are likeliest (0.35) *and* restoration is slowest (0.15, so an expected outage of ~6.7 h).

Dynamics: pump first, then the students draw. $L' = \max(0,\ \min(L + \text{inflow}, 10) - D)$; overflow is charged; the diesel drum is redelivered at midnight and **does not roll over**.

Reward $R(s, a) = -\big(\text{pump cost} + 0.5 \cdot \text{overflow} + 20 \cdot \mathbb{E}_D[\text{unmet}]\big)$, with pump cost 1.0 on grid and 6.0 on diesel. Note that the realised reward depends on the realised demand D, and D is *not* recoverable from s' (once the tank hits zero you cannot tell how much demand went unmet). So r is not a function of (s, a, s') — this is the disturbance form $r = r(s, a, w)$ with $w = (D, g')$. Value Iteration only ever needs $R(s, a) = \mathbb{E}_w[r]$, which is what we tabulate; the Bellman operator is still a γ-contraction. Q-learning must consume the **realised** r, or the demand distribution would have leaked into the supposedly model-free agent.

Discount $\gamma = 0.99$. One step is one hour, so $1/(1 - \gamma) = 100$ h of effective lookahead — comfortably past the 24-hour cycle the whole story depends on. $\gamma = 0$ is kept as the myopic control.

2.1.3 B1.3 Why it is genuinely sequential

The claim ”greedy fails here” is not an assertion we are willing to make for free, so we measure it two ways.

- **The formally correct greedy policy** — value iteration at $\gamma = 0$, i.e. maximise this hour's expected reward and ignore the future entirely — loses **120.62%** of the optimal value. It is not a strawman: it is exactly what ”act myopically” means, done correctly, and it is catastrophic. It never pumps pre-emptively, because pumping always costs something *this* hour and the payoff is entirely in the next few.
- **The best reactive rule** — the two-threshold caretaker policy (grid up and tank below θ_g → pump; grid down and tank below θ_f → burn diesel), with **both thresholds tuned by grid search** so the heuristic gets its best shot — still loses **14.51%**. It has the same action set as the optimal policy, including the generator. What it cannot do is read a clock: it does not know that 18:00 is coming.

The mechanism is visible in a single state. At 14:00, tank 4/10, grid up: pumping costs **0.97 more** this hour than idling, and the optimal policy pumps anyway, because its Q^* advantage is **+4.69**. That means **+5.66 of value comes purely from the future** — from the outage that has not happened yet. A greedy rule cannot see that number; it only sees the 0.97.

2.1.4 B1.4 Can an explicit P be written down?

Yes, exactly. The two stochastic ingredients (demand, grid) are independent and both are small discrete distributions, so each row of P has at most $(6 + 1) \times 2 = 14$ non-zeros. We store P as a sparse $(|\mathcal{S}| \cdot |\mathcal{A}|) \times |\mathcal{S}|$ matrix — a dense $S \times A \times S$ array would be 99.9% zeros. This matters enormously for the project: because a true P exists, we can

1. run Value Iteration and get the *exact* optimum to compare against;
2. score every policy — VI's, Q-learning's, the baselines' — by **exact policy evaluation** under the true model, which removes Monte-Carlo noise from every comparison in the report;
3. deliberately hand the planner a **wrong** P and measure what the wrongness costs.

Point 3 is the reason this problem was worth building at all, and it is not hypothetical: Dhaka publishes a load-shedding schedule that is famously optimistic. A planner that believes the published schedule is a planner with a mis-specified P, and we can quantify exactly how much that belief costs it.

2.1.5 B1.5 Baselines

- **Myopic greedy** ($\gamma = 0$ value iteration) — the formally correct greedy policy, not a hand-crippled one.
- **Tuned caretaker rule** — the reactive two-threshold policy the hall actually runs, with both thresholds grid-searched.
- **Random over legal actions**, and **always idle**, as floors.
- **Certainty-equivalence VI** — build the maximum-likelihood MDP from *Q-learning's own samples* and plan on it. This is the arm that could sink the whole thesis, and we run it deliberately: if a model *learned* from n samples beats Q-learning trained on the same n samples, then ”model-free wins when the model is wrong” is the wrong lesson, and the right one is narrower — a wrong *prior* model loses to a *learned* model.

Action masking is what makes these comparisons mean anything. Without it, idle and pump/grid are bit-for-bit identical in all 792 outage states, so every policy-agreement number would be measuring how two algorithms happen to break ties.

2.1.6 B1.6 Validation methodology

Exact policy evaluation (a sparse linear solve, §5.5) under the **true** model for every policy in the report, reported as **regret %** against V^*; five independent seeds for every learned quantity, mean ± std; a check that Value Iteration's Bellman residuals decay at the rate contraction theory predicts (γ^k); a cross-check that VI's V^* agrees with the linear solve of V^{π^*} to machine precision; Monte-Carlo rollouts *only* to translate discounted return into numbers a hall warden cares about

(cost per day, dry hours per day); and a policy heatmap over (tank level × hour) that makes the pre-filling behaviour directly visible.

2.2 Problem B2 — Batch Cooking and Counter Admission in a Hall Dining Room (MDP)

2.2.1 B2.1 Informal statement

The hall dining room cooks rice and curry in **batches**, and a batch takes about 30 minutes from lighting the stove to being servable. Students do not arrive evenly: they arrive in a wall, twice a day, when classes let out. The manager decides, every ten minutes, whether to start another batch (fuel and labour, spent now, servable in half an hour) and whether to keep taking people into the queue or wave them off to the street stalls. Cook too little and the queue balks and the hall's reputation with it. Cook too much and the surplus is thrown out at closing.

2.2.2 B2.2 The MDP tuple

State $s = (R, p, q, t)$: $R \in 0 \dots 30$ ready portions in units of 5; $p \in \{0, 1, 2\}$ ten-minute slots remaining on the batch currently in the pot ($p = 0$: nothing cooking); $q \in 0 \dots 20$ students in the queue; $t \in 0 \dots 35$ ten-minute slot within a six-hour service window.

$$|\mathcal{S}| = 31 \times 3 \times 21 \times 36 = \mathbf{70{,}308}.$$

Actions $\{\mathsf{wait},\ \mathsf{start\ a\ batch},\ \mathsf{close\ the\ counter}\}$; start a batch is unavailable while $p > 0$ (one pot).

Transition: arrivals per slot Poisson with a slot-dependent rate (two sharp peaks); the server clears a deterministic number of portions per slot; each waiting student balks with a fixed per-slot probability. Independent, discrete, so P is writable.

Reward: goodwill per portion served, minus a balk penalty per student who gives up, minus a waste cost per portion still sitting in the pot at close, minus the fuel/labour cost of each batch started.

Discount: the service window is a natural **episode**, so this problem is honestly finite-horizon and $\gamma = 1$ with backward induction is the right formulation. That is a real structural difference from B1, and it costs us something — see §4.8.

2.2.3 B2.3 Why it is genuinely sequential

The 30-minute cook lead time is the entire argument. A greedy rule ("the queue is long, start cooking") is *structurally* half an hour late: by the time the batch is servable the rush has balked and gone, and the food becomes waste. To serve the 13:00 wall you must light the stove at 12:30, when the dining room is empty and every greedy signal says do nothing. Lookahead is not an optimisation here; it is the only thing that works.

2.2.4 B2.4 Can an explicit P be written down?

Yes, and that is almost a problem. Arrival rates are trivially measurable — you stand at the door with a tally counter for a week — so there is no *natural* source of a badly wrong prior model. We would have to invent one, and an invented mis-specification is a much weaker experiment than B1's, where the wrong model is a real published document that real people really plan against.

2.2.5 B2.5 Baselines

The fixed cook schedule the dining hall actually runs (two batches at fixed times); a reorder-point rule (start a batch whenever $R < r$, r tuned); myopic greedy; random.

2.2.6 B2.6 Validation methodology

Same skeleton as B1 — exact evaluation under the true model, regret against the backward-induction optimum, multi-seed learning curves. The visualisation is the weak point: the state is four-dimensional with no natural pair to hold fixed, so there is no single readable policy picture, only slices.

2.3 Problem B3 — Shuttle-Bus Dispatch on the Campus Loop (MDP)

2.3.1 B3.1 Informal statement

DU runs shuttle buses on a loop through Nilkhet and Shahbagh. Buses idle at the depot; the dispatcher decides every five minutes whether to **hold** a bus (let more riders accumulate, so the trip is cheaper per rider and the crowd at the stops thins out later) or **dispatch** it now (the people waiting now stop waiting, but the bus leaves half empty and the fleet is out of position for the 17:00 rush). Holding is an investment. Dispatching is irreversible for the next forty minutes: the bus is gone.

2.3.2 B3.2 The MDP tuple

State $s = (n_{\text{depot}},\ q_1..q_4,\ \text{ETA}_1..\text{ETA}_3,\ t)$: buses at the depot; queue length at each of four stops, bucketed; time-to-return of each bus en route, binned; five-minute slot over a 14-hour operating day.

Even after bucketing hard — four depot occupancies, six queue levels per stop, twelve ETA bins per bus, 168 slots —

$$|\mathcal{S}| \approx 4 \times 6^4 \times 12^3 \times 168 \approx \mathbf{1.5 \times 10^9},$$

which is not tabulable. Any tractable version requires aggregation so aggressive that the aggregated problem is a different problem.

Actions: $\{\mathsf{hold},\ \mathsf{dispatch}\}$ per idle bus. **Reward**: negative total passenger waiting time, minus fuel per trip, minus a large penalty per rider who gives up and walks.

2.3.3 B3.3 Why it is genuinely sequential

Same shape as the others: the cost of holding is paid now, the benefit is collected later, and a dispatched bus cannot be recalled. A greedy dispatcher ("send a bus whenever anyone is waiting") burns the fleet on empty trips in the quiet hour before the rush and then has nothing at the depot when 300 students appear at once.

2.3.4 B3.4 Can an explicit P be written down?

Not honestly, and this is what kills it. Travel time on the Nilkhet road is heavy-tailed and correlated with the hour, the weather and whatever is happening on the street that day. Arrivals are correlated *across* stops — a class lets out and sixty people materialise simultaneously, which is emphatically not Poisson. We could of course write *some* P down. But it would be a fiction, and the consequence is fatal for this assignment: **with no trustworthy P there is no true model to score policies against**, so there is no exact optimum, no regret axis, and no exact policy evaluation. Every comparison would degrade into Monte-Carlo rollouts with overlapping error bars. Worse, Value Iteration — half of the assignment — could not be run at all except on a model we had invented, which would make the VI-vs-Q-learning comparison a comparison between an algorithm and a fantasy.

2.3.5 B3.5 Baselines

Fixed timetable; headway-based dispatch; load-threshold dispatch ("go when the bus is 70% full or 20 minutes have passed").

2.3.6 B3.6 Validation methodology

Monte-Carlo rollouts only, with all the noise that implies. There is no exact scoring function available, which is exactly the objection above restated.

2.4 4.8 Formal selection of the decision-making project

Selected: Problem B1 — Overhead Water-Tank Pump Control, solved with Value Iteration and Q-learning.

The first two criteria are the assignment's own (Feasibility, Demonstrability). The other two are the ones this half of the assignment actually turns on: does the problem *need* lookahead, and does it let us tell a model-based algorithm apart from a model-free one in a way that means something.

Criterion	B1 · Water tank	B2 · Dining hall	B3 · Shuttle dispatch
Feasibility	★★★ $\lvert\mathcal{S}\rvert = 1584$; sparse P; VI converges in 2273 sweeps in **0.23 s**; a tabular Q-learner covers every legal (s, a)	★★ 70,308 states — VI still runs, but ~45× heavier per sweep, and tabular Q-learning needs far more samples before coverage is honest	★ $\sim 10^9$ states before aggregation; not tabulable, so neither algorithm runs as taught
Needs lookahead	★★★ myopic loses **120.62%**; even the *tuned* reactive rule loses **14.51%**	★★★ the 30-min cook lead time makes any greedy rule structurally late	★★★ holding is an investment; dispatch is irreversible
Demonstrability	★★★ policy is a 2-D picture (tank level × hour): you can *see* the agent pre-fill before the evening outage band	★★ 4-D state with no natural slice; only fragments of a policy are viewable	★ a dispatch log; nothing legible
Separates VI from QL	★★★ exact P exists **and** a genuinely wrong prior exists in the real world (the published load-shedding schedule) — so "how wrong can your model be?" is a real experiment	★ P is writable but trivially measurable, so a wrong-prior story is contrived; and the finite horizon removes the contraction/residual experiment entirely	✗ no honest P → no VI, no exact optimum, no regret axis. The comparison cannot be made

2.4.1 Why B1 wins, in detail

B1 is the only one of the three where **both halves of the assignment are actually runnable and actually comparable**. B3 fails at the first hurdle: a problem whose transition model cannot be written down is a problem where Value Iteration is not applicable, and half the assignment evaporates — it is a model-free problem by default, which is a much less interesting thing to report than a fair fight. B2 is legitimate and we nearly took it; the lead-time argument for sequentiality is arguably even cleaner than B1's. It loses on two counts. Its state is four-dimensional with no natural 2-D slice, so the single most persuasive artifact — a picture of the optimal policy that a professor can read in three seconds — does not exist. And its finite horizon quietly deletes an entire experiment: with backward induction there are no Bellman residuals to plot, no contraction rate to check against γ^k, and no stopping-tolerance bound to justify.

B1 keeps all of it. $\lvert\mathcal{S}\rvert = 1584$ is small enough that Value Iteration returns the exact optimum in 0.23 s, which means every other policy in the report can be scored as a *regret against a known optimum* rather than as a number with no yardstick. The continuing, discounted formulation gives us the residual-decay check ($\gamma = 0.99$, measured empirical contraction rate **0.9900** — theory says exactly γ, and it is exactly γ). The policy is two numbers deep, so the pre-fill behaviour draws itself. And the load-shedding schedule gives us something none of the alternatives do: a

naturally occurring wrong model. We can hand the planner the optimistic published schedule and measure what optimism costs — a planner who believes outages are four times rarer than they are loses 2.66%; one who believes the grid never fails loses 21.39%; one who is only 25% too pessimistic loses 0.11%. That curve is the finding of the whole half, and only B1 could produce it.

2.4.2 4.9 The design failure that made the experiment mean something

This section is here because it is the most valuable thing we learned, and leaving it out would make the report dishonest.

The first version of B1 had an unlimited generator. The state was (L, t, g), $|\mathcal{S}| = 528$, and the diesel action was always available at a cost of 6.0 per hour. Everything ran. Value Iteration converged, Q-learning learned, the plots were drawn, and the model-mismatch experiment — the centrepiece, the one the whole design existed to support — **measured nothing**. The regret curve was essentially flat across every wrong model we handed the planner. A planner who believed the grid never failed scored almost as well as the planner with the true model.

It took us an embarrassingly long time to see why, and the reason is completely elementary once stated. **With unlimited diesel, every mistake is recoverable one hour later at a bounded price.** Suppose the planner's optimistic model tells it not to bother pre-filling, and then the grid drops out at 18:00 with a half-empty tank. So what? It burns diesel. It pays 6.0 an hour, indefinitely, for as long as the outage lasts. The bad forecast cost it the *price difference* between grid and diesel and nothing else. There was no state the agent could get itself into that it could not buy its way out of, and so the value of knowing the future — which is the only thing a correct model buys you — was almost zero. We had built an MDP with no irreversibility in it, and then spent a week measuring how much lookahead was worth in a world where lookahead is worth nothing.

The fix was one state variable. Diesel became a finite daily ration F: two generator-hours, delivered at midnight, **not rolled over**. That is all. The state space went from 528 to 1584. And the problem changed character completely, because F is *exhaustible*: fuel burned at 07:00 on a false alarm is simply not there at 21:00. Now a wrong forecast has a consequence you cannot pay your way out of. Now the optimal policy has to reason about a resource across a whole day, and the model-mismatch experiment has something to measure — because the optimistic planner burns its ration early on outages it thinks will be short, and is holding an empty drum when the real evening outage arrives. The 21.39% figure for ”the grid never fails” exists only because of that ration. In the unlimited version it would have been noise.

(The no-roll-over rule is part of the fix, not a detail: if unused fuel accumulated, the agent could hoard it indefinitely and the constraint would stop binding.)

The lesson generalises, and it is the one we would give anyone designing an MDP for a course project: **a sequential decision problem is only interesting where something cannot be undone.** Irreversibility — an exhaustible resource, a lead time, a bus that has left — is not a complication you add for realism. It is the thing that makes the future worth predicting. Take it out and the optimal policy degenerates towards the greedy one, every planning-versus-learning comparison flattens, and you are left with a working program that answers no question at all.

Applications

Twelve cited real-world uses

15 July 2026

Contents

1 Part 2 — Exceptional Real-World Applications (Literature Review)

Twelve applications of population/swarm metaheuristics from peer-reviewed venues, spanning seven domains. Each entry gives (a) the problem, (b) the algorithm and why it fit, (c) the solution encoding, (d) the fitness/objective, (e) constraint handling, (f) reported results, and (g) the full citation.

Sourcing honesty. Every citation and headline number below was verified against the primary source (or, where noted, the publisher abstract/record). Figures the source did not let us confirm to the digit are tagged `[unverified]`; they are reported as the authors state them, not as our claims. This transparency is deliberate: an arXiv-bound report must not launder unverifiable numbers as fact.

1.1 Index

#	Domain	Problem	Algorithm	Year	Venue
1	Medicine / Bioinformatics	Cancer gene selection	Master–Slave Binary GWO	2021	BioMed Research International
2	Medicine	4D lung-SBRT radiotherapy planning	PSO (virtual-search)	2016	IEEE Trans. Biomed. Eng.
3	Aerospace / Structural	Spacecraft multi-gravity-assist trajectory	Self-adaptive DE	2022	Acta Astronautica

#	Domain	Problem	Algorithm	Year	Venue
4	Aerospace / Structural	Truss topology + sizing	Novelty-driven Binary PSO	2021	arXiv (Assimi et al.)
5	Energy	Distribution-net reconfiguration + DG siting	PSO	2025	PLOS ONE
6	Energy	Wind-farm micro-siting (position + hub height)	PSO	2021	Applied Sciences
7	Robotics	6-DOF welding-arm inverse kinematics	Grey Wolf Optimizer	2022	IJORAS
8	NAS	ImageNet classifier architecture search	Regularized (aging) Evolution	2019	AAAI
9	5G Wireless	Small-cell base-station siting (real field data)	Multi-objective Cuckoo Search + K-means	2023	PeerJ Comp. Sci.
10	Edge Computing	Edge-server placement	Hybrid GA + PSO	2024	PeerJ Comp. Sci.
11	Logistics	Container-terminal berth allocation	Genetic Algorithm	2018	Pesquisa Operacional
12	Logistics	Cold-chain refrigerated vehicle routing	Hybrid Tabu–GWO	2024	PLOS ONE

1.2 1 · Gene selection for cancer classification — Master–Slave Binary GWO

- **(a) Problem.** Select the few informative genes from high-dimensional microarray/biomedical datasets (thousands of features, tens of samples) so a downstream classifier diagnoses cancer accurately without noise from redundant genes.
- **(b) Algorithm + fit.** A **Master–Slave Binary Grey Wolf Optimizer (MSBGWO)**. Feature selection is a binary, high-dimensional, deceptive search (the ”small-n, large-p” regime); GWO's α/β/δ leadership gives strong exploitation, and the master–slave twist (each weak ”slave” wolf learns from an assigned ”master”) injects the diversity plain GWO lacks, escaping the local optima that trap BPSO/BGA here.
- **(c) Encoding.** Binary feature mask of length D = number of genes; bit $= 1$ keeps the gene. Continuous wolf positions are binarised through a sigmoid transfer function.
- **(d) Fitness.** fit $= 1 - \alpha\frac{S}{D} - \alpha\,\overline{\text{Acc}}$ with $\alpha = 0.8$, S = number of selected genes, D = total genes, $\overline{\text{Acc}}$ = mean KNN (k=5) accuracy — jointly rewards accuracy and sparsity.
- **(e) Constraints.** Unconstrained wrapper; feasibility is inherent to the binary mask (any subset is valid).
- **(f) Results.** Accuracy **1.000** on Leukemia and DLBCL, **0.996** on Ovarian, **0.957** on Colon, **0.850** on CNS, while selecting the fewest features; beats BGWO2, BGA, BPSO, DE, and SCA on accuracy, precision, recall, F-measure, and feature count `[verified]`.
- **(g) Citation.** Momanyi, E.; Segera, D. ”A Master-Slave Binary Grey Wolf Optimizer for Optimal Feature Selection in Biomedical Data Classification.” *BioMed Research International*, 2021:5556941. DOI 10.1155/2021/5556941.

1.3 2 · 4D lung-SBRT radiotherapy planning — PSO with a "virtual search" strategy

- **(a) Problem.** In 4D conformal radiotherapy for lung SBRT, choose the beam aperture monitor-unit (MU) weights across all respiratory phases to deliver a lethal tumour dose while sparing healthy tissue.
- **(b) Algorithm + fit. PSO** with a novel constraint-handling ("virtual search") strategy. The dose objective is a high-dimensional, constrained, non-linear function of continuous weights with no clean gradient — PSO searches it directly; the virtual-search normalisation keeps the swarm in the clinically feasible region.
- **(c) Encoding.** A particle is the weight vector $\vartheta = [\vartheta_1, \dots, \vartheta_{N_{\text{ap}}}]$, $0 \le \vartheta_k \le 3.3$; dimension **90–110** apertures for the tested cases.
- **(d) Fitness.** Dose-based MSE $F = \sum_i^{N_{\text{struct}}} \frac{1}{N_{\text{vox},i}} \left(\sum F_{i,f}^{\text{up}} + \sum F_{i,f}^{\text{low}} + F_i^{\max} \right)$ penalising over-, under-, and max-dose voxel violations; dose $\mathbf{D} = \sum_k \vartheta_k \mathbf{d}_k$.
- **(e) Constraints.** Three schemes compared; the proposed one iteratively rescales dose by $NF^t = D_{95}^{\text{presc}} / D_{95}^t$ (feasibility-restoring normalisation), vs a hard penalty of 10^{26} for violating 95 % tumour coverage at 54 Gy.
- **(f) Results.** Across 5 lung-SBRT patients the virtual-search PSO reached the smallest objective values and tightest convergence ranges; a dose-projection baseline found the global optimum in only 2/5 cases and needed ~200 vs 30 iterations `[verified]`.
- **(g) Citation.** Modiri, A.; Gu, X.; Hagan, A.M.; Sawant, A. "Radiotherapy Planning Using an Improved Search Strategy in Particle Swarm Optimization." *IEEE Trans. Biomedical Engineering*, 64(5):980–989, 2016 (landmark clinical reference; ~1 yr outside the 8-yr window). DOI 10.1109/TBME.2016.2585114.

1.4 3 · Deep-space multi-gravity-assist trajectory — Self-adaptive DE

- **(a) Problem.** Design propellant-efficient interplanetary trajectories that chain multiple planetary gravity assists plus deep-space manoeuvres — ESA's GTOP benchmark suite, built precisely because these are severely multimodal.
- **(b) Algorithm + fit. Self-adaptive ("self-learning") Differential Evolution** — mutation strategy and (F, CR) switch mid-run based on recent success, plus re-initialisation of stagnated sub-populations. DE's vector-difference mutation suits the continuous decision variables; the adaptive+restart machinery targets DE's premature-convergence weakness that makes GTOP hard.
- **(c) Encoding.** Real-valued MGA-1DSM vector: launch epoch t_0; hyperbolic excess velocity $[V_\infty, \text{RA}, \text{dec}]$; per-leg times-of-flight; per-flyby swing-by parameters; dimension $\approx 4 + 3(n-1)$ for n planets `[unverified – standard GTOP chromosome, not itemised in the paper text obtained]`.
- **(d) Fitness.** Minimise total mission Δv (equiv. maximise delivered mass), a highly multimodal scalar of the trajectory vector `[unverified exact form]`.
- **(e) Constraints.** Box bounds (launch window, TOF ranges, min flyby altitude); continuity satisfied by the encoding; explicit re-initialisation to escape local optima.
- **(f) Results.** Solved 6 well-known ESA GTOP problems; matched the known best on **5 of 6** and set a **new best-known on 4 of 6** `[verified headline; per-problem delta-v paywalled, unverified]`.
- **(g) Citation.** Choi, J.H.; Lee, J.; Park, C. "Deep-space trajectory optimizations using differential evolution with self-learning." *Acta Astronautica*, 191:258–269, 2022. DOI 10.1016/j.actaastro.2021.11.014.

1.5 4 · Truss topology + sizing — Novelty-driven Binary PSO

- **(a) Problem.** Minimum-weight design of trusses, choosing both topology (which members exist) and sizing (discrete cross-sections) — a combinatorial, multimodal structural-optimisation problem.
- **(b) Algorithm + fit. Novelty-driven Binary PSO** in a bilevel scheme: the upper level uses binary PSO to *discover diverse topologies by maximising novelty* (not just fitness), the lower level does discrete member sizing. The novelty drive combats the premature convergence that plagues weight-only search on this deceptive landscape.

- **(c) Encoding.** Binary particle at the upper level (member presence); discrete-catalogue section indices at the lower level.
- **(d) Fitness.** Minimise structural weight subject to stress/displacement limits; the upper level additionally maximises a novelty metric over the archive of found designs.
- **(e) Constraints.** Stress and nodal-displacement limits (standard penalty/repair in truss BPSO).
- **(f) Results.** Authors report the method "outperforms the current state-of-the-art" and returns *multiple* high-quality designs `[verified qualitative claim; the abstract obtained lists no per-truss weights]`.
- **(g) Citation.** Assimi, H.; Neumann, F.; Wagner, M.; Li, X. "Novelty-Driven Binary Particle Swarm Optimisation for Truss Optimisation Problems." arXiv:2112.07875, 2021.

1.6 5 · Distribution-network reconfiguration + DG siting — PSO (real Ethiopian feeder)

- **(a) Problem.** Jointly choose tie-switch states (reconfiguration) and the location/size of distributed generation (DG) on the **real Wolaita Sodo MV feeder, Ethiopia** (114 transformers, ≈27 MVA) to cut losses and fix severe under-voltage — not a toy IEEE 33/69-bus feeder.
- **(b) Algorithm + fit. PSO**. The search mixes a combinatorial part (switch states that must keep the feeder radial — a discontinuous graph constraint) with a continuous part (DG MW), evaluated through a backward/forward-sweep load flow that jumps discontinuously on topology change. No usable gradient exists; PSO searches switch+DG combinations directly.
- **(c) Encoding.** Each particle encodes 4 tie-switch open/closed assignments (restricted to radial/fundamental-loop combinations) plus DG bus index + MW rating; swarm $n = 20$, $I_{\max} = 20$.
- **(d) Fitness.** Weighted sum $F = w_1 \Delta P_{\text{loss}} + w_2 \Delta Q_{\text{loss}} + w_3$ VD (real loss $\sum I^2 R$, reactive loss $\sum I^2 X$, voltage-deviation index).
- **(e) Constraints.** Radiality enforced by *restricting the search space* to spanning-tree combinations (encoding-level repair, not a penalty); voltage/ current/DG-size limits checked as hard feasibility in the load flow.
- **(f) Results.** Combined reconfig+DG cut active loss **1631.10 → 455.66 kW (−72.06 %)**, raised min voltage **0.7537 → 0.9550 p.u.**, cut max deviation **24.63 % → 4.5 %**, annual energy loss **16.669 → 4.164 GWh**, ≈16.40 M ETB/yr savings on a 22.07 M ETB investment (~6-yr payback) `[verified]`.
- **(g) Citation.** Alemayehu, B.; Mishra, S.; Tejani, G.G.; Tripathi, S. "Network reconfiguration and DG based compensation of Wolaita Sodo distribution system by using particle swarm optimisation." *PLOS ONE*, 20(10):e0335512, 2025. DOI 10.1371/journal.pone.0335512.

1.7 6 · Wind-farm micro-siting with variable hub height — PSO (real Manjil site)

- **(a) Problem.** Re-optimise the layout of Iran's **Manjil/Rudbar** wind complex by jointly choosing each turbine's (x, y) position **and** hub height, using the site's real wind regime, to cut wake losses and cost of energy.
- **(b) Algorithm + fit. PSO**. Jensen wake interactions make even the 2D layout multimodal; adding hub height as a third variable means a taller turbine can clear a shorter one's wake, so the optimum changes *discontinuously* as heights cross wake cones — a non-smooth landscape ideal for derivative-free swarm search.
- **(c) Encoding.** Per turbine a triple (x_i, y_i, h_i) — planar coordinates + hub height from a discrete menu $\{40, 50, 60\}$ m; $\approx 3 \times 21 = 63$-D for the 21-turbine case `[dimension is our derivation, unverified]`.
- **(d) Fitness.** Minimise cost of energy $CoE = (C_{\text{cap}}(h) + C_{\text{O\&M}})/AEP(x, y, h)$, with AEP from the wind-rose-weighted power curve minus pairwise Jensen wake deficits.
- **(e) Constraints.** Min inter-turbine spacing and site-boundary limits; h_i restricted to the discrete menu; infeasible layouts penalised `[penalty coefficients unverified]`.
- **(f) Results.** Optimised layout **+10.75 %** power and **−9.42 %** levelised cost vs baseline; the mixed-height 21-turbine config (2×60 m, 4×50 m, rest 40 m) produced 7.75 MW `[verified`

`headline; granular AEP/$MWh omitted as unverifiable]`.

- **(g) Citation.** Yeghikian, M.; Ahmadi, A.; Dashti, R.; Esmaeilion, F.; Mahmoudan, A.; Hoseinzadeh, S.; Garcia, D.A. "Wind Farm Layout Optimization with Different Hub Heights in Manjil Wind Farm Using PSO." *Applied Sciences*, 11(20):9746, 2021. DOI 10.3390/app11209746.

1.8 7 · 6-DOF welding-arm inverse kinematics — Grey Wolf Optimizer

- **(a) Problem.** Solve the inverse kinematics of a new 6-DOF revolute arm for automated oil-and-gas pipeline welding: find the six joint angles driving the end-effector along a rectangular weld path — analytically messy because many joint combinations reach the same pose.
- **(b) Algorithm + fit. GWO** (vs PSO/WOA/JFO and an improved I-GWO). On the smooth-but-redundant, low-dimensional, box-constrained IK landscape, GWO's α/β/δ exploitation plus its $a : 2 \to 0$ decay converge far more reliably than the competitors.
- **(c) Encoding.** Continuous 6-D angle vector $[\theta_1, \dots, \theta_6]$, box-bounded per Denavit–Hartenberg limits; population 50, 10 iterations, solved per waypoint over 6 waypoints.
- **(d) Fitness.** Minimise position MSE $= \frac{1}{N} \sum_{i \in \{x,y,z\}} (P_i^{\text{ref}} - P_i)^2$ (forward-kinematics vs reference).
- **(e) Constraints.** Hard box-clipping of each joint angle to its DH range during updates.
- **(f) Results.** GWO per-waypoint MSE **8.4×10^{-7}–6.5×10^{-5}**; I-GWO best (**1.6×10^{-7}–9.8×10^{-7}**); PSO 2–4 orders worse (**1.7×10^{-3}–3.4×10^{-3}**); only GWO/I-GWO "effectively solved" the task `[verified]`.
- **(g) Citation.** Nyong-Bassey, B.E.; Epemu, A.M. "Inverse Kinematics Analysis of Novel 6-DOF Robotic Arm Manipulator for Oil and Gas Welding Using Meta-Heuristic Algorithms." *IJORAS*, 4:13–22, 2022. DOI 10.33093/ijoras.2022.4.3.

1.9 8 · ImageNet architecture search — Regularized (aging) Evolution → AmoebaNet

- **(a) Problem.** Automatically discover a convolutional image classifier that beats the best human- and reinforcement-learning-designed architectures on ImageNet under a fixed search budget.
- **(b) Algorithm + fit. Regularized ("aging") Evolution** — a tournament-selection GA that each cycle discards the *oldest* member (not the worst), biasing survival toward genotypes with sustained recent success. Suits the huge, discrete, non-differentiable architecture space and, at equal compute, beat RL controllers — a genuinely non-obvious result.
- **(c) Encoding.** Two cell types (normal, reduction), each a sequence of 5 blocks; each block picks 2 hidden states + one op each from a discrete op set; network = stacked cells. Genome = discrete (state, state, op, op) tuples.
- **(d) Fitness.** Validation top-1 accuracy after a truncated training proxy; population $P = 100$, tournament sample $S = 25$, aging removal.
- **(e) Constraints.** No penalty — structural validity guaranteed by the mutation operator (one pointer/op changed at a time); age-based culling is the regulariser.
- **(f) Results.** AmoebaNet-A **82.8 %** top-1 / **96.1 %** top-5 at 86.7 M params (≈ NASNet-A RL 82.7/96.2 at 88.9 M); scaled to 469 M → **83.9 % / 96.6 %** ImageNet SOTA `[verified]`; the "evolution beats RL faster early" claim `[unverified – no exact multiplier]`.
- **(g) Citation.** Real, E.; Aggarwal, A.; Huang, Y.; Le, Q.V. "Regularized Evolution for Image Classifier Architecture Search." *AAAI-19*, 2019. arXiv:1802.01548; DOI 10.1609/aaai.v33i01.33014780.

1.10 9 · 5G small-cell base-station siting — Multi-objective Cuckoo Search + K-means

- **(a) Problem.** Site 5G small cells across real neighbourhoods of **Belém, Brazil** to maximise coverage while minimising transmit power, driven by real OpenCellID field data rather than a synthetic grid.
- **(b) Algorithm + fit. MOCS-KM** (multi-objective Cuckoo Search + K-means). Cuckoo Search's heavy-tailed Lévy flights escape the many local optima of a noisy real-field cover-

age surface; K-means turns the metaheuristic's choices into concrete cell clusters/powers. (Closest published cousin of this lab's own Wi-Fi placement project.)
- **(c) Encoding.** Low-dimensional mixed vector $[k, P_t]$: k = number of cells (integer 1–100) seeding K-means over real coordinates; P_t = transmit power (continuous, 30–40 dBm).
- **(d) Fitness.** $f = 0.7\,N_{\text{outage}} + 0.3\,P_{\text{diff}}$ (percent users in outage vs power above the 30 dBm floor).
- **(e) Constraints.** $P_r \geq -90$ dBm defines connected-vs-outage, enforced in fitness; k, P_t box-bounded.
- **(f) Results. 0 %** outage @700 MHz (12 cells, 30 dBm), **2.43 %** @2.3 GHz, **2.29 %** @3.5 GHz; max coverage **97.4 %**; runtime 552–993 s `[verified]`.
- **(g) Citation.** Ferreira, F.H.; Barros, F.J.B.; de Alcântara Neto, M.C.; Cardoso, E.; Francês, C.R.L.; Araújo, J. "Hybrid computational and real data-based positioning of small cells in 5G networks." *PeerJ Computer Science*, 2023. DOI 10.7717/peerj-cs.1412.

1.11 10 · Edge-server placement — Hybrid GA + PSO (GP4ESP)

- **(a) Problem.** Decide which base/edge stations to equip with limited edge servers so that overall request **response time** is minimised in a heterogeneous edge-computing network — crucially accounting for server *computing delay*, which prior work ignored (over-estimating service quality).
- **(b) Algorithm + fit.** A **hybrid GA + PSO** that fuses PSO's swarm cognition into GA's evolutionary operators. ESP is a combinatorial assignment problem with a rugged objective; the hybrid balances GA's global recombination with PSO's directed convergence.
- **(c) Encoding.** Selection/assignment representation over candidate edge sites (which stations host servers) `[standard ESP encoding; the paper's exact vector layout not quoted here]`.
- **(d) Fitness.** Minimise overall response time = network transmission delay + edge-server computing/queueing delay (the paper's key modelling addition).
- **(e) Constraints.** Limited edge-resource budget (number/capacity of servers).
- **(f) Results. 18.2 %–20.7 %** shorter overall response time vs **eleven** up-to-date ESP algorithms, stable as problem scale varies `[verified via publisher record/PMC]`.
- **(g) Citation.** Han, et al. "GP4ESP: a hybrid genetic algorithm and particle swarm optimization algorithm for edge server placement." *PeerJ Computer Science*, 2024. DOI 10.7717/peerj-cs.2439 (PMC11623000).

1.12 11 · Container-terminal berth allocation — Genetic Algorithm

- **(a) Problem.** Discrete berth allocation at the real **TECON Rio Grande** container terminal (Brazil): assign arriving vessels to berths and fix berthing order/time to minimise service delay and time-window penalties.
- **(b) Algorithm + fit. GA** (heuristic crossover), benchmarked head-to-head vs Simulated Annealing. GA is a population-based global search over the combinatorial, multimodal berthing-sequence space; SA is the single-point foil.
- **(c) Encoding.** Permutation chromosome — the ordered vessel sequence per berth; primary case 62 vessels over 3 berths (also 35/55/62 vessels × 5/10 berths).
- **(d) Fitness.** Minimise $Z^* = w_0 \sum v_i(\cdot) + w_1 \sum [\max(0, a_i - T) + \max(0, T + t - b_i)] + w_2 \sum [\cdot]$ (service cost + vessel and berth time-window penalties), weights $[1, 10, 10]$.
- **(e) Constraints.** Penalty relaxation — hard time-window constraints become soft penalty terms, allowing temporarily infeasible intermediate solutions.
- **(f) Results.** Primary case (30 runs): GA avg **185,951** vs SA **194,074** (**~4.2 %** lower), GA **20′50″** vs SA **29′04″** (**~28 %** faster), GA std **2,202** vs SA **3,267**; GA ≥ SA across all six scenarios `[verified]`.
- **(g) Citation.** Pereira, E.D.; Coelho, A.S.; Longaray, A.A.; Machado, C.M.S.; Munhoz, P.R. "Metaheuristic Analysis Applied to the Berth Allocation Problem: Case Study in a Port Container Terminal." *Pesquisa Operacional*, 38(2), 2018. DOI 10.1590/0101-7438.2018.038.02.0247.

1.13 12 · Cold-chain refrigerated vehicle routing — Hybrid Tabu–GWO

- **(a) Problem.** Route refrigerated vehicles for a real fresh-food e-commerce operator in China under vehicle-capacity, driving-range, dual-time-window, and perishable-freshness constraints.
- **(b) Algorithm + fit. Hybrid Tabu Search – Grey Wolf Optimizer (TGWO)**. GWO converges fast but has weak local search; grafting Tabu neighbourhood moves (insert/swap/2-opt) sharpens exploitation and escapes local optima on the multi-cost cold-chain landscape.
- **(c) Encoding.** Each wolf is a TSP-style permutation from the depot, decoded into multi-vehicle routes by sequential assignment; case dimension $D = 19$ (1 depot + 18 customers).
- **(d) Fitness.** Minimise $Z = C_1 + C_2 + C_3 + C_4$ (fixed vehicle + transport incl. refrigeration energy + cargo spoilage + time-window penalty).
- **(e) Constraints.** Large-coefficient penalty ($M{=}10^8$) for time-window breach; a repair-like decomposition opens a new route when capacity/range/ freshness would be violated.
- **(f) Results.** Total cost TS **17,314.8**, GWO **7,754.9**, TGWO **7,112.5**; TGWO saves **50.34 % / 30.66 %** in travel distance vs TS / GWO `[verified; the authors' "143.44 %" cost-saving figure uses a non-standard denominator and is flagged, not endorsed]`.
- **(g) Citation.** Zhang, H.; Yan, J.; Wang, L. "Hybrid Tabu-Grey wolf optimizer algorithm for enhancing fresh cold-chain logistics distribution." *PLOS ONE*, 19(8):e0306166, 2024. DOI 10.1371/journal.pone.0306166.

1.14 Ranking — the top 3 most exceptional

#1 — AmoebaNet / Regularized Evolution (App 8). The clever trick is *age-based regularization*: culling the oldest genotype instead of the worst turns a plain GA into a search that resists overfitting to lucky early winners. It was the first evolved architecture to *beat* both human- and RL-designed networks at ImageNet scale (83.9 % top-1), overturning the assumption that reinforcement learning was the right NAS paradigm. The mechanism is one line of code, and it changed how AutoML search is done.

#2 — Wolaita Sodo reconfiguration + DG (App 5). The clever move is *constraint-as-encoding*: rather than penalising non-radial grids, the search space itself is restricted to spanning-tree/fundamental-loop combinations, so every particle is feasible by construction — eliminating an entire class of wasted evaluations. Coupled with a real deployed 114-transformer Ethiopian feeder and audited economics (72 % loss cut, ~6-year payback in real ETB), it is the entry with the hardest real-world grounding and the largest tangible impact.

#3 — Manjil variable-hub-height wind siting (App 6). The clever trick is the *encoding*: promoting hub height to a free decision variable per turbine lets a tall turbine physically climb out of an upstream turbine's wake — turning a 2-D layout problem into a 3-D one where the extra dimension buys energy that no same-height layout can. It is a rare case where enlarging the search space *reduces* effective difficulty, and it is validated on a real, historic wind farm (+10.75 % power, −9.42 % cost).

Demo Day

How to show all three assignments

15 July 2026

Contents

1 Demo day — how to show the three assignments

Everything below works **offline**. The OpenStreetMap graph and the station data are already cached as `.pkl` files in `1/data/` and `2/data/`, so nothing downloads while he is watching. The plots are already generated too. Nothing here depends on the network.

Do a **full dry run the night before**, on the machine and the screen you will actually use. The single most common way this goes wrong is a laptop that has not run the code since March.

1.1 The one-page map

	Assignment	Where	How to show it
1	Search — Dhaka Pathfinder (BFS/DFS/UCS/Greedy/A*/Weighted A*)	`1/`	Streamlit map UI
2	Constraint satisfaction — fuel-crisis allocator	`2/`	Streamlit UI
3	Swarm + decision making	`3/`	Terminal + plots
—	The written submission covering all three	`3/report/main.pdf`	Hand him this

---

1.2 Before you walk in

```
# once, to be sure everything still runs
cd 1 && ./run.sh test && cd ..
cd 2 && ./run.sh test && cd ..
cd 3 && ./run.sh test && cd ..
```

```
# build the report
cd 3 && ./run.sh report        # -> 3/report/main.pdf
```

Then **fill in your name and email** in `3/report/main.tex` (lines 8 and 9). They currently say `Your Full Name` and `your.email@example.com`. Rebuild after you edit.

1.3 Assignment 1 — Dhaka Pathfinder

```
cd 1
./run.sh ui          # opens a Streamlit map in the browser
```

Pick a source and destination on the map, change the traveller context (gender, time of day, weather, vehicle), and show that **the route changes**. That is the whole point of the assignment: the cost model is not distance, it is what it actually costs *that person* to make *that trip*.

The number to have ready if he asks how the algorithms compare:

> "A*, UCS and Weighted A* all return the same optimal cost, 212,375 — that is the check that our heuristic is really admissible. Greedy Best-First expands **169** nodes where A* expands **13,362**, so it is about eighty times cheaper, and it pays for it with a route that is 38% worse. DFS is a disaster on a graph with 28,000 nodes: its median path costs eleven times optimal and it fails outright on 42 of 350 runs."

If the UI is slow to start, fall back to the pre-generated figures in `1/results/plots/`.

The weakness to own before he finds it: every edge attribute (traffic, safety, flooding) is *synthetic* — generated deterministically from OSM tags, not measured. Say that yourself. The algorithms are real; the data is plausible, not surveyed.

1.4 Assignment 2 — Fuel-crisis CSP

```
cd 2
./run.sh ui              # Streamlit UI, the easiest thing to show
./run.sh experiments     # if he wants to see the sweep run live
```

The story is the combinatorial blow-up:

> "Basic backtracking expands 20 nodes at N=10 and **220,763** at N=50 — four orders of magnitude for a five-fold increase in size, and at N=50 it never terminates on its own, it just hits our time cap. Add forward checking with MRV and it expands **15,880**, a 93% reduction, and it is the only complete solver still finding full assignments at that size."

Two things you should raise *yourself*, because they look like mistakes if he finds them first:

- **Min-conflicts does zero backtracks at every problem size.** That is not a bug and it is not an achievement — it is definitional. Min-conflicts never builds a partial assignment, so it has nothing to unwind. It starts complete and broken, and repairs.
- **Plain MRV scores a *worse* objective than basic backtracking at N=50** (4,439 vs 3,542) despite expanding four times fewer nodes. Reason: it leaves 40% of vehicles unassigned against 29%, and the unassigned penalty (w=100) dominates the objective. So our J confounds "quality of the assignment" with "how much of the problem was attempted." That is a flaw in our metric design, and we say so in the report.

  Note: `2/results/REPORT.md` is **stale** and its tables disagree with the shipped CSVs. Use the CSVs (`2/results/experiments_summary.csv`) and the report PDF. Do not open REPORT.md in front of him.

1.5 Assignment 3 — the two parts

This is the assignment with two parts, so it needs the most planning. Budget **ten minutes**: one to frame it, four for Part A, five for Part B.

1.5.1 Set up before he sits down

```
cd 3
./run.sh rl > /tmp/partB.txt     # 4 minutes. Do this EARLY. You will read from the file.
```

Have these open and ready to switch between, in this order:

1. A terminal, sitting in `3/`, cleared.
2. `results/plots/spatial_swarm.png`
3. `results/plots/collective_behaviour.png`
4. `/tmp/partB.txt` (or just `results/rl_full_run.log`, which is the same output)
5. `results/plots/rl_policy_maps.png`
6. `results/plots/rl_model_mismatch.png`
7. `report/main.pdf`, as the fallback for any number you are asked for.

1.5.2 Open with the framing (30 seconds)

Say this first, because it makes the two parts look like one assignment instead of two homeworks stapled together:

> ”The assignment had two parts, so I built two problems, both set in the same University of Dhaka residential hall. Part A places the hall's Wi-Fi with a swarm. Part B runs the hall's water pump with an MDP. Same building, two different kinds of hard.”

1.5.3 PART A — the swarm (about 4 minutes)

Run it live. It only takes 30 seconds, and running something live is worth a lot.

```
./run.sh swarm
```

Then show `spatial_swarm.png` and explain the problem in one breath:

> ”40 rooms, concrete walls, and I can afford 3 access points. The red stars are where the swarm put them. The orange lines are the swarm's best-known solution moving over time. The reason this needs a swarm and not calculus is the walls: every time the signal crosses one it drops 8 dB in a step, so the objective is piecewise-constant. There is no gradient to descend.”

Then show `collective_behaviour.png`. This is the figure the whole part exists for. Do not rush it.

> ”Every point on this chart costs exactly the same amount of computation — 3,030 evaluations. One particle on its own scores 76.8. Thirty particles that I have *forbidden from communicating* — I set the social coefficient c2 to zero — score 87.3. Random search scores 87.4. They are the same.
>
> Thirty particles that share a single number, the global best, score 89.9.
>
> So the population is not what helps. The communication is. That is the ant-in-a-bowl point from your briefing, and this is the experiment that proves it.”

If he pushes on fairness, you have the answer ready: the budget is identical in every bar, and the code *asserts* it at runtime — `assert problem.n_fitness_calls == cfg.n_evals`.

Have in your pocket, if asked: PSO beats random by 2.47 points, Wilcoxon p = 3.05e-05 over 15 seeds; and PSO connects all 40 rooms where random search strands one.

1.5.4 PART B — the decision making (about 5 minutes)

Show the printed table from `/tmp/partB.txt` (the E2 block). Frame the problem:

> ”Same hall, water tank on the roof, electric pump. Load-shedding kills the power worst in the evening, exactly when everyone wants a shower. There is a diesel generator, but the hall buys only two hours of diesel a day. So you have to pump *before* the power goes, and you have to not waste the diesel.”

Then prove the problem is worth solving at all, which is the first thing he will wonder:

> ”The formally-correct greedy policy — that is value iteration with gamma set to zero, not a strawman — loses by 121%. Even the best tuned two-threshold rule, with the generator and both thresholds grid-searched, still loses 14.5%. So the lookahead is doing real work.”

Show `rl_policy_maps.png`. Point at the top-right panel:

> ”Hour of day across, tank level up. Red is 'burn diesel'. Watch the red region *grow* as you enter the shaded evening window — at 7am with a low tank it idles, at 7pm with the same tank it burns diesel, because it knows the outage will be long. That is the agent anticipating.
>
> Bottom row is Q-learning. Same shape, but speckled. That is what 'has not seen enough data yet' looks like.”

If he wants it sharper, the witness state is in the printout (E6):

> ”At 2pm with a 4/10 tank, pumping costs 0.97 *more* that hour than idling. The optimal policy pumps anyway, because the action is worth +4.69 overall. Over five of those points come purely from the future.”

Show `rl_model_mismatch.png`. This is the one he will remember, and it is a negative result about our own idea. Lead with that, do not bury it.

> ”Here is where the assignment actually asks 'why this algorithm in this setting'. I gave Value Iteration a *deliberately wrong* model — I told it outages are rarer than they really are, which is exactly what Dhaka's published load-shedding schedule does — and then scored its policy in the real hall.
>
> I expected Q-learning to win. It does not. A planner whose model is wrong by a factor of four still beats Q-learning after 720,000 steps. Then I added the baseline that could have destroyed my whole thesis: I estimated a model from Q-learning's *own samples* and planned on that. It beat Q-learning by a factor of ten on identical data.
>
> So the honest answer to 'which one, and when' is: if you can write the transition structure down at all, write it down and plan — a roughly-right model is worth more than 82 simulated years of experience. Model-free is what you reach for when you *cannot write the model down*, not when the model is merely wrong.”

1.5.5 The sentence to close on

> ”The two parts answer two different questions. Part A shows that the *communication* is what makes a population work. Part B shows that *having a model at all* beats not having one, even a bad model — which is not what I set out to prove.”

That is a student who ran the experiment that could have embarrassed him, and reported it. It is the best thing you have.

1.6 Two things to mention about the template

He said he would provide the report template, and he did. Two things worth telling him:

1. **It does not compile as shipped.** Line 19 of `main.tex` has a bare `\hline`, which is a LaTeX error outside a tabular (`Misplaced \noalign`) and stops the build dead. We replaced it with `\noindent\rule{\textwidth}{0.4pt}`.
2. **`references.bib` has duplicate keys** — `russell2021ai` and `bishop2006prml` are each defined twice, which makes BibTeX emit "Repeated entry".

Neither is a big deal, but every student in the class will hit both, and pointing them out is the kind of thing that reads well.

1.7 If something breaks in the room

- Streamlit will not start → show the pre-generated PNGs in `1/results/plots/` and `2/results/plots/`. They tell the same story.
- Anything in lab 3 is slow → all its figures are already in `3/results/plots/`. Nothing needs to run live.
- He asks for a number you do not have → it is in `3/report/main.pdf`. Open it and find it, out loud. Looking something up is not a failure; inventing a number is.